INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Ingeniería en Inteligencia Artificial

CÓMPUTO PARALELO

Fundamentos y Práctica del Cómputo Paralelo

Compendio de siete capítulos

Profesor:

José Alfredo Jiménez Benítez

Presentan:

Cruz Trejo Said Carbot
López Castro Magaly
Romero Rosales Lino Ehecatl
Ugalde Téllez Aarón

Grupo: 6BV1

Ciudad de México, a 11 de octubre de 2026

Índice general

Índice de figuras **v**

Índice de tablas **vii**

1. Introducción al cómputo paralelo **1**

1.1. Marco teórico 1

1.1.1. Contexto de la Serie de Fourier 1

1.1.2. Aplicaciones en la ciencia 1

1.1.3. Aplicaciones en la tecnología 3

1.1.4. Aplicaciones en la ingeniería 4

1.1.5. Aplicaciones en la inteligencia artificial 5

1.1.6. Aplicaciones en el cómputo paralelo 5

1.2. Soluciones de la Serie de Fourier de las funciones propuestas 7

1.2.1. Función propuesta por Said Carbot Cruz Trejo 7

1.2.2. Función propuesta por Magaly López Castro 10

1.2.3. Función propuesta por Lino Ehecatl Romero Rosales 13

1.2.4. Función propuesta por Aarón Ugalde Téllez 17

1.3. Conclusiones del capítulo 19

1.3.1. Conclusión — Said Carbot Cruz Trejo 19

1.3.2. Conclusión — Magaly López Castro 21

1.3.3. Conclusión — Lino Ehecatl Romero Rosales 22

1.3.4. Conclusión — Aarón Ugalde Téllez 23

2. Gráficas con Excel **24**

2.1. Gráficas originales de las funciones propuestas 24

2.1.1. Función propuesta por Said Carbot Cruz Trejo 24

2.1.2. Función propuesta por Magaly López Castro 24

2.1.3. Función propuesta por Lino Ehecatl Romero Rosales 25

2.1.4. Función propuesta por Aarón Ugalde Téllez 26

2.2. Gráficas obtenidas con la serie de Fourier 27
2.2.1. Aproximación de Said Carbot Cruz Trejo 27
2.2.2. Aproximación de Magaly López Castro 28
2.2.3. Aproximación de Lino Ehecatl Romero Rosales 30
2.2.4. Aproximación de Aarón Ugalde Téllez 31
2.3. Comparaciones entre las gráficas obtenidas 34
2.3.1. Comparación de Said Carbot Cruz Trejo 34
2.3.2. Comparación de Magaly López Castro 35
2.3.3. Comparación de Lino Ehecatl Romero Rosales 36
2.3.4. Comparación de Aarón Ugalde Téllez 36
2.4. Conclusiones del capítulo 38
2.4.1. Conclusión — Said Carbot Cruz Trejo 38
2.4.2. Conclusión — Magaly López Castro 39
2.4.3. Conclusión — Lino Ehecatl Romero Rosales 39
2.4.4. Conclusión — Aarón Ugalde Téllez 40

3. Modelos de programación y métricas de desempeño 41
3.1. Marco teórico 41
3.1.1. Procesos hijos con la llamada al sistema `fork()` en lenguaje C 41
3.1.2. Memoria compartida en lenguaje C 42
3.1.3. Semáforos y sincronización en lenguaje C 42
3.2. Programación con procesos en lenguaje C 44
3.2.1. Función propuesta por Aarón Ugalde Téllez 44
3.2.2. Función propuesta por Lino Ehecatl Romero Rosales 52
3.2.3. Función propuesta por Magaly Lopez Castro 55
3.2.4. Función propuesta por Said Carbot Cruz Trejo 59
3.3. Análisis de la implementación con procesos 65
3.3.1. Implementación de Aarón Ugalde Téllez 65
3.3.2. Implementación de Lino Ehecatl Romero Rosales 67
3.3.3. Implementación de Magaly Lopez Castro 68
3.3.4. Implementación de Said Carbot Cruz Trejo 69
3.4. Conclusiones del capítulo 70

3.4.1. Conclusión — Aarón Ugalde Téllez 70
3.4.2. Conclusión — Lino Ehecatl Romero Rosales 71
3.4.3. Conclusión — Magaly Lopez Castro 72
3.4.4. Conclusión — Said Carbot Cruz Trejo 73
3.5. Anexos 74
3.5.1. Fase 3: código de `fourier_aaron.c` 74
3.5.2. Fase 3: código de `fourier_f3_lino.c` 78
3.5.3. Fase 3: código de `fourier_x5.c` 80

4. Programación con Hilos **84**
4.1. Marco teórico 84
4.2. Programación con hilos en lenguaje C y en la plataforma GNU-Linux 84
4.2.1. Fragmentos de códigos explicados de la programación con hilos 85
4.2.2. Compilación 85
4.2.3. Ejecución de los programas 86
4.2.4. Gráficas, Figuras y su Explicación 86
4.2.5. Comparación de resultados con fases anteriores 87
4.3. Análisis de la implementación con hilos 87

5. Programación con memoria distribuida: MPI **93**
5.1. Introducción 93
5.2. Desarrollo 93
5.3. Conclusiones del capítulo 93

6. Cómputo en GPU: CUDA **94**
6.1. Introducción 94
6.2. Desarrollo 94
6.3. Conclusiones del capítulo 94

7. Diseño, análisis y caso de estudio integrador **95**
7.1. Introducción 95
7.2. Desarrollo 95
7.3. Conclusiones del capítulo 95

Referencias **96**

Índice de figuras

2.1. Gráfica de la función original $f(x) = x^5$ en el intervalo $[-\pi, \pi]$. 24

2.2. Comportamiento de la función original $f(x) = 5x^3 - 3x^2 + 1$. 25

2.3. Comportamiento de la función original $f(x) = -\frac{1}{4}x^4 + 3$. 25

2.4. Comportamiento de la función original $f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}$. 26

2.5. Coeficientes b_n obtenidos para las diez primeras iteraciones de $f(x) = x^5$. 27

2.6. Matriz de los diez primeros términos de la Serie de Fourier de $f(x) = x^5$ y su sumatoria. . 27

2.7. Aproximación de $f(x) = x^5$ mediante la suma de los diez primeros términos de la Serie de Fourier. 28

2.8. Matriz de cálculos con los términos obtenidos para las diez iteraciones ($n = 1$ a $n = 10$). . 28

2.9. Columna que representa la sumatoria total de los términos de la serie de Fourier, $F(x)$. . . 29

2.10. Aproximación $F(x)$ de la función $f(x) = 5x^3 - 3x^2 + 1$ obtenida con diez iteraciones de la Serie de Fourier. 30

2.11. Coeficientes a_n y b_n obtenidos para las diez primeras iteraciones de $f(x) = -\frac{1}{4}x^4 + 3$. . . . 30

2.12. Extracto de la matriz iterativa de los diez primeros armónicos, evaluada en $x \approx -\pi$. 31

2.13. Aproximación de $f(x) = -\frac{1}{4}x^4 + 3$ mediante la Serie de Fourier con diez armónicos. . . . 31

2.14. Extracto de la matriz de los diez armónicos, evaluada en $x \approx -\pi$, precedida por el término constante $a_0/2$. 32

2.15. Columna de sumatoria $F(x)$ junto al término constante $a_0/2$, repetido en cada renglón. . . 33

2.16. Aproximación $F(x)$ de $f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}$ mediante la Serie de Fourier con diez armónicos. 34

2.17. Comparación entre la gráfica original $f(x) = x^5$ y la aproximación por Serie de Fourier con diez iteraciones. 35

2.18. Superposición de la función original $f(x)$ frente a la aproximación de Fourier $F(x)$ con diez iteraciones. 35

2.19. Comparativa de ajuste: función original frente a la Serie de Fourier. 36

2.20. Comparativa de ajuste: función original $f(x)$ frente a la Serie de Fourier $F(x)$ con diez armónicos. 37

3.1. Compilación y ejecución del programa con 100 procesos hijos en la terminal. 49

3.2. Gráfica de $F_{100}(x)$ calculada en C con 100 procesos hijos, en los 64 nodos de $x = -3.1416$ a $x = 3.1584$. 50

3.3. Evidencia de compilación limpia, sin errores, usando `gcc`, y visualización del archivo binario. 53

3.4. Evidencia de la ejecución del programa y revisión del archivo CSV generado. 53

3.5. Reconstrucción analítica de la función $f(x) = -\frac{1}{4}x^4 + 3$ generada previamente en Excel. . 54

3.6. Reconstrucción de la función $f(x) = -\frac{1}{4}x^4 + 3$ graficada con los datos obtenidos mediante los 100 procesos en C. 54

3.7. Ejecución concurrente de los 100 procesos y exportación de datos. 56

3.8. Resultados en la matriz del archivo generado por el programa en C. 57

3.9. Gráfica generada en Excel a partir de los datos exportados por el programa en C, que muestra la magnitud de los términos hasta 100 iteraciones. 58

3.10. Gráfica con Excel generada en fases anteriores. 59

3.11. Creación del archivo `fourier_x5.c`. 61

3.12. Apertura del archivo desde la terminal para redactar el código (`fourier_x5.c`). 61

3.13. Apertura del archivo en TextEdit y codificación. 62

3.14. Compilación del programa con GCC (`gcc fourier_x5.c -o fourier_x5`). Como se observa, la compilación es limpia y no presenta errores ni advertencias. 62

3.15. Ejecución del programa (`./fourier`). Se muestra el mensaje que indica el inicio de los 100 procesos y la correcta generación del archivo CSV. 63

3.16. Salida en Excel. 63

3.17. Resultados de la fase 2 (capítulo 2): gráfica de la serie de Fourier calculada en Excel con un número reducido de términos ($N = 10$). 64

3.18. Resultados de la fase 3 (capítulo 3): gráfica de la serie de Fourier procesada en lenguaje C con 100 procesos concurrentes. 64

3.19. Uso del comando `lscpu` para ver la arquitectura del procesador. 68

3.20. Uso del comando `free -h` para verificar la RAM disponible. 68

4.1. Captura de pantalla de la compilación limpia del programa de hilos en la terminal de GNU-Linux. 86

4.2. Captura de pantalla de la ejecución de los programas y confirmación de la exportación de datos. 86

4.3. Gráficas de la serie de Fourier generadas en Excel a partir de los datos procesados por los hilos. 86

Índice de tablas

3.1. Función original y aproximaciones con 10 armónicos (Excel) y 100 armónicos (C, 100 procesos), en diez de los 64 nodos. 51

3.2. Error cuadrático medio respecto de $f(x)$ según la región de nodos considerada. 51

3.3. Comparativa de coeficientes: cálculo teórico frente a los resultados del programa en C. . . 53

3.4. Características de la implementación con procesos. 65

3.5. Especificaciones del equipo de cómputo empleado. 66

3.6. Detalles técnicos del software. 67

3.7. Hardware donde se probaron los procesos. 67

3.8. Características de la implementación de la fase 3 y equipo de cómputo. 68

3.9. Características de la implementación de la fase 3 y equipo de cómputo. 69

4.1. Características técnicas del equipo de cómputo empleado para la fase 4. 88

Capítulo 1. Introducción al cómputo paralelo

1.1 Marco teórico

1.1.1 Contexto de la Serie de Fourier

La serie de Fourier fue introducida por el matemático y físico francés Jean-Baptiste Joseph Fourier en 1822, en su obra fundamental *Théorie analytique de la chaleur* (Teoría analítica del calor) [1]. El problema central que motivó a Fourier no era puramente matemático, sino físico: buscaba modelar cómo el calor fluye y se distribuye a lo largo del tiempo en un material sólido, lo que lo llevó a formular la famosa ecuación del calor [2].

Para resolver esta ecuación diferencial, Fourier propuso una idea que rompió con los paradigmas de su época. Afirmó que cualquier función periódica, incluso aquellas que presentaban saltos o discontinuidades, podía ser representada analíticamente como una suma infinita de funciones trigonométricas simples, específicamente senos y cosenos [1]. Hasta ese momento, las herramientas convencionales como las series de Taylor estaban estrictamente limitadas a funciones continuas y derivables [3].

Esta audaz afirmación generó un fuerte debate en la comunidad científica. Matemáticos consagrados de la Academia Francesa, como Joseph-Louis Lagrange y Pierre-Simon Laplace, rechazaron inicialmente su trabajo. Ellos argumentaban que una suma de funciones continuas y suaves (como los senos) jamás podría modelar esquinas, picos o discontinuidades abruptas en una gráfica [2]. Aunque Fourier no pudo ofrecer una demostración matemática rigurosa en ese momento para calmar a sus críticos, su intuición era correcta y los resultados prácticos eran innegables.

No fue hasta 1829 cuando el matemático alemán Peter Gustav Lejeune Dirichlet logró establecer las condiciones formales (conocidas hoy como las condiciones de Dirichlet) que garantizan la convergencia de estas series [3]. En retrospectiva, el trabajo de Fourier no solo resolvió un problema de termodinámica, sino que obligó a los matemáticos del siglo XIX a redefinir conceptos fundamentales, como el significado mismo de lo que es una ”función” o una ”integral”, sentando las bases del análisis matemático moderno.

1.1.2 Aplicaciones en la ciencia

Imagenología médica: resonancia magnética

En la resonancia magnética (RM), los datos crudos que capta el escáner no forman directamente una imagen: se almacenan en una matriz conocida como “espacio-k”, que corresponde matemáticamente al dominio de frecuencias de la imagen final [4], [5]. Solo al aplicar una transformada de Fourier bidimensional a esos datos se reconstruye la imagen anatómica que observa el radiólogo [4]. El centro del espacio-k concentra la información sobre el contraste y la forma general de la imagen, mientras que sus regiones periféricas contienen los detalles finos y los bordes de las estructuras [6]. Comprender esta relación permite a los técnicos optimizar la resolución espacial, la resolución temporal y la calidad final de las imágenes clínicas [4].

Espectroscopía química y análisis de materiales

La espectroscopía infrarroja por transformada de Fourier (FTIR) es una de las técnicas analíticas más usadas en química, farmacia, ciencia de materiales y control de calidad industrial. En lugar de analizar una sola longitud de onda a la vez, el instrumento hace pasar un haz con múltiples frecuencias simultáneas a través de un interferómetro de Michelson y de la muestra; luego, un procesamiento de Fourier convierte esos datos brutos en el espectro de absorción o emisión característico del material [7], [8]. Esta técnica permite identificar grupos funcionales y familias químicas, detectar impurezas en fármacos o alimentos y diferenciar materiales de composición similar, como distintos tipos de plásticos [9], [10].

Acústica y música

Todo sonido periódico —la nota de un instrumento musical, la voz humana— puede describirse como la suma de una frecuencia fundamental y sus armónicos, es decir, como una serie de Fourier. Esta descomposición revela el espectro armónico del sonido, que es el principal responsable del timbre distintivo de cada instrumento o voz [11], [12]. La síntesis aditiva, una técnica habitual en producción musical y diseño de sonido, aplica el principio inverso: construye timbres complejos combinando ondas sinusoidales simples con las amplitudes y fases adecuadas [13], [14]. Este marco también ha influido en el desarrollo de la música por computadora, desde la imitación de instrumentos hasta la generación de nuevas texturas sonoras [15].

Sismología e ingeniería estructural

En ingeniería civil y sismología, la transformada de Fourier permite extraer las frecuencias naturales de vibración de un edificio, un puente o el propio terreno a partir de registros de aceleración. Analizar el espectro de Fourier de un acelerograma sísmico revela las frecuencias dominantes de un movimiento de tierra y los modos de vibración resonantes de una estructura [16], [17]. Esta información es clave para evaluar el riesgo sísmico y para el monitoreo continuo de la salud estructural de edificaciones, donde variantes como la Transformada de Fourier de Tiempo Corto (STFT) permiten detectar cambios repentinos —como la propagación de una grieta— en la respuesta dinámica de una estructura [18].

Mecánica cuántica

En la física cuántica, la transformada de Fourier no es solo una herramienta de cálculo: describe una relación física fundamental. La función de onda de una partícula en el espacio de posiciones y su función de onda en el espacio de momentos están relacionadas exactamente por una transformada de Fourier [19], [20]. Esta relación matemática es la razón formal detrás del principio de incertidumbre de Heisenberg: cuanto más localizada está una partícula en el espacio de posición, más deslocalizada resulta su distribución de momento, y viceversa [20], [21].

Astrofísica: detección de ondas gravitacionales

En la astronomía observacional moderna, el análisis de Fourier es indispensable para extraer señales físicas débiles de datos ruidosos. Un ejemplo destacado es la detección de ondas gravitacionales por observatorios como LIGO, donde las señales registradas por los interferómetros se analizan en el dominio de la

frecuencia mediante transformadas de Fourier para identificar y caracterizar el paso de una onda gravitacional a través del detector [22].

1.1.3 Aplicaciones en la tecnología

Procesamiento de señales y telecomunicaciones

La aplicación más extendida de las series y transformadas de Fourier está en el procesamiento digital de señales. Al convertir una señal del dominio del tiempo al dominio de la frecuencia, es posible filtrar ruido, modular señales, comprimirlas y adaptarlas a las condiciones del canal de transmisión [23], [24].

En telecomunicaciones, la transformada de Fourier es la base de la multiplexación, la técnica que permite combinar varias señales en un solo medio de transmisión y luego separarlas correctamente en el destino mediante la transformada inversa [24]. La llegada del algoritmo de la Transformada Rápida de Fourier (FFT), publicado por Cooley y Tukey en 1965, redujo drásticamente el costo computacional de este análisis y permitió su uso masivo en sistemas de comunicación modernos, radares y equipos de prueba electrónica [25], [26]. El propio IEEE reconoce esta relevancia a través del "IEEE Fourier Award for Signal Processing", otorgado a contribuciones destacadas en el campo del procesamiento de señales [27].

Compresión y procesamiento de imágenes digitales

El formato de compresión de imágenes JPEG, uno de los más usados en el mundo, se apoya en una variante de las series de Fourier llamada Transformada Discreta del Coseno (DCT). El algoritmo divide la imagen en bloques de 8×8 píxeles y expresa el contenido de cada bloque como una suma de funciones coseno de distintas frecuencias [28], [29]. Las frecuencias bajas concentran la información sobre la forma general de la imagen, mientras que las frecuencias altas describen los detalles finos y los bordes; al eliminar o simplificar estas últimas mediante cuantización, es posible reducir el tamaño del archivo con una pérdida de calidad casi imperceptible para el ojo humano [29]. La misma familia de transformadas está detrás de otros estándares multimedia ampliamente usados, como MPEG y las normas de compresión de video de la serie H [30].

Al revisar estas ocho áreas —procesamiento de señales, compresión de imágenes, imagenología médica, espectroscopía, acústica, sismología, mecánica cuántica y astrofísica— resulta evidente que las series y transformadas de Fourier no son un recurso matemático más entre muchos, sino un lenguaje común que atraviesa de manera transversal a la ciencia y la tecnología contemporáneas. Su idea central —que un fenómeno complejo puede descomponerse en componentes de frecuencia simples, analizarse por separado y luego reconstruirse sin pérdida de información esencial— resulta tan útil para comprimir una fotografía como para reconstruir una imagen médica, afinar un instrumento musical, evaluar el riesgo sísmico de un edificio, describir el comportamiento cuántico de una partícula subatómica o detectar el paso de una onda gravitacional a miles de millones de años luz de distancia.

Lo que hace especialmente notable a esta herramienta es su capacidad de unificación. En telecomunicaciones, el análisis de frecuencias permite que múltiples señales compartan un mismo canal físico sin interferirse, sosteniendo la infraestructura de internet, la telefonía móvil y la transmisión de radio y televisión. En el procesamiento de imágenes, el mismo principio matemático que Fourier usó para modelar la

difusión del calor en una lámina metálica es hoy la base del formato JPEG y de los estándares de video más usados del mundo, permitiendo que miles de millones de fotografías y videos se almacenen y transmitan de forma eficiente cada día. En medicina, la reconstrucción por Fourier del espacio-k convierte señales de radiofrecuencia en imágenes anatómicas de altísima resolución sin necesidad de radiación ionizante, siendo hoy una herramienta diagnóstica indispensable. En química analítica, la espectroscopía FTIR aplica el mismo principio para identificar sustancias y detectar impurezas con una precisión que sería inalcanzable con métodos dispersivos tradicionales.

En la física fundamental, el papel de Fourier es aún más profundo: no se trata solo de una técnica de cálculo conveniente, sino de una relación que refleja la naturaleza misma de la realidad cuántica, donde posición y momento son variables conjugadas ligadas precisamente por una transformada de Fourier, lo que explica formalmente el principio de incertidumbre de Heisenberg. Y en la frontera de la astrofísica observacional, el análisis espectral permite a instrumentos como LIGO extraer señales gravitacionales extraordinariamente débiles del ruido de fondo, abriendo una ventana completamente nueva para observar el universo.

Doscientos años después de que Joseph Fourier formulara su método para resolver un problema de conducción térmica, la vigencia de su idea en campos que él jamás podría haber imaginado —la resonancia magnética, las telecomunicaciones digitales, la detección de ondas gravitacionales— es quizás la mejor demostración de que las matemáticas fundamentales, cuando capturan una estructura verdaderamente universal de la naturaleza, trascienden por completo el problema original que las inspiró. Las series de Fourier no solo resolvieron una ecuación diferencial del siglo XIX: proporcionaron una manera nueva de pensar cualquier fenómeno que varíe en el tiempo o en el espacio, y esa forma de pensar sigue, hoy, en el centro mismo de la innovación científica y tecnológica.

1.1.4 Aplicaciones en la ingeniería

El análisis de Fourier representa uno de los pilares matemáticos más fundamentales en la ingeniería moderna, actuando como el puente principal entre el dominio del tiempo y el dominio de la frecuencia [31]. Su aplicación se extiende a prácticamente todas las ramas de la ingeniería aplicada, permitiendo descomponer fenómenos físicos complejos en sumas de ondas senoidales y cosenoidales más simples de analizar.

Particularmente en el procesamiento digital de señales, las series y transformadas de Fourier son herramientas indispensables [32]. Los ingenieros utilizan estas fórmulas para diseñar filtros digitales que limpian el ruido de las transmisiones, analizar la respuesta en frecuencia de sistemas de control, y comprimir datos de audio y video. Al convertir una señal discreta a su espectro de frecuencias mediante operaciones matriciales complejas, es posible identificar y aislar componentes no deseados antes de reconstruir la señal original [32].

Además, en áreas como la ingeniería mecánica y civil, el análisis de Fourier es vital para el estudio de vibraciones mecánicas [33]. Permite predecir cómo responderán las estructuras (como puentes o motores) a fuerzas externas periódicas, identificando frecuencias de resonancia críticas que podrían causar fallas catastróficas. De igual manera, en la ingeniería de telecomunicaciones, es la base matemática para modular señales portadoras, calcular el ancho de banda necesario para un canal de transmisión y optimizar el flujo de información a través de redes inalámbricas [31].

1.1.5 Aplicaciones en la inteligencia artificial

En la intersección del análisis matemático y la inteligencia artificial, las series de Fourier están impulsando el desarrollo de modelos de aprendizaje más rápidos y eficientes. Desde el punto de vista del hardware y la aceleración física, la computación fotónica está utilizando luz láser a través de cristales de silicio para calcular transformadas de Fourier de manera instantánea [34]. En lugar de depender de interruptores eléctricos tradicionales, este método aprovecha las propiedades de las ondas de luz, lo cual resulta extremadamente útil en inteligencia artificial para procesar imágenes masivas. Esto permite reducir drásticamente la carga computacional de operaciones densas, bajando su complejidad de un nivel $\mathcal{O}(N^4)$ a uno mucho más manejable de $\mathcal{O}(N^2)$.

Desde la perspectiva algorítmica, el análisis de Fourier se ha integrado profundamente en paradigmas avanzados de la inteligencia artificial, como el aprendizaje por refuerzo [35]. En entornos donde un agente debe tomar decisiones en espacios de estado continuos y altamente dimensionales, las bases de Fourier se utilizan para aproximar funciones de valor con gran precisión [35]. Al representar el entorno a través de sus componentes de frecuencia, los agentes pueden generalizar sus experiencias pasadas de forma más eficiente, acelerando la convergencia hacia políticas óptimas.

Asimismo, en el campo de la optimización basada en inteligencia de enjambres, el análisis espectral de Fourier permite a los investigadores evaluar la dinámica poblacional de los algoritmos [36]. Al tratar las trayectorias de búsqueda de los agentes como señales en el tiempo, las series de Fourier ayudan a entender el equilibrio entre la exploración de nuevas soluciones y la explotación de las mejores áreas encontradas, permitiendo ajustar los parámetros matemáticos para evitar mínimos locales [36].

1.1.6 Aplicaciones en el cómputo paralelo

Usar series de Fourier en el cómputo paralelo es un tema muy importante hoy en día en la ingeniería de software. Para entender cómo las computadoras procesan grandes cantidades de datos usando estas matemáticas, sirve mucho revisar de dónde viene esta tecnología, qué estamos haciendo ahora y qué se espera para el futuro. Este recorrido muestra cómo han evolucionado las máquinas para hacer cálculos trigonométricos mucho más rápido.

Pasado

Los fundamentos teóricos pueden ser rastreados hasta 1805 con Carl Friedrich Gauss, cuya intención era calcular las órbitas y posiciones de cuerpos celestes como los asteroides Pallas y Juno. El método que ideó consistía en dividir los datos a la mitad repetidamente para evitar repetir multiplicaciones trigonométricas innecesarias. Sin embargo, llevar estas ideas a las computadoras electrónicas modernas no pasó sino hasta 1965, gracias a los trabajos de James Cooley y John Tukey. El algoritmo de ambos surgió a petición del gobierno de Estados Unidos, que necesitaba analizar vibraciones en la tierra para detectar pruebas de bombas atómicas de otros países. Como las señales de los sismos se registran en función del tiempo, era necesario pasarlas a frecuencias; con esto, lograron reducir la cantidad de operaciones matemáticas de un nivel muy pesado ($\mathcal{O}(N^2)$) a uno mucho más eficiente ($\mathcal{O}(N \log N)$), funcionando mediante divisiones de información y guardando resultados temporales en la memoria para no recalcular todo desde cero [37].

Cuando se empezaron a usar redes de computadoras (clústeres) para procesar estos datos, el problema principal dejó de ser la matemática y pasó a ser el tráfico de información entre las máquinas. Para procesar matrices gigantes de datos en tres dimensiones, la técnica tradicional consistía en cortar el cubo de datos en rebanadas planas y mandarle una parte a cada servidor. Para que las computadoras se pusieran de acuerdo, usaban un comando llamado `MPI_Alltoall`. El problema ocurre cuando manejas millones de puntos: al cortar la matriz en una sola dirección, el comando inunda los cables de la red con una cantidad exagerada de mensajes, saturando los puertos de red y congelando el sistema. Por ejemplo, en simulaciones de fluidos con una cuadrícula de $N \times N \times N$, si cortabas la matriz en 512 partes, el software ya no te dejaba usar más de 512 procesadores al mismo tiempo, frenando por completo las investigaciones en los laboratorios [38].

Presente

Para solucionar este problema de los límites en las redes, la industria empezó a usar tarjetas gráficas (GPUs), que tienen miles de núcleos de procesamiento. En lugar de hacer cortes en una sola dirección, los programadores cambiaron a una técnica llamada división en "lápices", la cual corta la matriz tridimensional en dos direcciones formando cilindros delgados de datos. Esto permite usar miles de computadoras a la vez sin que la red colapse en simulaciones de turbulencia de aire con millones de puntos. Para facilitarle la vida a los programadores, se crearon bibliotecas especializadas como heFFTe, la cual ayuda a resolver estos cálculos en supercomputadoras que buscan alcanzar la "exaescala" (un trillón de operaciones por segundo) reorganizando la memoria de la computadora para que las GPUs lean los datos de forma continua. Además, se usan comandos de red asíncronos para enviar datos mientras los procesadores siguen calculando, aunque el límite actual sigue siendo el tiempo que toma mover terabytes de información a través de los cables de fibra óptica y los conmutadores del centro de datos [39].

Futuro

Mirando hacia los próximos diez años, los laboratorios de investigación están proponiendo dejar atrás los electrones y los cables de cobre tradicionales para usar luz láser (computación fotónica). En lugar de transistores eléctricos, los nuevos diseños envían pulsos de luz a través de guías hechas en cristales de silicio, donde las ondas chocan y se combinan entre sí. Gracias a este fenómeno físico, la luz realiza la transformada de Fourier de manera instantánea mientras viaja por el aire dentro del cristal, sin encender ni apagar interruptores eléctricos, algo muy útil en inteligencia artificial para procesar imágenes rápidamente a centímetros de distancia y bajar el costo de las operaciones matemáticas de un nivel pesado ($\mathcal{O}(N^4)$) a uno mucho más rápido ($\mathcal{O}(N^2)$) [34].

Otra tecnología que viene fuerte son los chips gigantes a escala de oblea. Empresas como Cerebras fabrican placas de silicio completas del tamaño de un plato de comida sin cortar los chips en pedazos pequeños, lo que permite poner la memoria a una distancia microscópica de los procesadores. En estas máquinas, transformar cubos gigantes de datos toma solo fracciones de milisegundo porque los mensajes viajan por pistas internas de metal dentro del mismo plato de silicio en lugar de salir a una red externa. En resumen, tanto la luz como los chips gigantes prometen eliminar los retrasos causados por los cables de red, haciendo que el procesamiento de estas operaciones dependa directamente de las nuevas propiedades físicas de los materiales [40].

1.2 Soluciones de la Serie de Fourier de las funciones propuestas

1.2.1 Función propuesta por Said Carbot Cruz Trejo

Said Carbal Cruz Trejo | 09 | 03 | 26

- Fourier Serie Por x^5 in $[-\pi, \pi]$

① Let sea $L=\pi$

② Como $x^5 \Rightarrow$ impar $\Rightarrow$ Cosenos $\Rightarrow$ 0 ∴ $a_0=0$ ∧ $a_n=0$

③ Para $b_n \Rightarrow$ ∴ $L=\pi$ en $\frac{n\pi x}{L} \Rightarrow \frac{n\pi x}{\pi} = nx$ [1]

3.1) Así que $b_n = \frac{2}{L}\int_0^L f(x)\,\mathrm{Sen}(nx)\,dx$ [2]

Sostituyendo $\Rightarrow b_n = \frac{2}{\pi}\int_0^{\pi} x^5\,\mathrm{Sen}(nx)\,dx$ [2.1]

④ Aplico tabular $\Rightarrow$

	$f(x)$	$\int \mathrm{Sen}(nx)$	Resultado → [3]
1	$+\ x^5$		
2	$-\ 5x^4$	$-\frac{1}{n}\cos(nx) \Rightarrow -\frac{x^5}{n}\cos(nx)$	+
3	$+\ 20x^3$	$-\frac{1}{n^2}\mathrm{Sen}(nx) \Rightarrow \frac{5x^4}{n^2}\mathrm{Sen}(nx)$	+
4	$-\ 60x^2$	$-\frac{1}{n^3}\cos(nx) \Rightarrow +\frac{20x^3}{n^3}\cos(nx)$	+
5	$+\ 120x$	$+\frac{1}{n^4}\mathrm{Sen}(nx) \Rightarrow -\frac{60x^2}{n^4}\mathrm{Sen}(nx)$	+
6	$-\ 120$	$-\frac{1}{n^5}\cos(nx) \Rightarrow -\frac{120x}{n^5}\cos(nx)$	+
7	$+\ 0$	$-\frac{1}{n^6}\mathrm{Sen}(nx) \Rightarrow +\frac{120}{n^6}\mathrm{Sen}(nx)$	

⑤ Evaluando [3]

- Como todo lo que tengo $x=0 \Rightarrow$ en $x=0$ todo se hace 0
- en $L=\pi \Rightarrow$ todo en $\mathrm{Sen}\,0=0 \Rightarrow$ solo $\cos(n\pi)$ ∧ como

$\cos(n\pi)=(-1)^n \Rightarrow I(\pi) = -\frac{\pi^5}{n}(-1)^n + \frac{20\pi^3}{n^3}(-1)^n - \frac{120\pi}{n^5}(-1)^n$ [4]

JEAN BOOK

⑥ Multiplicar por conveniente $\frac{2}{\pi}$ en [2.1] usando [4]

$$\pi\, b_n = \frac{2}{\pi}\left[-\frac{\pi^5}{n}(-1)^n + \frac{20\pi^3}{n^3}(-1)^n - \frac{120\pi}{n^5}(-1)^n\right]$$

Como se puede sacar el signo negativo en $(-1)^n = (-1)^{n+1}$

$$b_n = 2(-1)^{n+1}\left[\frac{\pi^4}{n} - \frac{20\pi^2}{n^3} + \frac{120}{n^5}\right] \quad [5]$$

⑦ Sustituyendo [5] en la formula general de S.F.T

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty}\left[a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \operatorname{Sen}\left(\frac{n\pi x}{L}\right)\right] \quad [6]$$

Pero como $a_0 = a_n = 0 = 0$ en [6] solo queda quedado [5]

$$f(x) = \sum_{n=1}^{\infty} 2(-1)^{n+1}\left[\frac{\pi^4}{n} - \frac{20\pi^4}{n^3} + \frac{120}{n^5}\right]\operatorname{Sen}(nx) \quad [7]$$

⑧ Como no pide coeficientes se deja general como en [7] ahí están todos los coeficientes de la S.F.

1.2.2 Función propuesta por Magaly López Castro

$$f(x) = 5x^3 - 3x^2 + 1$$

$$a_0 = \frac{1}{\pi}\int_{-\pi}^{\pi} 5x^3 - 3x^2 + 1\,dx$$

$$= \frac{1}{\pi}\left[\frac{5x^4}{4} - x^3 + 1x\Big|_{-\pi}^{\pi}\right]$$

$$= \frac{1}{\pi}\left[\frac{5(\pi)^4}{4} - (\pi)^3 + (\pi) - \left[\frac{5(-\pi)^4}{4} - (-\pi)^3 + (-\pi)\right]\right]$$

$$= \frac{1}{\pi}\left[\frac{5\pi^4}{4} - \pi^3 + \pi\right] - \left[\frac{5\pi^4}{4} + \pi^3 - \pi\right] = -2\pi^2 + 2$$

$$\frac{a_0}{2} = \frac{-2\pi^2 + 2}{2} = -\pi^2 + 1 = 1 - \pi^2$$

$$b_n = \frac{1}{\pi}\int_{-\pi}^{\pi} (5x^3 - 3x^2 + 1)\sin(nx)\,dx$$

checcamos paridad

$(-3x^2 + 1)\sin(nx) \rightarrow$ impar $= 0$

$\therefore$

$$b_n = \frac{1}{\pi}\int_{-\pi}^{\pi} 5x^3 \sin(nx)\,dx$$

$5x^3$	$\sin(nx)$	
$15x^2$	$-\frac{\cos(nx)}{n}$	$+$
$30x$	$-\frac{\sin(nx)}{n^2}$	$-$
30	$\frac{\cos(nx)}{n^3}$	$+$
0	$\frac{\sin(nx)}{n^4}$	$-$

$$= \frac{1}{\pi}\left[-\frac{5x^3}{n}\cos(nx) + \frac{15x^2}{n^2}\overbrace{\sin(nx)}^{0} + \frac{30x}{n^3}\cos(nx) - \frac{30}{n^4}\overbrace{\sin(nx)}^{0}\Big|_{-\pi}^{\pi}\right]$$

$$= \frac{1}{\pi}\left[-\frac{5x^3}{n}\cos(nx) + \frac{30x}{n^3}\cos(nx)\Big|_{-\pi}^{\pi}\right]$$

$$= \frac{1}{\pi}\left[\left(-\frac{5\pi^3}{n} + \frac{30\pi}{n^3}\right)(-1)^n\right] - \left[\left(\frac{5\pi^3}{n} - \frac{30\pi}{n^3}\right)(-1)^n\right] =$$

$$= \frac{1}{\pi}\left[2\left(-\frac{5\pi^3}{n} + \frac{30\pi}{n^3}\right)(-1)^n\right] = \left[\left(-\frac{10\pi^3}{n} + \frac{60\pi}{n^3}\right)(-1)^n\right]\frac{1}{\pi}$$

$$= \frac{1}{\pi}\left(-\frac{10\pi^3}{n} + \frac{60\pi}{n^3}\right)(-1^n) = \left(-\frac{10\pi^2}{n} + \frac{60}{n^3}\right)(-1)^n = \frac{10(-1)^{n+1}(\pi^2 n^2 - 6)}{n^3}$$

$$a_n = \frac{1}{\pi}\int_{-\pi}^{\pi} (-3x^2+1)\cos(nx)\,dx$$

$-3x^2+1$	$\cos(nx)$	
$-6x$	$\frac{\sin(nx)}{n}$	
-6	$\frac{\cos(nx)}{n^2}$	
0	$-\frac{\sin(nx)}{n^3}$	

$$= \frac{1}{\pi}\left[(-3x^2+1)\frac{\sin(nx)}{n}\Big|^{0} - 6x\frac{\cos(nx)}{n^2} + 6\frac{\sin(nx)}{n^3}\Big|^{0}\right]_{-\pi}^{\pi}$$

$$= \frac{1}{\pi}\left[-\frac{6x\cos(nx)}{n^2}\Big|_{-\pi}^{\pi}\right] = \left[\left[-\frac{6\pi}{n^2}\cos(nx)\right] - \left[\frac{-6(-\pi)}{\pi^2}\cos(-n\pi)\right]\right]\frac{1}{\pi}$$

$$= \frac{1}{\pi}\left[\left(-\frac{6\pi}{n^2}(-1)^n\right) - \left(\frac{6\pi}{n^2}(-1)^n\right)\right] = \left[-\frac{12\pi}{n^2}(-1)^n\right]\frac{1}{\pi} = \frac{-12(-1)^n}{n^2}$$

1.2.3 Función propuesta por Lino Ehecatl Romero Rosales

$$F(x) = -\frac{1}{4}x^4 + 3$$

$$-\pi \leq x \leq \pi$$

$$a_0 = \frac{1}{\pi}\int_{-\pi}^{\pi}\left(-\frac{1}{4}x^4 + 3\right)dx$$

$$= \frac{1}{\pi}\left(-\frac{1}{4}\int_{-\pi}^{\pi}x^4\,dx + 3\int_{-\pi}^{\pi}dx\right)$$

$$= \frac{1}{\pi}\left[-\frac{x^5}{20} + 3x\Big|_{-\pi}^{\pi}\right]$$

$$= \frac{1}{\pi}\left[-\frac{\pi^5}{20} + 3\pi - \frac{\pi^5}{20} + 3\pi\right]$$

$$= \frac{1}{\pi}\left(-\frac{\pi^5}{10} + 6\pi\right)$$

$$a_0 = -\frac{\pi^4}{10} + 6 = 6 - \frac{\pi^4}{10}$$

$$a_0/2 = 3 - \frac{\pi^4}{20}$$

Usamos integración por partes al ser un producto.

$$a_n = \frac{1}{\pi}\int_{-\pi}^{\pi}\left(-\frac{x^4}{4} + 3\right)\cos(nx)\,dx$$

$u = -\frac{x^4}{4} + 3$

$du = -x^3\,dx$

$dv = \cos(nx)\,dx$

$v = \frac{\sin(nx)}{n}$

$$\left(-\frac{x^4}{4}+3\right)\left(\frac{\sin(nx)}{n}\right)-\int\frac{-x^3\sin(nx)}{n}dx$$

$$=\left(-\frac{x^4}{4}+3\right)\left(\frac{\sin(nx)}{n}\right)+\frac{1}{n}\int x^3\sin(nx)\,dx$$

$u=x^3 \qquad dv=\sin(nx)\,dx$

$du=3x^2dx \qquad v=-\frac{\cos(nx)}{n}$

$$\int x^3\sin(nx)\,dx=\frac{x^3\cos(nx)}{n}-\frac{3}{n}\int x^2\cos(nx)\,dx$$

$$\int x^2\cos(nx)\,dx=\frac{x^2\sin(nx)}{n}-\frac{2}{n}\int x\sin(nx)\,dx$$

$u=x^2 \qquad dv=\cos(nx)$

$du=2xdx \qquad v=\frac{\sin(nx)}{n}$

$$\int x\sin(nx)\,dx=-\frac{x\cos(nx)}{n}+\frac{1}{n}\int\cos(nx)\,dx$$

$u=x \qquad dv=\sin(nx)\,dx$

$du=dx \qquad v=\frac{-\cos(nx)}{n}$

$$=-\frac{x\cos(nx)}{n}+\frac{\sin(nx)}{n^2}$$

$$\int x\sin(nx)\,dx=-\frac{x\cos(nx)}{n}-\frac{x\cos(nx)}{n^2}+\frac{\sin(nx)}{n^3}$$

$$\int x^2\cos(nx)\,dx=\frac{x^2\sin(nx)}{n}+\frac{2x\cos(nx)}{n^2}+\frac{2x\cos(nx)}{n^3}-\frac{2\sin(nx)}{n^4}$$

$$\int x^3\sin(nx)\,dx=\frac{x^3\cos(nx)}{n}-\frac{3x^2\sin(nx)}{n^2}-\frac{6x\cos(nx)}{n^3}-\frac{6x\cos(nx)}{n^4}+\frac{6\sin(nx)}{n^5}$$

$$\frac{1}{n}\int x^3\sin(nx)\,dx=\frac{x^3\cos(nx)}{n^2}-\frac{3x^2\sin(nx)}{n^3}-\frac{6x\cos(nx)}{n^4}-\frac{6x\cos(nx)}{n^5}+\frac{6\sin(nx)}{n^6}$$

$$a_n = \frac{1}{\pi}\left[\frac{\left(-\frac{1}{4}x^4+3\right)\sin(nx)}{n} - \frac{x^3\cos(nx)}{n^2} + \frac{3x^2\sin(nx)}{n^3} + \frac{6x\cos(nx)}{n^4} - \frac{6\sin(nx)}{n^3}\right]_{-\pi}^{\pi}$$

$$a_n = \frac{1}{\pi}\left[\left(\frac{\left(-\frac{1}{4}\pi^4+3\right)\sin(n\pi)}{n} - \frac{\pi^3\cos(n\pi)}{n^2} + \frac{3\pi^2\sin(n\pi)}{n^3} + \frac{6\pi\cos(n\pi)}{n^4} - \frac{6\sin(n\pi)}{n^3}\right)\right.$$

$$\left. - \left(\frac{\left(-\frac{1}{4}(-\pi)^4+3\right)\sin(-n\pi)}{n} - \frac{(-\pi)^3\cos(n\pi)}{n^2} + \frac{3(-\pi)^2\sin(-n\pi)}{n^3} + \frac{-6\pi\cos(-n\pi)}{n^4} - \frac{6\sin(-n\pi)}{n^3}\right)\right]$$

$$a_n = \frac{1}{\pi}\left[\frac{-\pi^3(-1)^n}{n^2} + \frac{6\pi(-1)^n}{n^4} + \frac{\pi^3(-1)^n}{n^2} - \frac{6\pi(-1)^n}{n^4}\right]$$

$$a_n = \frac{1}{\pi}\left[\frac{-2\pi^3(-1)^n}{n^2} + \frac{12\pi(-1)^n}{n^4}\right]$$

$$a_n = (-1)^n\left(-\frac{2\pi^2}{n^2} + \frac{12}{n^4}\right)$$

Como la función es par bn es igual a 0

$$f(x) = 3 - \frac{\pi^4}{20} + \sum_{n=1}^{\infty}(-1)^{n+1}\left(\frac{2\pi^2}{n^2} - \frac{12}{n^4}\right)\cos(nx)$$

1.2.4 Función propuesta por Aarón Ugalde Téllez

Calculo de valor medio

$$a_0 = \frac{1}{2\pi}\int_{-\pi}^{\pi}\left(-\frac{5}{6}x^2+\frac{1}{3}x-\frac{2}{7}\right)dx = \frac{1}{\pi}\int_{0}^{\pi}\left(-\frac{5}{6}x^2-\frac{2}{7}\right)dx$$

al ser una función impar se elimina en el intervalo

$$a_0 = \frac{1}{\pi}\left[-\frac{5}{18}x^3-\frac{2}{7}x\right]_0^{\pi} = -\frac{5}{18}\pi^2-\frac{2}{7} = -3.027$$

$$a_n = \frac{2}{\pi}\int_0^{\pi}\left(-\frac{5}{6}x^2-\frac{2}{7}\right)\cos(nx)\,dx$$

$$a_n = \frac{-5}{3\pi}\int_0^{\pi}x^2\cos(nx)\,dx = -\frac{5}{3\pi}\left[\frac{2\pi(-1)^n}{n^2}\right] = \frac{10(-1)^n}{3n^2} = \frac{10(-1)^{n+1}}{3n^2}$$

$$b_n = \frac{2}{\pi}\int_0^{\pi}\left(\frac{1}{3}x\right)\sin(nx)\,dx = \frac{2}{3\pi}\int_0^{\pi}x\sin(nx)\,dx$$

$$b_n = \frac{2}{3\pi}\left[\frac{-\pi(-1)^n}{n}\right] = \frac{-2(-1)^n}{3n} = \frac{2(-1)^{n+1}}{3n}$$

$$f(x) = \left(-\frac{5}{18}\pi^2-\frac{2}{7}\right)+\sum_{n=1}^{\infty}\left[\frac{10(-1)^{n+1}}{3n^2}\cos(nx)+\frac{2(-1)^{n+1}}{3n}\sin(nx)\right]$$

1.3 Conclusiones del capítulo

1.3.1 Conclusión — Said Carbot Cruz Trejo

Las series de Fourier siempre me han parecido un dolor de cabeza. De esas cosas donde si me equivoco tantito al principio, todo lo demás ya no sirve. A mí me las enseñó el profesor Toala. Sus materias han sido, por mucho, de las que más me han costado en ESCOM, al punto de pensar más de una vez si de verdad quería seguir con ellas. Pero, al mismo tiempo, siento que aprendí bastante bien a resolver una serie de Fourier para prácticamente cualquier función.

La cosa es la práctica y la verdad es que estoy oxidado. Cuando menos lo importante es que sé qué trucos usar ya y qué representan.

El problema para mí estaba en el método tradicional de integración por partes. Había ejercicios en los que parecía que nunca terminaría, integraciones una tras otra, signos que se cruzaban y despejes que terminaban siendo un desastre.

Por eso empecé a usar el método tabular. Cuando las funciones lo permiten, para mí resulta muchísimo más sencillo. En lugar de hacer cada integración por partes por separado.

También aprendí algunos trucos que terminaban siendo muy útiles en ejercicios y exámenes. Por ejemplo, cuando aparecía un coseno evaluado en π, podía usar directamente que $\cos(\pi) = (-1)^n$, en lugar de dejar toda la expresión llena de términos. Algo parecido pasaba cuando identificaba desde el principio si una función era par o impar. En esos casos podía saber que alguno de los coeficientes a_n o b_n era cero y evitarme una cantidad de cuentas que realmente no necesitaba hacer. Y cuando me pedían obtener los coeficientes en general, podía trabajar directamente con la fórmula en función de n, en lugar de calcularlos uno por uno.

Ya después de todo esto, empecé a ver que las series y transformadas de Fourier no eran solamente un tema más de matemáticas que tenía que resolver para una materia. Vi que pasar una señal del dominio del tiempo al dominio de la frecuencia se usa en muchas de las áreas que me interesan, y ahí fue cuando empezó a hacerme más sentido fuera del salón de clases.

En mi caso, esto lo pude relacionar directamente con lo que he estado haciendo en computación cuántica y simulación molecular con Qiskit. Ahí aparece la Transformada Cuántica de Fourier, o QFT. La idea cambia bastante porque ya no estoy trabajando simplemente con una señal clásica.

La QFT me permite trabajar con las fases y amplitudes del estado cuántico y es una parte importante de varios algoritmos cuánticos relacionados con la estimación de fase y problemas de eigenvalores y eigenvectores. Al principio la conexión con las series de Fourier me parecía bastante abstracta, pero después de haber trabajado con ellas entendí mejor de dónde venía la idea.

Y es que en IA, en procesamiento de imágenes por ejemplo, puedo pasar una matriz de píxeles al dominio de la frecuencia y trabajar con las propiedades de la señal desde otra perspectiva. Esto puede utilizarse para hacer más eficientes determinadas operaciones, como las convoluciones, y es una de las razones por las que las transformadas de Fourier siguen apareciendo en áreas que a primera vista, parecen bastante alejadas de las matemáticas que estaba viendo en clase.

Al final, lo que empezó para mí como una forma de sobrevivir a los ejercicios de series de Fourier terminó

teniendo bastante más sentido cuando lo relacioné con lo que ya estaba haciendo en computación cuántica e IA.

No fue solamente aprender a resolver integrales de una forma más rápida; fue empezar a reconocer la misma idea matemática apareciendo en problemas completamente distintos.

1.3.2 Conclusión — Magaly López Castro

Para mí, las series de Fourier siempre tuvieron esa vibra de ser un tema donde un signo menos perdido te arruina horas de trabajo. Estando en ESCOM, la carga matemática a veces abruma, y al estar tan acostumbrada a pensar en términos de algoritmos y programación competitiva, hacer toda la "talacha" manual de integrales y despejes me resultaba bastante frustrante al principio (y, siendo sincera, todavía lo es).

Sin embargo, mi perspectiva cambió cuando dejé de verlo como un problema de cálculo tradicional y empecé a buscarle la lógica matemática. Así como en los concursos del ICPC o resolviendo problemas en Codeforces siempre busco la manera de optimizar el código para no dar *Time Limit Exceeded*, en Fourier aprendí a identificar rápido las simetrías. Saber desde el inicio si una función era par o impar y que un coeficiente a_n o b_n se hacía cero automáticamente, me ahorraba páginas enteras de cuentas que no necesitaba hacer. Esto fue posible, sobre todo, gracias a los tips que nos dio el profesor de Procesamiento de Señales para facilitarnos el trabajo.

Pero lo que realmente hizo que el tema me hiciera clic fue conectarlo con mi carrera en Inteligencia Artificial. Pasar una señal del dominio del tiempo al de la frecuencia dejó de ser un concepto abstracto en el pizarrón cuando vi su peso en el *machine learning*. Por ejemplo, en el procesamiento de imágenes o visión artificial, aplicar estas transformadas hace que operaciones súper pesadas, como las convoluciones, se vuelvan muchísimo más eficientes.

Tener que trabajarlo en Excel fue súper extraño porque, siendo sincera, nunca jamás lo había abordado de una forma más computacional. Lo habitual era sacar los coeficientes y ahí se quedaba; esta es la primera vez que uso esos coeficientes para evaluar puntos y aproximarlos a mi función, viéndolo de una manera mucho más gráfica y sin tanta "talacha".

Espero seguir descubriendo más sobre cómo se aplica Fourier o cómo se puede relacionar con el cómputo paralelo, para que un concepto que solo veía matemáticamente pueda transformarse en una herramienta tangible en mi día a día.

1.3.3 Conclusión — Lino Ehecatl Romero Rosales

Realizar esta práctica me dejó aprendizajes muy valiosos, sobre todo en la forma en que conectamos la teoría analítica con la programación y las aplicaciones reales de nuestra carrera. Al resolver a mano la serie de Fourier para la función propuesta, el reto no fue tanto la dificultad de las matemáticas sino la disciplina de observar cómo se descompone la señal paso a paso. Resolver la serie para encontrar los coeficientes , y te obliga a entender la estructura interna de la función. Sin embargo, el verdadero aprendizaje ocurre al tomar esos resultados en papel y pasarlos a scripts en Python o MATLAB. Validar de forma computacional que, al agregar más armónicos, la superposición de senos y cosenos reconstruye perfectamente la función original, es una forma excelente de comprobar que nuestras matemáticas están correctas y listas para aplicarse. Al investigar el contexto y las aplicaciones para este capítulo, me pareció fascinante ver cómo algo que comenzó como un modelo térmico es hoy el lenguaje universal de la tecnología. En el ámbito de la ingeniería, y recordando temas específicos de procesamiento digital de señales, me queda claro que sin el análisis de Fourier estaríamos trabajando a ciegas. Entender que podemos tomar cualquier señal del mundo real, trasladarla a sus frecuencias mediante operaciones de matrices y diseñarle filtros para aislar ruido o rescatar información, le da un sentido totalmente práctico a las matemáticas puras. Ya no son solo fórmulas, son las herramientas exactas con las que vamos a optimizar sistemas en nuestra vida diaria. Pero definitivamente, el aprendizaje que más me entusiasmó fue dimensionar el impacto de Fourier en la inteligencia artificial. Al investigar, me di cuenta de que estas series trascienden las señales eléctricas y se meten de lleno en la optimización de algoritmos complejos. Ver cómo las bases de Fourier se aplican para aproximar funciones de valor en modelos de aprendizaje por refuerzo, o cómo el análisis espectral ayuda a evaluar el comportamiento de los agentes en la inteligencia de enjambres para evitar que se estanquen en mínimos locales, me demostró que este tema está más vigente que nunca. Incluso descubrir el avance de la computación fotónica, usando luz láser para procesar transformadas instantáneamente y acelerar el entrenamiento de modelos pesados, te abre los ojos a lo que viene. En conclusión, esta práctica fue un excelente ejercicio de integración. Más allá del trabajo matematico de resolver la función algebraica, me sirvió para consolidar mi visión como ingeniero. Me comprobó que dominar el análisis de Fourier es tener la llave para modelar matemáticamente problemas complejos y programar soluciones verdaderamente eficientes, tanto en la ingeniería clásica como en las arquitecturas modernas de inteligencia artificial.

1.3.4 Conclusión — Aarón Ugalde Téllez

Al revisar estas ocho áreas resulta evidente que las series y transformadas de Fourier no son un recurso matemático más entre muchos, sino un lenguaje común que atraviesa de manera transversal a la ciencia y la tecnología contemporáneas. Su idea central —que un fenómeno complejo puede descomponerse en componentes de frecuencia simples, analizarse por separado y luego reconstruirse sin pérdida de información esencial— resulta tan útil para comprimir una fotografía como para reconstruir una imagen médica, afinar un instrumento musical, evaluar el riesgo sísmico de un edificio, describir el comportamiento cuántico de una partícula subatómica o detectar el paso de una onda gravitacional a miles de millones de años luz de distancia.

Lo que hace especialmente notable a esta herramienta es su capacidad de unificación. En telecomunicaciones, el análisis de frecuencias permite que múltiples señales compartan un mismo canal físico sin interferirse, sosteniendo la infraestructura de internet, la telefonía móvil y la transmisión de radio y televisión. En el procesamiento de imágenes, el mismo principio matemático que Fourier usó para modelar la difusión del calor en una lámina metálica es hoy la base del formato JPEG y de los estándares de video más usados del mundo, permitiendo que miles de millones de fotografías y videos se almacenen y transmitan de forma eficiente cada día. En medicina, la reconstrucción por Fourier del espacio-k convierte señales de radiofrecuencia en imágenes anatómicas de altísima resolución sin necesidad de radiación ionizante, siendo hoy una herramienta diagnóstica indispensable. En química analítica, la espectroscopía FTIR aplica el mismo principio para identificar sustancias y detectar impurezas con una precisión que sería inalcanzable con métodos dispersivos tradicionales.

En la física fundamental, el papel de Fourier es aún más profundo: no se trata solo de una técnica de cálculo conveniente, sino de una relación que refleja la naturaleza misma de la realidad cuántica, donde posición y momento son variables conjugadas ligadas precisamente por una transformada de Fourier, lo que explica formalmente el principio de incertidumbre de Heisenberg. Y en la frontera de la astrofísica observacional, el análisis espectral permite a instrumentos como LIGO extraer señales gravitacionales extraordinariamente débiles del ruido de fondo, abriendo una ventana completamente nueva para observar el universo.

Doscientos años después de que Joseph Fourier formulara su método para resolver un problema de conducción térmica, la vigencia de su idea en campos que él jamás podría haber imaginado —la resonancia magnética, las telecomunicaciones digitales, la detección de ondas gravitacionales— es quizás la mejor demostración de que las matemáticas fundamentales, cuando capturan una estructura verdaderamente universal de la naturaleza, trascienden por completo el problema original que las inspiró. Las series de Fourier no solo resolvieron una ecuación diferencial del siglo XIX: proporcionaron una manera nueva de pensar cualquier fenómeno que varíe en el tiempo o en el espacio, y esa forma de pensar sigue, hoy, en el centro mismo de la innovación científica y tecnológica.

Capítulo 2. Gráficas con Excel

En este capítulo se lleva la Serie de Fourier resuelta analíticamente en el capítulo anterior a una hoja de cálculo. Cada integrante tabuló su función en el intervalo $[-\pi, \pi]$, construyó una matriz con los diez primeros armónicos, sumó los términos para obtener la aproximación $F(x)$ y superpuso ambas curvas para valorar la convergencia.

2.1 Gráficas originales de las funciones propuestas

2.1.1 Función propuesta por Said Carbot Cruz Trejo

La información obtenida y modelada en la hoja de cálculo se basa en el comportamiento de la función original $f(x) = x^5$. A continuación se muestra su representación gráfica.

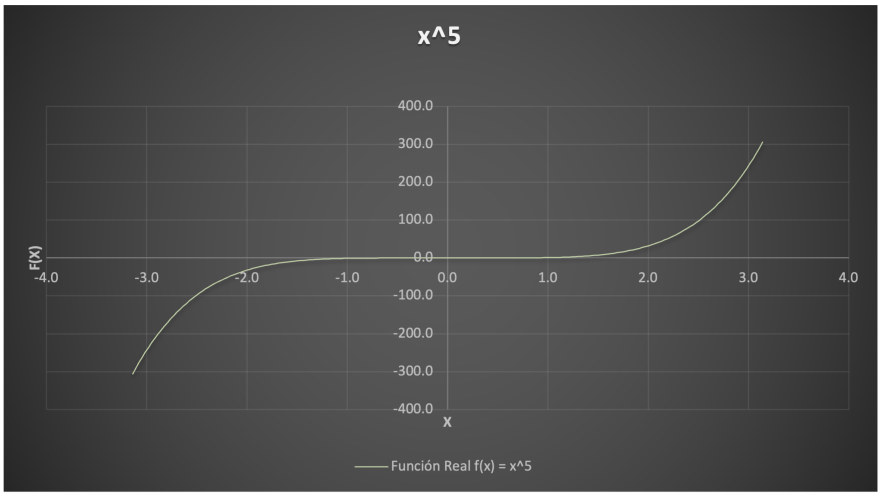


Figura 2.1. Gráfica de la función original $f(x) = x^5$ en el intervalo $[-\pi, \pi]$.

Esta gráfica representa el crecimiento polinomial de quinto grado en los límites de π y $-\pi$, lo cual es fundamental para acotar los saltos periódicos que se modelan en los pasos posteriores.

2.1.2 Función propuesta por Magaly López Castro

Para analizar el comportamiento de la señal original se tabuló la función polinomial $f(x) = 5x^3 - 3x^2 + 1$ dentro de una hoja de cálculo. La evaluación se realizó tomando valores de x con incrementos constantes, lo que generó los puntos base de la primera columna de la tabla.

Como se observa en la figura 2.2, la gráfica resultante ilustra el comportamiento cúbico característico de la función, destacando su crecimiento no lineal en los extremos del intervalo evaluado.

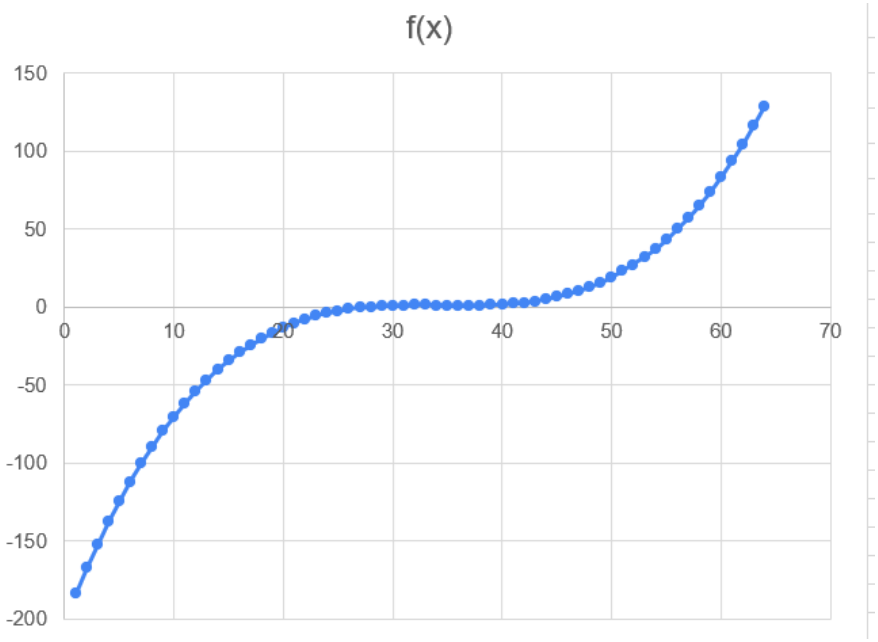


Figura 2.2. Comportamiento de la función original $f(x) = 5x^3 - 3x^2 + 1$.

2.1.3 Función propuesta por Lino Ehecatl Romero Rosales

En esta sección se analiza el comportamiento de la función $f(x) = -\frac{1}{4}x^4 + 3$, restringida al intervalo $[-\pi, \pi]$. Para llevar a cabo la representación numérica se estructuró una hoja de cálculo inicializando la variable x en el límite inferior $-\pi$ y incrementándola uniformemente, para después evaluar la ecuación en cada nodo y obtener la columna de la variable dependiente.

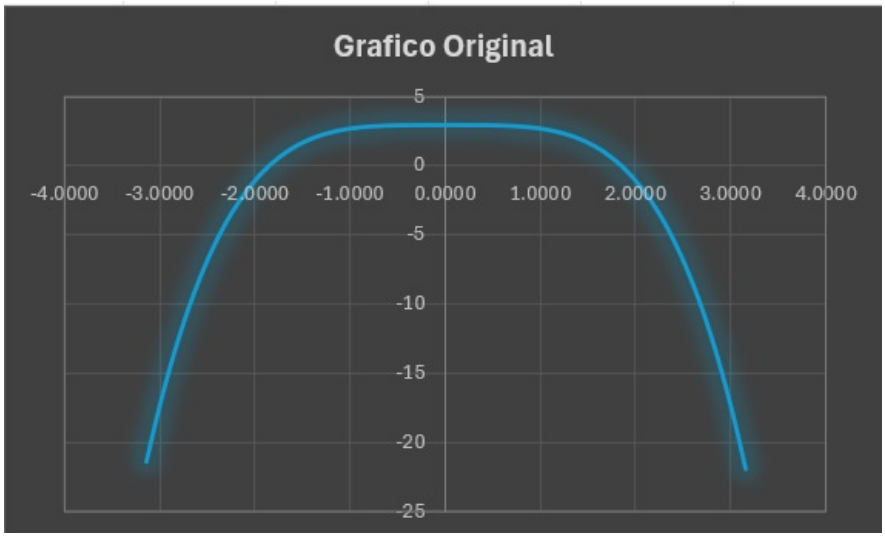


Figura 2.3. Comportamiento de la función original $f(x) = -\frac{1}{4}x^4 + 3$.

Como se observa en la figura 2.3, la curva tiene una simetría par respecto al eje vertical y su vértice

representa el punto máximo de la función dentro de su dominio. Esta representación sirve como la base con la cual se mide la exactitud de la serie de Fourier.

2.1.4 Función propuesta por Aarón Ugalde Téllez

La función asignada es el polinomio de segundo grado

$$f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}, \qquad x \in [-\pi, \pi]. \tag{2.1}$$

A diferencia de las anteriores, esta función no posee simetría: el término $-\frac{5}{6}x^2$ y la constante $-\frac{2}{7}$ son pares, mientras que el término $\frac{1}{3}x$ es impar. En consecuencia, su desarrollo en serie requiere tanto los coeficientes a_n del coseno como los b_n del seno, y ninguna de las dos familias se anula.

Para la tabulación se inicializó la columna de la variable independiente en $x = -\pi$ y se incrementó con paso constante hasta $x = \pi$, evaluando la ecuación (2.1) en cada nodo para obtener la columna de la variable dependiente.

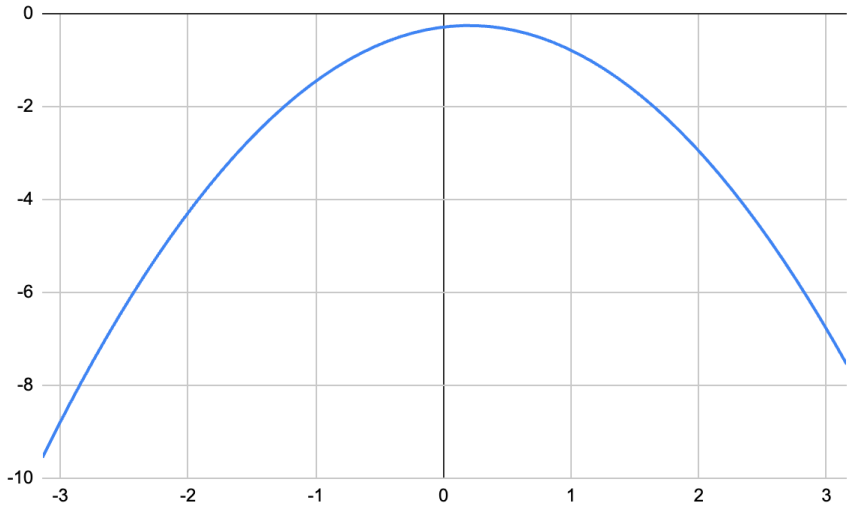


Figura 2.4. Comportamiento de la función original $f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}$.

La curva es una parábola cóncava hacia abajo cuyo vértice se desplaza ligeramente a la derecha del origen, en $x = \frac{1}{5}$, a diferencia de una parábola par centrada en cero. Ese desplazamiento es la huella del término lineal y es también la razón por la que la función toma valores distintos en los dos extremos del intervalo: $f(\pi) \approx -7.4632$ frente a $f(-\pi) \approx -9.5576$. Esta diferencia resulta determinante para interpretar el comportamiento de la serie en los bordes, como se discute en la sección 2.3.4.

2.2 Gráficas obtenidas con la serie de Fourier

2.2.1 Aproximación de Said Carbot Cruz Trejo

Para aproximar la función se consideran los primeros diez términos. Todos los cálculos, incluyendo la matriz de iteraciones y la suma de los términos, se realizaron en la hoja de cálculo y se resumen en las figuras siguientes.

Al tratarse de una función impar, los coeficientes a_n se anulan y el desarrollo queda determinado únicamente por los coeficientes b_n del seno, cuyos valores para las diez iteraciones se muestran a continuación.

n	Coeficiente b_n
1	40.03400602
2	-55.56106903
3	51.30541219
4	-42.77041777
5	35.88216301
6	-30.67285669
7	26.69447436
8	-23.58853413
9	21.10898702
10	-19.08943403

Figura 2.5. Coeficientes b_n obtenidos para las diez primeras iteraciones de $f(x) = x^5$.

A continuación se muestra la matriz para los primeros diez términos y la sumatoria de la serie, extraída de la hoja de cálculo con propósitos de referencia.

x	f(x) = x^5	Fourier (N=10)	Término n=1	Término n=2	Término n=3	Término n=4	Término n=5	Término n=6	Término n=7	Término n=8	Término n=9	Término n=10
-3.1	-306.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
-3.1100	-290.9478	-50.0198	-1.2638	-3.5062	-4.8525	-5.3873	-5.6412	-5.7761	-5.8520	-5.8951	-5.9180	-5.9276

Figura 2.6. Matriz de los diez primeros términos de la Serie de Fourier de $f(x) = x^5$ y su sumatoria.

A partir de esa matriz se obtuvo la siguiente gráfica. Como puede observarse, se parece a la figura original, pero necesitaría más términos para aproximarse mejor.

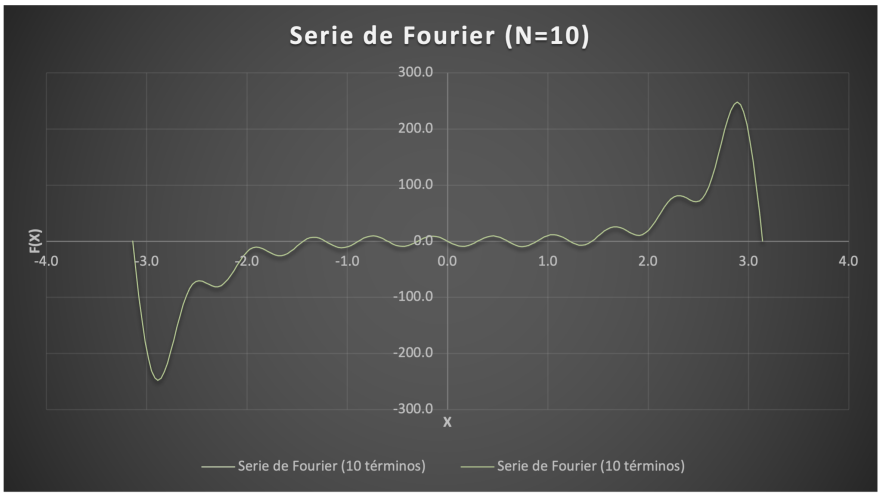


Figura 2.7. Aproximación de $f(x) = x^5$ mediante la suma de los diez primeros términos de la Serie de Fourier.

Como se observa en la figura 2.7, la suma de los términos logra moldear el centro del intervalo aproximándose a la función impar, mostrando las oscilaciones características del fenómeno de Gibbs en los extremos.

2.2.2 Aproximación de Magaly López Castro

Una vez obtenida la función original se procedió a calcular su aproximación analítica mediante la Serie de Fourier. El proceso se llevó a cabo estructurando una matriz de resultados en la hoja de cálculo [41], calculando individualmente los términos de las primeras diez iteraciones ($n = 1$ hasta $n = 10$) junto con el coeficiente fundamental a_0.

x	f(x)	F(x)	a0	1	2	3	4	5	6	7	8	9	10
-3.1416	-183.6414227	-27.4602497	-8.8696044	-11.99971572	-2.999385134	-1.332657247	-0.749302487	-0.479292569	-0.332620515	-0.244181893	-0.186781825	-0.147428528	-0.119279346

Figura 2.8. Matriz de cálculos con los términos obtenidos para las diez iteraciones ($n = 1$ a $n = 10$).

Posteriormente se calculó la sumatoria total de los términos evaluados en cada iteración para obtener la función aproximada $F(x)$.

F(x)
-27.4602497
-110.2475386
-160.4143184
-165.8418743
-138.9549884
-105.3208680
-85.1984798
-81.9769763
-84.7915855
-80.9740190
-67.0529538
-49.8738215
-38.6962160
-36.4836171
-38.0928464
-36.0255201
-27.6271582
-17.0184750
-10.5303742
-10.4888763
-13.2959621
-13.2729141

Figura 2.9. Columna que representa la sumatoria total de los términos de la serie de Fourier, $F(x)$.

Al graficar esa columna se obtiene la aproximación de la serie por sí sola, mostrada en la figura 2.10. La curva reproduce la tendencia creciente del polinomio cúbico y mantiene el tramo central relativamente plano, pero presenta oscilaciones visibles a lo largo del recorrido y un descenso brusco en el extremo derecho. Ambos efectos son consecuencia de haber truncado la sumatoria en diez iteraciones.

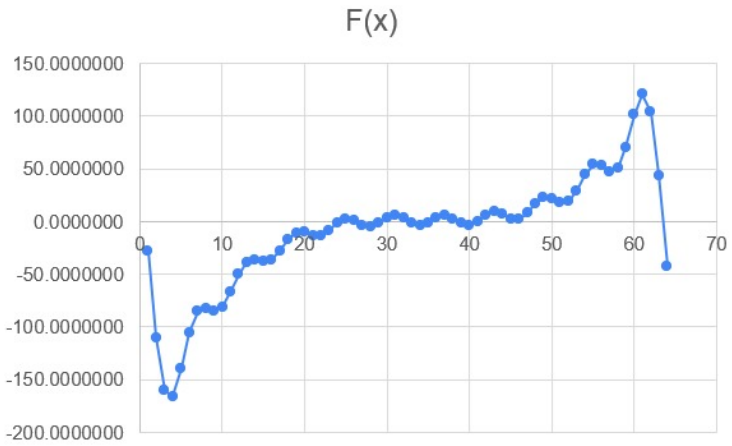


Figura 2.10. Aproximación $F(x)$ de la función $f(x) = 5x^3 - 3x^2 + 1$ obtenida con diez iteraciones de la Serie de Fourier.

Conviene notar que el eje horizontal de esta gráfica corresponde al número de renglón de la hoja de cálculo y no directamente al valor de x, por lo que el intervalo $[-\pi, \pi]$ aparece recorrido de izquierda a derecha en los sesenta y cuatro nodos evaluados.

2.2.3 Aproximación de Lino Ehecatl Romero Rosales

Para la reconstrucción de la señal mediante series de Fourier se empleó una matriz iterada en la hoja de cálculo. Aprovechando la simetría par de la función, todos los coeficientes b_n equivalen a cero, por lo que el modelo se optimizó usando únicamente el término constante $a_0/2 = -1.8704$ y los coeficientes a_n del coseno. La figura 2.11 muestra los valores obtenidos para las diez iteraciones: la columna b_n aparece completa en ceros, lo que confirma numéricamente la simetría par de la función, y los a_n alternan de signo mientras decrecen en magnitud desde 7.7392 hasta -0.1962.

n	a_n	b_n
1	7.7392088	0
2	-4.1848022	0
3	2.04509727	0
4	-1.1868256	0
5	0.77036835	0
6	-0.5390521	0
7	0.39784308	0
8	-0.3054955	0
9	0.24186495	0
10	-0.1961921	0

Figura 2.11. Coeficientes a_n y b_n obtenidos para las diez primeras iteraciones de $f(x) = -\frac{1}{4}x^4 + 3$.

Las columnas del sistema representan las iteraciones desde $n = 1$ hasta $n = 10$, y el cálculo de cada celda de la matriz multiplica el coeficiente a_n iterado por $\cos(nx)$. La figura 2.12 presenta un extracto de esa matriz de resultados correspondiente al extremo inferior del intervalo, en $x \approx -\pi$. En ese punto $\cos(n\pi) = (-1)^n$, de modo que cada término reproduce el coeficiente a_n con el signo invertido en los órdenes impares; por eso todos los valores de la fila resultan negativos aunque los coeficientes originales alternen.

n	n	n	n	n	n	n	n	n	n
1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
-7.7392088	-4.1848022	-2.0450973	-1.1868255	-0.7703684	-0.5390521	-0.3978431	-0.3054954	-0.2418649	-0.1961921

Figura 2.12. Extracto de la matriz iterativa de los diez primeros armónicos, evaluada en $x \approx -\pi$.

Para obtener la aproximación final $F(x)$ se configuró una columna de sumatoria que agrupa $a_0/2$ con la suma horizontal de los diez armónicos para cada valor de x. La representación gráfica del resultado es la siguiente.

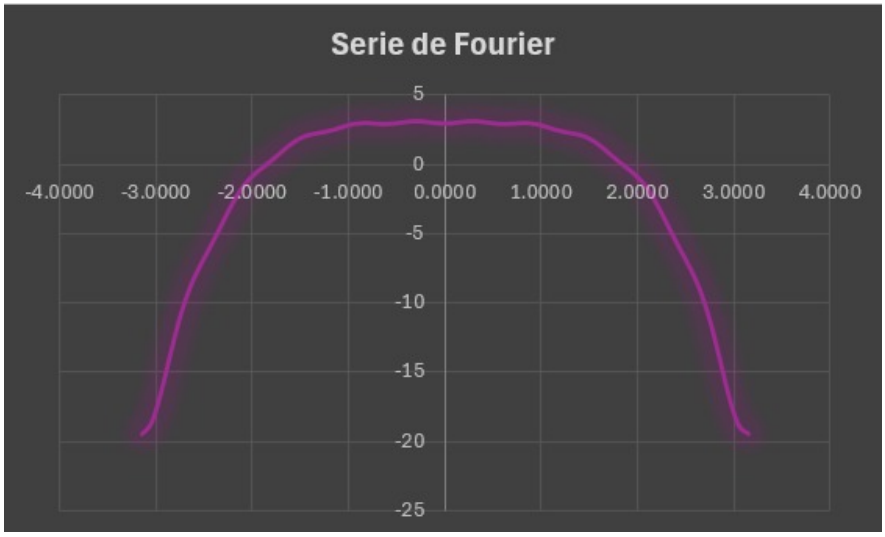


Figura 2.13. Aproximación de $f(x) = -\frac{1}{4}x^4 + 3$ mediante la Serie de Fourier con diez armónicos.

La figura 2.13 demuestra cómo la suma sucesiva de ondas coseno de distintas frecuencias y amplitudes adquiere una curvatura similar a la propuesta en el problema original.

2.2.4 Aproximación de Aarón Ugalde Téllez

Puesto que la función carece de simetría, la hoja de cálculo debe contemplar las dos familias de coeficientes. Integrando la ecuación (2.1) sobre el intervalo se obtienen las expresiones cerradas que alimentan la matriz:

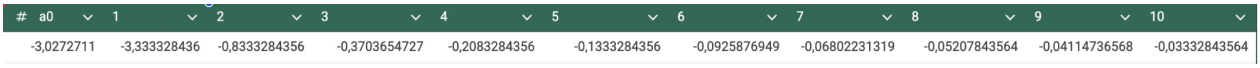
$$\frac{a_0}{2} = -\frac{5\pi^2}{18} - \frac{2}{7} \approx -3.0273, \tag{2.2}$$

$$a_n = \frac{10\,(-1)^{n+1}}{3n^2}, \tag{2.3}$$

$$b_n = \frac{2\,(-1)^{n+1}}{3n}. \tag{2.4}$$

Conviene notar el distinto ritmo de decaimiento de ambas familias: los a_n decrecen como $1/n^2$, mientras que los b_n lo hacen apenas como $1/n$. Son estos últimos, los que provienen del término lineal, los que dominan el error con pocos armónicos y los que gobiernan el comportamiento de la serie en los extremos del intervalo.

Con esos coeficientes se construyó la matriz, en la que cada celda corresponde a un armónico completo, es decir, a la suma de su término en coseno y su término en seno. La figura 2.14 muestra un extracto tomado del extremo inferior del intervalo, en $x \approx -\pi$. Esa fila permite verificar el modelo sin recurrir a la gráfica: como $\operatorname{sen}(n\pi) = 0$, la contribución de los senos se anula en ese punto y cada celda se reduce a $a_n \cos(n\pi) = a_n(-1)^n$. El resultado es que todos los términos aparecen negativos pese a que los coeficientes alternan de signo, y sus magnitudes reproducen una a una las que dan las expresiones anteriores.



# a0	1	2	3	4	5	6	7	8	9	10
-3,0272711	-3,333328436	-0,8333284356	-0,3703654727	-0,2083284356	-0,1333284356	-0,0925876949	-0,06802231319	-0,05207843564	-0,04114736568	-0,03332843564

Figura 2.14. Extracto de la matriz de los diez armónicos, evaluada en $x \approx -\pi$, precedida por el término constante $a_0/2$.

La columna de sumatoria concentra el resultado del modelo. Sumando el término constante $a_0/2 = -3.0272711$ con los diez armónicos de esa misma fila se obtiene $F(-\pi) \approx -8.1931$, que es exactamente el primer valor de la columna $F(x)$ de la figura 2.15. Esta coincidencia confirma que las referencias de la hoja están bien ancladas y que la matriz suma lo que debe.

# F(x)	# a0
-8,1931145	-3,0272711
-8,6569817	-3,0272711
-8,6379195	-3,0272711
-8,1861175	-3,0272711
-7,5202343	-3,0272711
-6,8828747	-3,0272711
-6,4007627	-3,0272711
-6,0451118	-3,0272711
-5,7044060	-3,0272711
-5,2972188	-3,0272711
-4,8333337	-3,0272711
-4,3861508	-3,0272711
-4,0161351	-3,0272711
-3,7187723	-3,0272711
-3,4383847	-3,0272711
-3,1261634	-3,0272711

Figura 2.15. Columna de sumatoria $F(x)$ junto al término constante $a_0/2$, repetido en cada renglón.

Al graficar esa columna se obtiene la aproximación de la serie por sí sola.

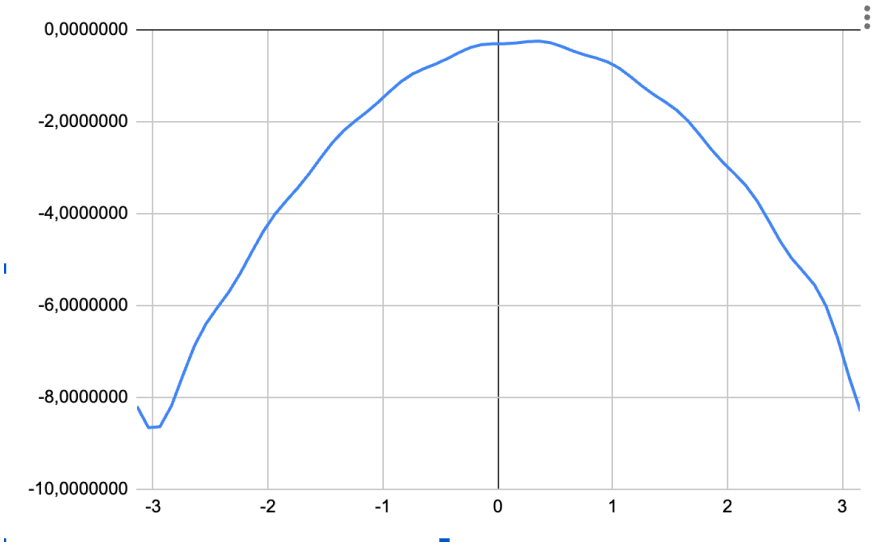


Figura 2.16. Aproximación $F(x)$ de $f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}$ mediante la Serie de Fourier con diez armónicos.

La curva conserva la forma de parábola invertida en casi todo el intervalo, pero se aparta de ella en los extremos: en lugar de continuar descendiendo hacia los valores de la función original, se dobla y forma un pequeño rizo cerca de $x = -\pi$. Ese rizo no es un error de la hoja de cálculo, sino la manifestación del comportamiento que se analiza en la sección 2.3.4.

2.3 Comparaciones entre las gráficas obtenidas

2.3.1 Comparación de Said Carbot Cruz Trejo

Al superponer la función original y la aproximación mediante la serie de Fourier puede analizarse el nivel de precisión alcanzado con diez iteraciones.

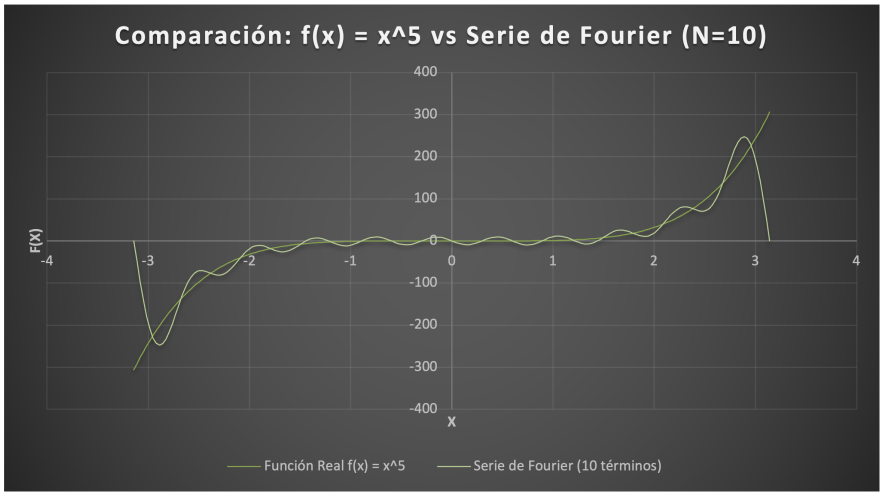


Figura 2.17. Comparación entre la gráfica original $f(x) = x^5$ y la aproximación por Serie de Fourier con diez iteraciones.

La gráfica ilustra cómo la serie persigue la trayectoria de la curva original. La mayor discrepancia se ubica en los bordes del intervalo debido a la naturaleza continua de las ondas senoidales intentando compensar los saltos de una función que, al extenderse periódicamente, se vuelve discontinua en π y $-\pi$.

2.3.2 Comparación de Magaly López Castro

Para valorar el ajuste se superpusieron en un mismo plano la señal original $f(x) = 5x^3 - 3x^2 + 1$, mostrada de forma aislada en la figura 2.2, y su aproximación $F(x)$ de la figura 2.10. El resultado se presenta en la figura 2.18, donde la curva azul corresponde a la función original y la roja a la serie de Fourier.

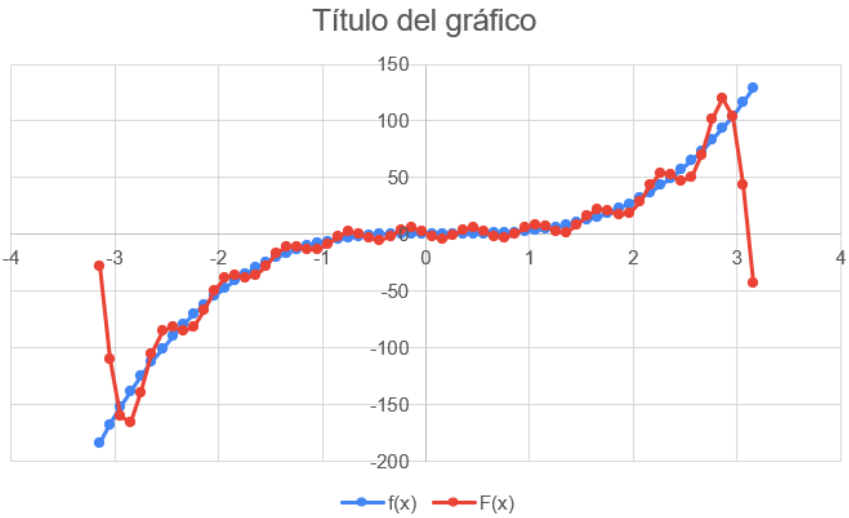


Figura 2.18. Superposición de la función original $f(x)$ frente a la aproximación de Fourier $F(x)$ con diez iteraciones.

La comparación revela que la matriz de diez iteraciones es suficiente para que la aproximación capture la tendencia de crecimiento cúbico de la señal base. Sin embargo, al superponerlas se hacen evidentes ciertas oscilaciones (*ripples*) en la curva aproximada, especialmente en los extremos del intervalo evaluado. Este comportamiento ilustra gráficamente el desafío de aproximar una curva polinomial utilizando una sumatoria finita de funciones periódicas. Queda demostrado visualmente que, aunque la serie de Fourier sigue el camino del polinomio original, requeriría un número mucho mayor de iteraciones para suavizarse y lograr un empalme exacto.

2.3.3 Comparación de Lino Ehecatl Romero Rosales

El objetivo final de la evaluación numérica es contrastar la señal original $f(x)$ con la señal reconstruida $F(x)$ para analizar el margen de error del modelo. Para ello se sobrepusieron los datos de ambas columnas de la hoja de cálculo en un mismo plano.

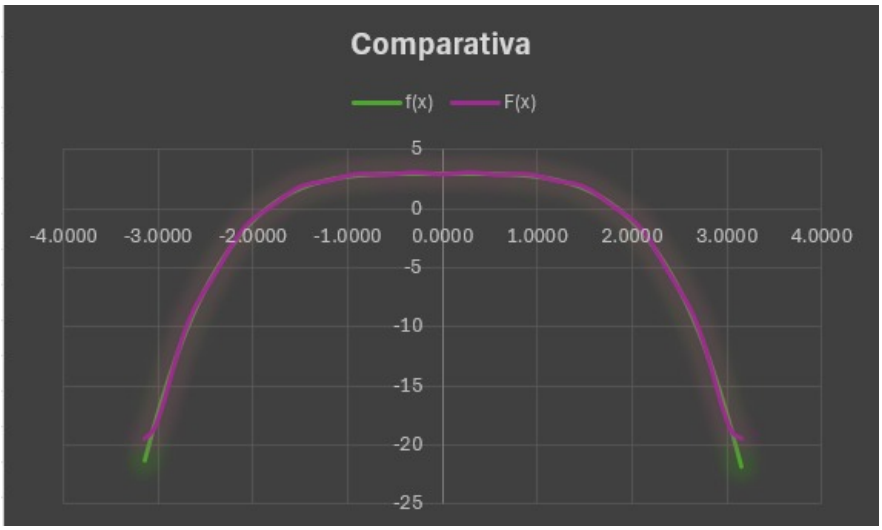


Figura 2.19. Comparativa de ajuste: función original frente a la Serie de Fourier.

Al analizar la figura 2.19 se aprecia que ambas gráficas tienen una convergencia muy parecida en la mayor parte del intervalo. La serie de Fourier con apenas diez iteraciones logra modelar la concavidad de la función y el ajuste es casi exacto en el centro del dominio; sin embargo, al observar los extremos del intervalo entre $-\pi$ y π se nota una leve desviación típica de las aproximaciones. Incrementar el número de iteraciones en la matriz permitiría suavizar este error y lograr una convergencia aún más cercana a la ecuación original.

2.3.4 Comparación de Aarón Ugalde Téllez

La superposición de $f(x)$ y $F(x)$ permite valorar el ajuste alcanzado con diez armónicos.

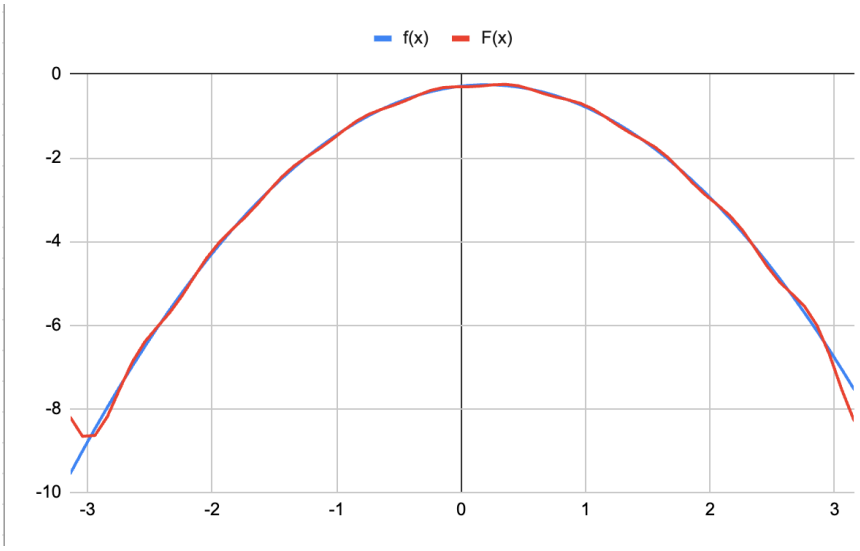


Figura 2.20. Comparativa de ajuste: función original $f(x)$ frente a la Serie de Fourier $F(x)$ con diez armónicos.

La figura 2.20 muestra un ajuste prácticamente indistinguible en todo el tramo central: entre $x = -2.5$ y $x = 2.5$ ambas curvas se superponen y sólo se separan lo suficiente para distinguirse por el grosor del trazo. La discrepancia se concentra en los extremos, donde la curva de la serie se desprende de la parábola y la cruza.

Este comportamiento se deduce de la propia función. Como se señaló en la sección 2.1.4, $f(-\pi) \neq f(\pi)$: al repetir el intervalo periódicamente aparece un salto de altura $f(-\pi) - f(\pi) = -\frac{2\pi}{3} \approx -2.0944$ en cada múltiplo impar de π. Una suma finita de senos y cosenos, que son funciones continuas, no puede reproducir una discontinuidad, de modo que la serie no converge al valor de la función en esos puntos sino al punto medio del salto:

$$F(\pm\pi) = \frac{f(\pi^-) + f(-\pi^+)}{2} = -\frac{5\pi^2}{6} - \frac{2}{7} \approx -8.5104. \tag{2.5}$$

La hoja de cálculo permite comprobarlo: con diez armónicos la columna de sumatoria arroja $F(-\pi) \approx -8.1931$, un valor que no se parece a $f(-\pi) \approx -9.5576$ pero que ya se aproxima al punto medio -8.5104 y seguiría acercándose a él conforme se agregaran términos. Es decir, la serie no está fallando en reproducir la función: está convergiendo correctamente, sólo que hacia un valor distinto del que toma f en ese punto.

Éste es el mismo fenómeno de Gibbs que reportan los compañeros cuya función también resulta discontinua al extenderse. La diferencia es de origen: en el caso de $f(x) = x^5$ el salto proviene de la propia paridad impar del polinomio, mientras que aquí lo introduce únicamente el término lineal $\frac{1}{3}x$. Si ese término se eliminara, la función quedaría par, los b_n se anularían y la extensión periódica sería continua, como ocurre con la función de cuarto grado analizada en la sección 2.3.3.

Por eso se espera un ajuste muy cercano en el interior del intervalo —donde la parábola es suave y los a_n, que decaen como $1/n^2$, convergen con rapidez— y una desviación concentrada en las cercanías de $\pm\pi$, que sólo se reduce lentamente al agregar más armónicos.

2.4 Conclusiones del capítulo

2.4.1 Conclusión — Said Carbot Cruz Trejo

En esta segunda fase del proyecto, el trabajo consistió en pasar del cálculo analítico de las series de Fourier a su implementación visual y numérica en una hoja de cálculo. Trabajar la teoría en papel me proporcionó las bases matemáticas, pero plasmar esos datos en una matriz me hizo enfrentar los problemas de las aproximaciones numéricas. Durante este proceso observé las diferencias prácticas entre una ecuación escrita y un modelo programado, especialmente porque tuve que traducir cada paso a un formato que el software pudiera procesar.

Al analizar la función asignada, $f(x) = x^5$, el primer desafío conceptual que encontré fue adaptar los límites de integración en el intervalo de $-\pi$ a π. Como se trata de una función impar que crece a la quinta potencia, los resultados que obtuve al evaluar los extremos fueron de una magnitud alta. Al realizar la extensión periódica para modelarla con senos y cosenos se produjeron saltos y discontinuidades en los bordes del intervalo. Tuve que comprender cómo la serie de Fourier intenta conectar el punto positivo π^5 con el negativo $-\pi^5$ a través de una suma infinita. Cuando grafiqué estas iteraciones identifiqué el fenómeno de Gibbs: observé cómo las ondas senoidales creaban oscilaciones en los extremos para compensar esos saltos periódicos. Ver este comportamiento en la gráfica me ayudó a entender lo que sucedía matemáticamente en los bordes.

Para llevar esto a la práctica construí una matriz de resultados en la hoja de cálculo. Un aspecto que me costó trabajo fue el cálculo general dentro de Excel. Como no estoy familiarizado con las fórmulas de esta herramienta, el proceso inicial tomó tiempo. Tuve que buscar en línea cómo utilizar las funciones matemáticas básicas, en particular cómo escribir y aplicar la función seno. Aprendí que la sintaxis requería comandos específicos como `SIN()` y que debía tener cuidado con la forma en que Excel interpreta los radianes. Esta curva de aprendizaje técnico representó una parte considerable de la práctica, ya que debía asegurarme de que cada celda realizara el cálculo correcto antes de extender la fórmula al resto de la tabla.

Sistematizar los cálculos me obligó a planear la estructura de la hoja. En lugar de calcular cada término uno por uno, la automatización dependió de formular las referencias para iterar la serie en los distintos valores de x. La función principal que utilicé como núcleo para construir el modelo fue `=$P$9*SIN($O$9*$A4)`. Esta instrucción contiene la lógica matemática del armónico. Entender el uso de las referencias absolutas y mixtas fue el punto central para armar la matriz: la celda `$P$9` fijó la columna y la fila para mantener estable el valor del coeficiente de Fourier b_n; la celda `$O$9` ancló el valor de n, que dictaba la frecuencia del armónico; y `$A4` ancló sólo la columna de la variable independiente x, dejando la fila libre. Al arrastrar la fórmula hacia los lados para sumar iteraciones y hacia abajo para avanzar en el eje x, Excel evaluó los términos de la serie.

Después procedí a sumar estos términos armónicos en una columna final para ver cómo se construía la aproximación. Al cumplir con el requerimiento de calcular al menos diez iteraciones observé el cambio en la gráfica paso a paso. En las primeras iteraciones la gráfica mostraba una onda que no se asemejaba al polinomio de quinto grado; conforme fui sumando el quinto, sexto y décimo término registré cómo la gráfica se aplanaba en el centro del intervalo. La forma resultante se acercó a la pendiente original de x^5, dejando fuera de esta convergencia las inestabilidades en los límites causadas por los saltos mencionados.

En conclusión, esta fase del proyecto me demostró que el uso de hojas de cálculo sirve para validar algoritmos y visualizar el comportamiento de ecuaciones complejas. El desarrollo de la práctica me obligó a buscar información técnica sobre el uso del software y a pensar de forma matricial y paralela. Considero que estos conceptos son aplicables a mi formación en Ingeniería en Inteligencia Artificial, ya que me permitieron traducir una suma teórica de términos en un sistema discreto, programable y con resultados visuales directos.

2.4.2 Conclusión — Magaly López Castro

El desarrollo de esta fase del proyecto representó un cambio de paradigma interesante en la forma en que abordo los problemas matemáticos. Estando en ESCOM la carga de matemáticas suele ser pesada y, como estudiante enfocada en Inteligencia Artificial, estoy muy acostumbrada a resolver retos mediante el desarrollo de software puro, implementando algoritmos en C++ o Python, o buscando la máxima optimización de código para competencias. Por eso, tener que trasladar el análisis matemático de la serie de Fourier a una herramienta ofimática como Excel fue, al principio, una experiencia sumamente extraña.

En la programación tradicional suelo utilizar ciclos repetitivos o librerías específicas para procesar cálculos masivos. Sin embargo, estructurar este problema computacionalmente en una hoja de cálculo —armando la matriz de resultados columna por columna para iterar desde $n = 1$ hasta $n = 10$— me obligó a entender la serie desde otra perspectiva. Fue la primera vez que dejé de ver a Fourier como un ejercicio de "talacha" manual de integrales y despejes en el cuaderno, para convertirlo en una evaluación dinámica de miles de puntos.

Esta experiencia me demostró que herramientas como Excel son muy valiosas para visualizar la convergencia matemática sin tanta abstracción. Ver cómo la sumatoria de las celdas lograba que la curva de la aproximación se acercara paso a paso a mi función original $5x^3 - 3x^2 + 1$ hizo que los coeficientes a_n y b_n dejaran de ser simples números en un papel para convertirse en elementos gráficos con peso real.

Además, este ejercicio computacional me permitió reflexionar sobre la importancia del procesamiento de señales en mi carrera. La idea de reconstruir una señal compleja mediante sumas controladas es la misma base que se utiliza hoy en día para el procesamiento de imágenes digitales y la visión artificial. Al final, estructurar la serie de Fourier en una matriz tabular no sólo agilizó el cálculo, sino que transformó un concepto que antes veía como un obstáculo puramente analítico en un algoritmo gráfico, ayudándome a conectar la teoría matemática con el cómputo práctico que utilizo en mi día a día.

2.4.3 Conclusión — Lino Ehecatl Romero Rosales

Esta parte del proyecto me pareció muy interesante porque pasamos de hacer puros cálculos matemáticos y resolver integrales, a ver realmente cómo se comporta la serie de Fourier de forma visual. No creía que fuera posible llegar a graficarla en Excel, pero una vez que definí bien mis columnas y el tamaño de iteraciones para x me di cuenta de lo útil que es esta herramienta para automatizar todo el proceso. Fue muy entretenido comprobar que los coeficientes a_n y b_n que sacamos de nuestra fórmula e integrales funcionaban al meterlos en la ecuación.

La parte que más me ayudó a comprender el tema fue construir la matriz con iteraciones. Cuando resuelves las ecuaciones de forma teórica no se logra apreciar qué significa cada término de la sumatoria, pero al

ir programando columna por columna en la hoja de cálculo pude notar cómo cada armónica nueva que agregábamos iba corrigiendo un poco más la curva. Además, fue una ventaja que mi función fuera par, ya que al saber que los términos del seno valían cero la matriz quedó mucho más limpia y fue más rápido cancelar b_n y trabajar únicamente con los cosenos.

Al momento de graficar y sobreponer la aproximación con la función original llegué a comprender de una mejor manera el concepto de convergencia. Me sorprendió bastante ver que, con sólo usar diez iteraciones, la parte del centro de la parábola quedaba casi emparejada de forma exacta. Sin embargo, también fue un gran apoyo para notar directamente en la gráfica que en los extremos del intervalo, acercándonos a $-\pi$ y π, a la serie todavía se le dificultaba un poco igualar a la original, dejando esa pequeña desviación.

Como conclusión general, esta experiencia me demostró que herramientas como Excel son útiles para nuestra carrera y que no son sólo hojas para hacer cálculos, sino que podemos conseguir aproximaciones para graficar o modificar las ondas. Durante esta actividad se me permitió comprobar cómo aplicar la teoría para digitalizar señales y obtener una perspectiva más real. Me llevo como aprendizaje que la serie de Fourier no es sólo construir señales a base de integrales, sino también poder digitalizarlas, y que si en un futuro necesitamos más precisión en los extremos bastará con arrastrar la fórmula para agregar más iteraciones y hacer que el error sea casi invisible.

2.4.4 Conclusión — Aarón Ugalde Téllez

Lo que más me marcó de esta fase fue descubrir que el término más pequeño de mi función era el que mandaba en el resultado. A simple vista, $\frac{1}{3}x$ parecía despreciable frente al $-\frac{5}{6}x^2$ que domina la forma de la parábola, y sin embargo fue ese sumando el que determinó casi todo el comportamiento de la serie: es el que impide que la función sea par, el que obliga a calcular los coeficientes b_n y el que rompe la continuidad de la extensión periódica en los extremos del intervalo.

Al comparar mi caso con el de mis compañeros esto quedó mucho más claro. La función de Lino es par y su matriz quedó limpia, con los senos cancelados; la de Said es impar y sólo necesitó los senos. La mía no tiene ninguna de las dos simetrías, así que tuve que arrastrar las dos familias de coeficientes en la misma hoja. Esa incomodidad resultó ser la parte más instructiva, porque me obligó a entender qué aporta cada familia en lugar de aplicar una plantilla.

El detalle que no esperaba fue el de la velocidad de convergencia. Los coeficientes a_n decaen como $1/n^2$ y los b_n apenas como $1/n$, de modo que al décimo armónico los primeros ya son prácticamente insignificantes mientras que los segundos siguen aportando. Ver esa diferencia en la tabla explicó por qué agregar más términos mejora tan poco el ajuste en los bordes: no es que la serie esté mal calculada, es que la parte que falta converge lentamente por naturaleza.

Trabajar esto en una hoja de cálculo en lugar de escribir un programa también cambió mi manera de verlo. Estoy acostumbrado a resolver este tipo de problemas con un ciclo que recorre los armónicos, donde el resultado aparece de golpe al final. En Excel la matriz está toda a la vista al mismo tiempo: cada columna es un armónico y se puede observar su contribución individual antes de sumarla. Esa visibilidad hizo evidente algo que un ciclo esconde, y es que la serie de Fourier no es una fórmula que se evalúa, sino una acumulación de correcciones sucesivas sobre una aproximación cada vez mejor.

Capítulo 3. Modelos de programación y métricas de desempeño

En este capítulo se lleva a programas en lenguaje C el cálculo de la Serie de Fourier de la función asignada a cada integrante, que se resolvió analíticamente en el capítulo 1 y se graficó en hojas de cálculo en el capítulo 2. A diferencia de la hoja de cálculo, que evalúa todo de forma secuencial, cada programa reparte el trabajo entre cien procesos hijos creados con `fork()`: cada hijo calcula un armónico, los resultados se reúnen en una región de memoria compartida y un semáforo garantiza que ese acceso se haga sin condiciones de carrera.

3.1 Marco teórico

Dentro del cómputo paralelo, la concurrencia permite que varias tareas matemáticas se ejecuten al mismo tiempo, lo que puede reducir el tiempo de procesamiento. Para repartir el cálculo de los 100 armónicos de la serie entre procesos independientes y garantizar la integridad de la información se recurre a llamadas al sistema y a mecanismos de comunicación entre procesos (IPC) y de sincronización [42], [43]. Las subsecciones siguientes describen los tres elementos que emplean los programas de este capítulo: la creación de procesos con `fork()`, la memoria compartida y los semáforos.

3.1.1 Procesos hijos con la llamada al sistema `fork()` en lenguaje C

En los sistemas operativos tipo UNIX y GNU/Linux, la concurrencia a nivel de sistema se maneja sobre todo mediante la creación de procesos. La interfaz POSIX (*Portable Operating System Interface*) define la llamada al sistema `fork()`, que es el mecanismo fundamental para crear nuevos procesos en lenguaje C [42].

Cuando un programa invoca `fork()`, el sistema operativo suspende momentáneamente la ejecución del proceso invocador (el "proceso padre") y crea un clon exacto de éste, el "proceso hijo". El nuevo proceso recibe una copia independiente del espacio de direcciones de memoria del padre, que incluye el segmento de código, los datos estáticos, el montículo (*heap*) y la pila (*stack*) [44]. Aunque ambos procesos comparten el mismo código fuente, sus espacios de memoria están aislados: una variable que el hijo modifica no altera el valor de esa misma variable en la memoria del padre.

La característica más importante de `fork()` es que, tras una ejecución exitosa, padre e hijo reanudan su ejecución de forma concurrente, justo en la instrucción siguiente a la llamada. Para que el programa distinga qué proceso ejecuta cada bloque de código, `fork()` devuelve valores distintos (de tipo `pid_t`) a cada uno [43]:

- **Al proceso hijo:** la función devuelve exactamente el valor `0`.
- **Al proceso padre:** la función devuelve el identificador de proceso (PID), un entero positivo que corresponde al nuevo hijo.

- **En caso de error:** si el sistema no tiene recursos suficientes (por ejemplo, memoria RAM disponible) para crear el proceso, la función devuelve `-1` al padre y la bifurcación se cancela.

Esta dualidad en el valor de retorno permite usar estructuras condicionales (`if-else`) para asignar rutinas específicas a cada flujo de ejecución. En las implementaciones modernas del núcleo de Linux, `fork()` está muy optimizada mediante una técnica de gestión de memoria llamada *Copy-on-Write* (copia en escritura), que retrasa la copia real de las páginas de memoria hasta que alguno de los dos procesos intenta modificarlas; así se reduce considerablemente la sobrecarga y el tiempo de creación del proceso [42].

3.1.2 Memoria compartida en lenguaje C

La memoria compartida es una de las técnicas de comunicación entre procesos más rápidas en los sistemas basados en UNIX y GNU/Linux. A diferencia de los *pipes* o los *sockets*, en los que los datos se copian una y otra vez entre el espacio de usuario y el del núcleo, la memoria compartida proyecta directamente una región de memoria física en el espacio de direcciones de varios procesos, que pueden acceder a ella al mismo tiempo [44].

Con el estándar System V en GNU/Linux, este segmento se administra principalmente con cuatro funciones en C:

1. `shmget()`: solicita al sistema un bloque nuevo de memoria o permite conectarse a uno existente mediante una clave, y devuelve un identificador del segmento.
2. `shmat()`: acopla el segmento al proceso que la llama y devuelve un puntero a la dirección desde la cual se puede leer o escribir.
3. `shmdt()`: hace lo contrario de la anterior; cuando el proceso termina de usar la memoria, la desacopla de su espacio de direcciones.
4. `shmctl()`: funciona como panel de control del segmento, pues permite consultar su estado, cambiar permisos o destruirlo (con `IPC_RMID`) para que deje de ocupar memoria.

Si cien procesos escriben resultados al mismo tiempo en la misma región de memoria, el programa puede fallar o producir datos corruptos, el problema conocido como *race condition*. Por eso es necesario serializar el acceso con herramientas como los semáforos, que se describen en la siguiente subsección (sección 3.1.3).

3.1.3 Semáforos y sincronización en lenguaje C

Cuando varios procesos cooperan sobre un recurso común en memoria compartida aparece el problema de la *condición de carrera* (*race condition*): dos o más procesos leen y modifican al mismo tiempo el mismo dato, de modo que el resultado final depende del orden en que el planificador intercale sus instrucciones, y ese orden no es determinista [42]. Una instrucción tan simple como `f_x[i] += t` parece una sola operación, pero el procesador la ejecuta como tres: leer el valor, sumarle `t` y escribir el resultado. Si dos procesos leen el mismo valor antes de que cualquiera de ellos escriba, una de las dos sumas se pierde.

El mecanismo clásico para lograr la *exclusión mutua* en estas *secciones críticas* es el semáforo, propuesto por Edsger W. Dijkstra a mediados de los años sesenta [42]. Un semáforo es una variable entera administrada por el sistema operativo que sólo puede modificarse mediante dos operaciones atómicas, es decir, indivisibles:

1. **Operación** *P* (del holandés *Proberen*, "probar"; también *wait*): si el valor del semáforo es mayor que cero, lo decrementa y el proceso continúa; si vale cero, el proceso se bloquea y queda en la cola del semáforo hasta que otro proceso ejecute *V*.
2. **Operación** *V* (del holandés *Verhogen*, "incrementar"; también *signal* o *post*): incrementa el valor del semáforo y, si hay procesos bloqueados en su cola, despierta a uno de ellos.

Un semáforo inicializado en 1 se denomina *binario* o *mutex*: el primer proceso que ejecuta *P* entra a la sección crítica y cualquier otro que llegue mientras tanto se bloquea hasta que el primero ejecute *V* [45].

En GNU/Linux, y en general en los sistemas POSIX, los semáforos de la familia System V se administran como *conjuntos* de semáforos que el núcleo identifica mediante el mecanismo IPC (*Inter-Process Communication*) [44]. Las tres llamadas al sistema fundamentales son:

- `semget(key_t key, int nsems, int semflg)`: crea un conjunto de `nsems` semáforos, o accede a uno existente, y devuelve su identificador. Cuando los procesos que lo comparten son parientes, como ocurre aquí, basta la clave `IPC_PRIVATE`: el padre crea el conjunto antes de `fork()` y los hijos heredan el identificador.
- `semctl(int semid, int semnum, int cmd, ...)`: ejecuta operaciones de control sobre el conjunto, como fijar el valor inicial de un semáforo (`SETVAL`) o eliminar el conjunto del sistema (`IPC_RMID`).
- `semop(int semid, struct sembuf *sops, size_t nsops)`: ejecuta de manera atómica una lista de operaciones *P* o *V* descritas con la estructura `struct sembuf` del código 3.1.

```
1 struct sembuf {
2    unsigned short sem_num; // índice del semáforo dentro del conjunto (0 si
        hay uno solo)
3    short       sem_op; // -1: operación P (wait) +1: operación V (signal)
4    short       sem_flg; // banderas de control (por ejemplo, SEM_UNDO)
5 };
```

Código 3.1. Estructura `sembuf`, que describe una operación atómica sobre un semáforo System V.

Como el núcleo conserva los conjuntos de semáforos aunque el programa que los creó termine, el programa debe eliminarlos de forma explícita con `semctl(..., IPC_RMID)`; de lo contrario quedan como recursos huérfanos que se pueden listar con el comando `ipcs`.

3.2 Programación con procesos en lenguaje C

3.2.1 Función propuesta por Aarón Ugalde Téllez

Serie de Fourier por evaluar

La función asignada es la misma que se resolvió a mano en el capítulo 1 y se tabuló en el capítulo 2:

$$f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}, \qquad x \in [-\pi, \pi]. \tag{3.1}$$

Sus coeficientes de Fourier son

$$\frac{a_0}{2} = -\frac{5\pi^2}{18} - \frac{2}{7} \approx -3.0273, \tag{3.2}$$

$$a_n = \frac{10\,(-1)^{n+1}}{3n^2}, \tag{3.3}$$

$$b_n = \frac{2\,(-1)^{n+1}}{3n}, \tag{3.4}$$

y la aproximación que calcula el programa conserva los primeros cien armónicos:

$$F_{100}(x) = \frac{a_0}{2} + \sum_{n=1}^{100}\left[\frac{10\,(-1)^{n+1}}{3n^2}\cos(nx) + \frac{2\,(-1)^{n+1}}{3n}\sin(nx)\right]. \tag{3.5}$$

El capítulo anterior se limitó a diez armónicos, $F_{10}(x)$; aquí el número de términos se multiplica por diez y cada término lo calcula un proceso distinto. Los valores de x son los mismos 64 nodos de la hoja de cálculo, $x_k = -3.1416 + 0.1\,k$ con $k = 0, 1, \ldots, 63$, es decir, de -3.1416 a 3.1584. Conviene notar que el último nodo queda 0.0168 por encima de π, fuera del intervalo $[-\pi, \pi]$; esto tiene consecuencias visibles en la gráfica (sección 3.2.1).

Diseño de la solución con procesos

Como cada armónico n es independiente de los demás, el cálculo se reparte de forma natural: el hijo número p calcula el armónico $n = p$ en los 64 nodos. Lo único que los procesos comparten es el vector donde se suman las contribuciones, y por eso es ahí donde intervienen la memoria compartida y el semáforo. El programa sigue estos pasos:

1. El padre crea, con `shmget` y `shmat`, una región de memoria compartida para 64 valores de tipo `double` (512 bytes) y escribe en cada posición el término constante $a_0/2$.
2. Crea con `semget` un semáforo System V y lo inicializa en 1 con `semctl`, de modo que la sección crítica comienza libre.
3. Mediante un ciclo con `fork()` crea los 100 procesos hijos.

4. Cada hijo calcula su armónico en un arreglo *local* de 64 términos, sin sincronización alguna. Después ejecuta P, suma su arreglo al vector compartido, ejecuta V, se desacopla de la memoria compartida y termina.

5. El padre espera a los cien hijos con `wait()`, escribe el vector en un archivo CSV y libera la memoria compartida y el semáforo.

La sección crítica se mantuvo lo más corta posible a propósito: el cálculo trigonométrico, que es la parte costosa, se hace fuera de ella sobre datos locales de cada proceso, y el semáforo sólo protege las 64 sumas sobre el vector compartido.

Fragmentos de código explicados

El código completo se encuentra en los anexos (sección 3.5); en los fragmentos siguientes se omiten las comprobaciones de error para concentrarse en la lógica.

El código 3.2 muestra la preparación de los recursos compartidos. `shmget` con la clave `IPC_PRIVATE` crea un segmento nuevo de 512 bytes y `shmat` lo proyecta en el espacio de direcciones del padre. Como los hijos se crean después con `fork()`, heredan esa proyección y ven la misma memoria física: a diferencia del resto de la memoria del padre, que se duplica, ésta no se copia. El semáforo se crea con un solo elemento y `semctl` con `SETVAL` lo deja en 1, valor que lo convierte en un *mutex*.

```
size_t shm_size = NUM_PUNTOS * sizeof(double);
int shm_id = shmget(IPC_PRIVATE, shm_size, IPC_CREAT | 0666);
double *f_x = (double *)shmat(shm_id, NULL, 0);

double a0 = (-5.0 / 18.0) * (M_PI * M_PI) - (2.0 / 7.0); // a0/2
for (int i = 0; i < NUM_PUNTOS; i++) {
   f_x[i] = a0;
}

int sem_id = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
union semun arg;
arg.val = 1;                // sección crítica libre
semctl(sem_id, 0, SETVAL, arg);
```

Código 3.2. Creación de la memoria compartida y del semáforo en el proceso padre.

Las operaciones P y V del código 3.3 son envoltorios de `semop`. Cada una arma una `struct sembuf` (código 3.1) sobre el semáforo 0: con `sem_op = -1` el proceso se bloquea si el semáforo vale cero, y con `sem_op = 1` libera la sección crítica.

```
void sem_wait_op(int sem_id) {
   struct sembuf sb = {0, -1, 0}; // semáforo 0, operación P
   semop(sem_id, &sb, 1);
}

void sem_post_op(int sem_id) {
```

```
    struct sembuf sb = {0, 1, 0}; // semáforo 0, operación V
    semop(sem_id, &sb, 1);
}
```

Código 3.3. Operaciones P (`sem_wait_op`) y V (`sem_post_op`) sobre el semáforo.

En el código 3.4 está el ciclo que crea a los hijos. `fork()` devuelve 0 en el proceso hijo, el PID del hijo en el padre y un valor negativo si falla; por eso el bloque `if (pid == 0)` es exclusivo de los hijos. Como la variable `p` del ciclo se copia en cada hijo, éste sabe qué armónico le toca. Los coeficientes a_n y b_n son las expresiones (3.3) y (3.4), y el factor $(-1)^{n+1}$ se obtiene comprobando la paridad de $n+1$. La acumulación queda encerrada entre P y V: aunque los cien hijos terminen de calcular casi al mismo tiempo, el semáforo los deja entrar de uno en uno, de modo que ninguna suma se pierde. El `exit` final es indispensable, pues sin él cada hijo continuaría el ciclo y crearía nuevos procesos.

```
for (int p = 1; p <= NUM_PROCESOS; p++) {
   pid_t pid = fork();
   if (pid == 0) {                    // --- proceso hijo ---
      int n = p;                      // el hijo p calcula el armónico n = p
      double local_terms[NUM_PUNTOS];
      double signo = ((n + 1) % 2 == 0) ? 1.0 : -1.0; // (-1)^(n+1)
      double an = (10.0 * signo) / (3.0 * n * n);
      double bn = (2.0 * signo) / (3.0 * n);

      for (int i = 0; i < NUM_PUNTOS; i++) { // cálculo local, sin sincronizar
         double x = x_values[i];
         local_terms[i] = an * cos(n * x) + bn * sin(n * x);
      }

      sem_wait_op(sem_id);            // P: entrar a la sección crítica
      for (int i = 0; i < NUM_PUNTOS; i++) {
         f_x[i] += local_terms[i];    // suma en memoria compartida
      }
      sem_post_op(sem_id);            // V: salir de la sección crítica

      shmdt(f_x);
      exit(EXIT_SUCCESS);             // el hijo no debe seguir el ciclo
   }
}
```

Código 3.4. Creación de los 100 hijos: cálculo local del armónico y acumulación protegida por el semáforo.

Por último, el padre (código 3.5) llama cien veces a `wait()`: cada llamada recoge el estado de terminación de un hijo, lo que evita que queden procesos zombis. Con los 64 resultados ya acumulados, el padre los escribe en `fourier_aaron_64puntos.csv` (con `fprintf`, seis decimales) y libera los recursos con `IPC_RMID`, ya que el núcleo no los elimina por sí solo.

```
for (int i = 0; i < NUM_PROCESOS; i++) {
   wait(NULL);                // recoge a cada hijo: sin zombis
```

```
}
/* ... se escribe f_x[] en fourier_aaron_64puntos.csv ... */
shmdt(f_x);
shmctl(shm_id, IPC_RMID, NULL); // elimina la memoria compartida
semctl(sem_id, 0, IPC_RMID);  // elimina el semáforo
```

Código 3.5. El padre espera a los hijos y libera los recursos IPC.

Compilación y ejecución

Las evidencias de esta sección se obtuvieron en un equipo Apple Silicon (arquitectura `arm64`) con macOS, donde el comando `gcc` corresponde al compilador Apple clang 21 de las herramientas de línea de comandos del sistema; la ficha técnica del equipo está en la sección 3.3.1. El programa sólo usa llamadas POSIX y System V IPC, que son las mismas que ofrece el núcleo de GNU/Linux. La única diferencia entre ambas plataformas es la unión `semun` que exige `semctl`: la biblioteca glibc de GNU/Linux no la declara y el programa debe definirla, mientras que el encabezado `<sys/sem.h>` de macOS ya la trae y redefinirla produce un error de compilación. Por eso la definición va protegida como muestra el código 3.6, de modo que el mismo archivo fuente sea válido en ambos sistemas.

```
#if defined(__linux__)
union semun {
   int val;
   struct semid_ds *buf;
   unsigned short *array;
};
#endif
```

Código 3.6. Definición de `union semun` sólo en GNU/Linux.

El código 3.7 muestra la compilación con `-Wall` y optimización `-O2`, enlazando la biblioteca matemática con `-lm`. La compilación termina sin errores ni advertencias: el compilador no imprime ningún mensaje y devuelve el código de salida cero. Como control adicional, el mismo archivo se compiló con `-Wall -Wextra -pedantic`, que tampoco produjo mensajes.

```
$ gcc --version | head -1
Apple clang version 21.0.0 (clang-2100.1.1.101)
$ gcc -Wall -O2 fourier_aaron.c -o fourier_aaron -lm
$ echo $?
0
```

Código 3.7. Compilación del programa sin errores ni advertencias.

La ejecución (código 3.8) muestra primero al padre y su PID, después una línea por cada hijo con su PID y el armónico que procesó, y al final la confirmación de que los cien hijos terminaron y de que se liberaron la memoria compartida y el semáforo. En esta ejecución los PIDs de los hijos resultaron consecutivos, porque el padre los crea uno tras otro. El orden en que los hijos imprimen su línea no está garantizado, pues depende del planificador; en esta ejecución coincidió con el orden de creación.

```
$ ./fourier_aaron
======================================================================
 CÓMPUTO PARALELO - CAPÍTULO 3: PROGRAMACIÓN CON PROCESOS
 Integrante: Aarón Ugalde Téllez
 Evaluación de Serie de Fourier con 100 Procesos Hijos (64 Puntos)
======================================================================

[PADRE | PID: 6729] Creando 100 procesos hijos...

[HIJO #001 | PID: 6730] Armónico n = 1 procesado para los 64 puntos.
[HIJO #002 | PID: 6731] Armónico n = 2 procesado para los 64 puntos.
[HIJO #003 | PID: 6732] Armónico n = 3 procesado para los 64 puntos.
...
[HIJO #098 | PID: 6827] Armónico n = 98 procesado para los 64 puntos.
[HIJO #099 | PID: 6828] Armónico n = 99 procesado para los 64 puntos.
[HIJO #100 | PID: 6829] Armónico n = 100 procesado para los 64 puntos.

[PADRE] Los 100 procesos hijos han completado el cálculo.
[PADRE] Resultados guardados en el archivo 'fourier_aaron_64puntos.csv'.
[PADRE] Recursos IPC (SHM y semáforos) liberados correctamente.
======================================================================
```

Código 3.8. Ejecución del programa con 100 procesos hijos.

La figura 3.1 es la captura de esa sesión en la terminal. Al comparar la salida de ejecuciones sucesivas, el archivo CSV resultó idéntico byte a byte: aunque el orden en que los hijos entran a la sección crítica cambia de una ejecución a otra, la diferencia que eso introduce al sumar números de punto flotante es de un orden muy inferior a los seis decimales con que se escribe el archivo.

```
→  codigo git:(master) ✗ ./a.out
[HIJO #047 | PID: 11820] Armónico n =  47 procesado para los 64 puntos.
[HIJO #048 | PID: 11821] Armónico n =  48 procesado para los 64 puntos.
[HIJO #049 | PID: 11822] Armónico n =  49 procesado para los 64 puntos.
[HIJO #050 | PID: 11823] Armónico n =  50 procesado para los 64 puntos.
[HIJO #051 | PID: 11824] Armónico n =  51 procesado para los 64 puntos.
[HIJO #052 | PID: 11825] Armónico n =  52 procesado para los 64 puntos.
[HIJO #053 | PID: 11826] Armónico n =  53 procesado para los 64 puntos.
[HIJO #054 | PID: 11827] Armónico n =  54 procesado para los 64 puntos.
[HIJO #055 | PID: 11828] Armónico n =  55 procesado para los 64 puntos.
[HIJO #056 | PID: 11829] Armónico n =  56 procesado para los 64 puntos.
[HIJO #057 | PID: 11830] Armónico n =  57 procesado para los 64 puntos.
[HIJO #058 | PID: 11831] Armónico n =  58 procesado para los 64 puntos.
[HIJO #059 | PID: 11832] Armónico n =  59 procesado para los 64 puntos.
[HIJO #060 | PID: 11833] Armónico n =  60 procesado para los 64 puntos.
[HIJO #061 | PID: 11834] Armónico n =  61 procesado para los 64 puntos.
[HIJO #062 | PID: 11835] Armónico n =  62 procesado para los 64 puntos.
[HIJO #063 | PID: 11836] Armónico n =  63 procesado para los 64 puntos.
[HIJO #064 | PID: 11837] Armónico n =  64 procesado para los 64 puntos.
[HIJO #065 | PID: 11838] Armónico n =  65 procesado para los 64 puntos.
[HIJO #066 | PID: 11839] Armónico n =  66 procesado para los 64 puntos.
[HIJO #067 | PID: 11840] Armónico n =  67 procesado para los 64 puntos.
[HIJO #068 | PID: 11841] Armónico n =  68 procesado para los 64 puntos.
[HIJO #069 | PID: 11842] Armónico n =  69 procesado para los 64 puntos.
[HIJO #070 | PID: 11843] Armónico n =  70 procesado para los 64 puntos.
[HIJO #071 | PID: 11844] Armónico n =  71 procesado para los 64 puntos.
[HIJO #072 | PID: 11845] Armónico n =  72 procesado para los 64 puntos.
[HIJO #073 | PID: 11846] Armónico n =  73 procesado para los 64 puntos.
[HIJO #074 | PID: 11847] Armónico n =  74 procesado para los 64 puntos.
[HIJO #075 | PID: 11848] Armónico n =  75 procesado para los 64 puntos.
[HIJO #076 | PID: 11849] Armónico n =  76 procesado para los 64 puntos.
[HIJO #077 | PID: 11850] Armónico n =  77 procesado para los 64 puntos.
[HIJO #078 | PID: 11851] Armónico n =  78 procesado para los 64 puntos.
[HIJO #079 | PID: 11852] Armónico n =  79 procesado para los 64 puntos.
[HIJO #080 | PID: 11853] Armónico n =  80 procesado para los 64 puntos.
[HIJO #081 | PID: 11854] Armónico n =  81 procesado para los 64 puntos.
[HIJO #082 | PID: 11855] Armónico n =  82 procesado para los 64 puntos.
[HIJO #083 | PID: 11856] Armónico n =  83 procesado para los 64 puntos.
[HIJO #084 | PID: 11857] Armónico n =  84 procesado para los 64 puntos.
[HIJO #085 | PID: 11858] Armónico n =  85 procesado para los 64 puntos.
[HIJO #086 | PID: 11859] Armónico n =  86 procesado para los 64 puntos.
[HIJO #087 | PID: 11860] Armónico n =  87 procesado para los 64 puntos.
[HIJO #088 | PID: 11861] Armónico n =  88 procesado para los 64 puntos.
[HIJO #089 | PID: 11862] Armónico n =  89 procesado para los 64 puntos.
[HIJO #090 | PID: 11863] Armónico n =  90 procesado para los 64 puntos.
[HIJO #091 | PID: 11864] Armónico n =  91 procesado para los 64 puntos.
[HIJO #092 | PID: 11865] Armónico n =  92 procesado para los 64 puntos.
[HIJO #093 | PID: 11866] Armónico n =  93 procesado para los 64 puntos.
[HIJO #094 | PID: 11867] Armónico n =  94 procesado para los 64 puntos.
[HIJO #095 | PID: 11868] Armónico n =  95 procesado para los 64 puntos.
[HIJO #096 | PID: 11869] Armónico n =  96 procesado para los 64 puntos.
[HIJO #097 | PID: 11870] Armónico n =  97 procesado para los 64 puntos.
[HIJO #098 | PID: 11871] Armónico n =  98 procesado para los 64 puntos.
[HIJO #099 | PID: 11872] Armónico n =  99 procesado para los 64 puntos.
[HIJO #100 | PID: 11873] Armónico n = 100 procesado para los 64 puntos.

[PADRE] Los 100 procesos hijos han completado el cálculo.
[PADRE] Resultados guardados en el archivo 'fourier_aaron_64puntos.csv'.
[PADRE] Recursos IPC (SHM y semáforos) liberados correctamente.
================================================================
→  codigo git:(master) ✗
```

Figura 3.1. Compilación y ejecución del programa con 100 procesos hijos en la terminal.

Resultados obtenidos y gráfica

El padre escribe los 64 valores acumulados en `fourier_aaron_64puntos.csv`, un archivo de texto plano con dos columnas, `x` y `f_fourier`, una fila por nodo y seis decimales. Ese archivo se importó en la hoja de cálculo de la fase anterior para generar la gráfica de la figura 3.2.

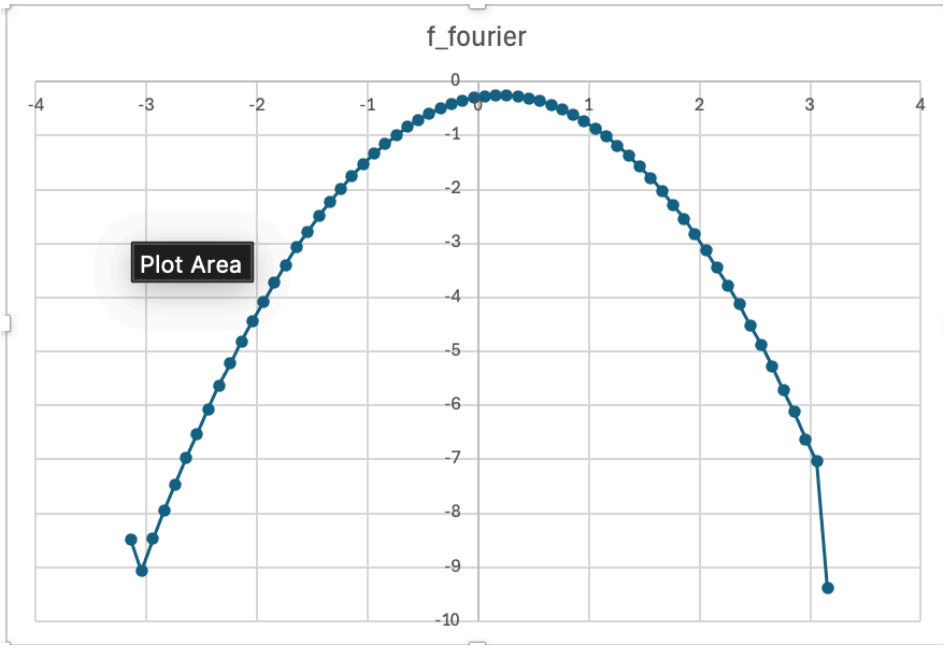


Figura 3.2. Gráfica de $F_{100}(x)$ calculada en C con 100 procesos hijos, en los 64 nodos de $x = -3.1416$ a $x = 3.1584$.

La gráfica cuenta cuatro cosas:

- **Forma parabólica.** La curva es una parábola invertida. Su vértice no está en el origen sino en $x = \frac{1}{5}$, desplazado por el término lineal $\frac{1}{3}x$, y allí $f\left(\frac{1}{5}\right) = -0.2524$. Entre los nodos, el máximo de la serie es -0.2530 en $x = 0.1584$. Toda la curva es negativa, porque el máximo de la parábola ya es menor que cero.

- **Muestreo discreto.** Cada marcador es uno de los 64 nodos; la línea sólo los une. En el interior del intervalo los nodos caen sobre la parábola y la curva es suave.

- **Extremo izquierdo: fenómeno de Gibbs.** La función original vale $f(-\pi) \approx -9.5576$ y $f(\pi) \approx -7.4632$, de modo que al repetirla periódicamente aparece un salto de $-\frac{2\pi}{3} \approx -2.0944$ en cada múltiplo impar de π [2]. Una suma de senos y cosenos no puede reproducir un salto: en el salto converge al punto medio, $-\frac{5\pi^2}{6} - \frac{2}{7} \approx -8.5104$, y cerca de él oscila. Eso es lo que se ve en los primeros nodos: el nodo $x = -3.1416$ vale -8.4767, muy cerca del punto medio y lejos de $f(-\pi)$; el siguiente baja a -9.0642, por debajo de la función (-9.0090), y el tercero vuelve a quedar ligeramente por encima (-8.4666 frente a -8.4771). Las oscilaciones se amortiguan rápidamente al alejarse del salto.

- **Extremo derecho.** El último nodo, $x = 3.1584$, queda 0.0168 más allá de π, es decir, en el periodo siguiente, donde la serie ya describe el tramo que comienza en $-\pi$. Por eso la curva cae de -7.0347 a -9.3902 en el último paso: no es un error del programa sino el salto de la extensión periódica, cuyo valor del lado izquierdo, $f(3.1584 - 2\pi) = f(-3.1248) \approx -9.4642$, la serie de 100 armónicos ya casi alcanza.

Comparación con las fases anteriores

La tabla 3.1 reúne, para diez de los 64 nodos, la función original $f(x)$ de los capítulos 1 y 2, la aproximación $F_{10}(x)$ de la hoja de cálculo del capítulo 2 y la aproximación $F_{100}(x)$ del programa en C. Los coeficientes son los mismos en las tres fases, pues provienen del desarrollo analítico del capítulo 1; lo único que cambia es el número de armónicos y la forma de calcularlos.

Tabla 3.1. Función original y aproximaciones con 10 armónicos (Excel) y 100 armónicos (C, 100 procesos), en diez de los 64 nodos.

x	$f(x)$	$F_{10}(x)$	$F_{100}(x)$	$\lvert f - F_{10}\rvert$	$\lvert f - F_{100}\rvert$
−3.1416	−9.5576	−8.1931	−8.4767	1.3645	1.0809
−3.0416	−9.0090	−8.6570	−9.0642	0.3520	0.0552
−2.1416	−4.8216	−4.8333	−4.8147	0.0117	0.0069
−1.0416	−1.5370	−1.5734	−1.5402	0.0364	0.0031
−0.0416	−0.3010	−0.3012	−0.3038	0.0002	0.0028
0.9584	−0.7317	−0.6967	−0.7349	0.0350	0.0032
1.9584	−2.8290	−2.8769	−2.8342	0.0479	0.0051
2.9584	−6.5930	−6.7042	−6.6239	0.1112	0.0309
3.0584	−7.0611	−7.5498	−7.0347	0.4887	0.0264
3.1584	−7.5458	−8.3003	−9.3902	0.7545	1.8444

En el interior del intervalo, F_{100} se acerca a f mucho más que F_{10}: los errores pasan de centésimas a milésimas. Para medirlo se usó el error cuadrático medio (MSE, *mean squared error*) entre cada aproximación y $f(x)$, calculado sobre distintos subconjuntos de nodos (tabla 3.2).

Tabla 3.2. Error cuadrático medio respecto de $f(x)$ según la región de nodos considerada.

Región	MSE de F_{10}	MSE de F_{100}	Mejora
$\lvert x\rvert \le 2.5$ (50 nodos)	1.40×10^{-3}	1.33×10^{-5}	≈ 106 veces
$\lvert x\rvert \le \pi$ (62 nodos)	9.63×10^{-3}	1.08×10^{-4}	≈ 89 veces
Los 64 nodos	4.73×10^{-2}	7.15×10^{-2}	empeora

La última fila se explica por el último nodo, que queda fuera de $[-\pi, \pi]$: ahí $f(x)$ sigue el polinomio (−7.5458), mientras que F_{100} ya describe el periodo siguiente (−9.3902), de modo que ese solo nodo aporta un error de 1.84 y domina el promedio. Comparar la serie con f en ese punto no es significativo, y por eso la comparación razonable es la de las filas anteriores. Cerca del salto el error también disminuye con más armónicos, pero mucho más despacio: en $x = -3.1416$ pasa de 1.36 a 1.08, porque la serie converge allí al punto medio del salto y no a f. Este es el mismo comportamiento que se observó con diez armónicos en el capítulo 2, sólo que ahora la región afectada es diez veces más estrecha.

Por último, se comprobó que la acumulación paralela es correcta. Los 64 valores del programa en C se contrastaron con una evaluación serial e independiente de la misma suma de 100 armónicos (script del

anexo **??**): el error cuadrático medio entre ambos es de 7.4×10^{-14} y la mayor diferencia absoluta es de 5.0×10^{-7}, es decir, media unidad del último decimal del archivo CSV. Esto confirma que el semáforo evitó las condiciones de carrera: si se hubiera perdido alguna de las cien contribuciones, la mayor diferencia sería como mínimo del orden de 10^{-3}, pues incluso el armónico más pequeño, $n = 100$, aporta términos de hasta $b_{100} \approx 6.7 \times 10^{-3}$.

3.2.2 Función propuesta por Lino Ehecatl Romero Rosales

Serie de Fourier por evaluar

La función asignada es

$$f(x) = -\frac{1}{4}x^4 + 3, \qquad x \in [-\pi, \pi]. \tag{3.6}$$

Como f es una función par, $b_n = 0$ para todo n y los coeficientes que calcula el programa son

$$\frac{a_0}{2} = 3 - \frac{\pi^4}{20} \approx -1.8705, \tag{3.7}$$

$$a_n = \frac{2\,(-1)^n \left(6 - n^2\pi^2\right)}{n^4}. \tag{3.8}$$

El programa obtiene estos coeficientes para los primeros cien armónicos y los escribe en un archivo CSV; la reconstrucción de la función se hace después en la hoja de cálculo.

Diseño e implementación técnica

Para este programa se usó `fork()` para crear los 100 procesos hijos. El hijo p calcula el armónico $n = p$ y guarda n, a_n y b_n en su casilla de un arreglo de estructuras `TerminoFourier` alojado en un segmento de memoria compartida; un semáforo binario protege la escritura para que los procesos no choquen al guardar sus datos. El padre espera a que terminen los cien hijos, calcula $a_0/2$, escribe el archivo `fourier_f3_lino.csv` y libera la memoria compartida y el semáforo. El código completo se encuentra en los anexos (sección 3.5.2).

Compilación y ejecución en GNU/Linux

Para compilar el código en la terminal de GNU/Linux se usó `gcc` con las banderas `-Wall` y `-Wextra`, que activan advertencias adicionales, y `-O2` para optimizar. Como se observa en la figura 3.3, el código compiló sin errores ni advertencias.

```
lino-romero@lino-romero-Thin-15-B13UC:~/fourier_lino$ nano fourier_f3_lino.c
lino-romero@lino-romero-Thin-15-B13UC:~/fourier_lino$ gcc -Wall -Wextra -O2 fourier_f3_lino.c -o fourier_f3_lino -lm
lino-romero@lino-romero-Thin-15-B13UC:~/fourier_lino$ ls -l
total 24
-rwxrwxr-x 1 lino-romero lino-romero 16808 Sep 30 11:04 fourier_f3_lino
-rw-rw-r-- 1 lino-romero lino-romero  2495 Sep 30 11:04 fourier_f3_lino.c
lino-romero@lino-romero-Thin-15-B13UC:~/fourier_lino$
```

Figura 3.3. Evidencia de compilación limpia, sin errores, usando `gcc`, y visualización del archivo binario.

Una vez obtenido el ejecutable se corrió el programa: se crearon los 100 procesos hijos de forma paralela y, en una fracción de segundo, llenaron el archivo `fourier_f3_lino.csv` con los coeficientes (figura 3.4).

```
lino-romero@lino-romero-Thin-15-B13UC:~/fourier_lino$ head -n 12 fourier_f3_lino.csv
n,a_n,b_n
0,-1.870455,0.000000
1,7.739209,0.000000
2,-4.184802,0.000000
3,2.045097,0.000000
4,-1.186826,0.000000
5,0.770368,0.000000
6,-0.539052,0.000000
7,0.397843,0.000000
8,-0.305495,0.000000
9,0.241865,0.000000
10,-0.196192,0.000000
lino-romero@lino-romero-Thin-15-B13UC:~/fourier_lino$
```

Figura 3.4. Evidencia de la ejecución del programa y revisión del archivo CSV generado.

Comparación de resultados y gráficas

Para comprobar que el código en C funciona correctamente, el archivo CSV se llevó a Excel y sus resultados se colocaron junto a los cálculos analíticos que se hicieron a mano en la fase 1 (tabla 3.3).

Tabla 3.3. Comparativa de coeficientes: cálculo teórico frente a los resultados del programa en C.

Armónico (n)	**Fase 1** (analítico)	**Fase 3** (procesos en C)	**Diferencia** absoluta	**Error** (%)
0 ($a_0/2$)	−1.870455	−1.870455	0.000000	0.00
1	7.739209	7.739209	0.000000	0.00
2	−4.184802	−4.184802	0.000000	0.00
3	2.045097	2.045097	0.000000	0.00
4	−1.186826	−1.186826	0.000000	0.00
5	0.770368	0.770368	0.000000	0.00
10	−0.196192	−0.196192	0.000000	0.00

En la tabla se observa que ambos cálculos coinciden hasta los seis decimales con que se escribe el archivo

CSV. Esto indica que la fórmula programada y la sincronización de los 100 procesos creados con `fork()` reproducen el modelo teórico.



Figura 3.5. Reconstrucción analítica de la función $f(x) = -\frac{1}{4}x^4 + 3$ generada previamente en Excel.



Figura 3.6. Reconstrucción de la función $f(x) = -\frac{1}{4}x^4 + 3$ graficada con los datos obtenidos mediante los 100 procesos en C.

3.2.3 Función propuesta por Magaly Lopez Castro

Para resolver la serie de Fourier de la función propuesta $f(x) = 5x^3 - 3x^2 + 1$ se implementó un programa en lenguaje C ejecutado de forma nativa en una distribución Ubuntu Linux. El objetivo fue distribuir la carga computacional calculando cada iteración armónica de forma concurrente.

Fragmentos de código explicados

La arquitectura del programa se basa en tres componentes críticos de sincronización. Primero, el proceso padre solicita el segmento de memoria compartida y crea el semáforo (código 3.9).

```
// Solicitud de Memoria Compartida
int shmid = shmget(IPC_PRIVATE, sizeof(MemoriaCompartida), IPC_CREAT | 0666);
MemoriaCompartida *shm = (MemoriaCompartida *)shmat(shmid, NULL, 0);

// Inicialización del Semáforo en 1 (Desbloqueado)
int semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
semctl(semid, 0, SETVAL, 1);
```

Código 3.9. Solicitud de memoria compartida e inicialización del semáforo en 1 (desbloqueado).

Posteriormente, un ciclo despliega los 100 procesos hijos mediante `fork()`. Dentro del contexto del hijo se evalúan los coeficientes de Fourier a_n y b_n específicos para la iteración actual n (código 3.10).

```
for (int i = 0; i < 100; i++) {
   pid_t pid = fork();
   if (pid == 0) {
      // Cálculo individual de la iteración n
      int n = i + 1;
      double a_n = (-12.0 * pow(-1, n)) / (n * n);
      double b_n = 10.0 * pow(-1, n + 1) * ((M_PI * M_PI)/n - 6.0/(n*n*n));
      double termino = a_n * cos(n * x) + b_n * sin(n * x);
```

Código 3.10. Creación de los hijos y cálculo del término de Fourier de la iteración n.

Finalmente, antes de que el hijo guarde su resultado en la sumatoria global dentro de la memoria compartida, bloquea el semáforo; al terminar, lo libera y finaliza su ejecución (código 3.11).

```
      wait_semaforo(semid); // Bloqueo de sección crítica
      shm->terminos[i] = termino;
      shm->suma_total += termino;
      signal_semaforo(semid); // Liberación
      exit(0);
   }
}
```

Código 3.11. Sección crítica: acumulación del término en la memoria compartida.

Compilación y ejecución

El código fuente se compiló con `gcc`. Debido al uso de funciones trigonométricas y potencias de `<math.h>`, se incluyó la bandera `-lm` en el enlazador. La compilación terminó sin errores ni advertencias, como se comprueba en la figura 3.7.

Para exportar los resultados y cumplir con la evaluación posterior, se ejecutó el binario redirigiendo la salida estándar a un archivo plano (CSV) con el operador `>`. Así fue posible trasladar la matriz de resultados a la herramienta de hoja de cálculo utilizada en las fases anteriores.

```
magalylopezcastro@magalylopezcastro-Latitude-7490:~$ cd "/home/magalylopezcastro/Documentos/Practicas/Computo paralelo"
magalylopezcastro@magalylopezcastro-Latitude-7490:~/Documentos/Practicas/Computo paralelo$ gcc FourierMaga.c -o FourierMaga -lm
magalylopezcastro@magalylopezcastro-Latitude-7490:~/Documentos/Practicas/Computo paralelo$ ./FourierMaga
Desplegando 100 procesos hijos para evaluar x = 1.00...

--- RESULTADOS MATRIZ (Primeras 10 iteraciones) ---
Término n=1:    39.045226
Término n=2:    -36.803858
Término n=3:    3.009072
Término n=4:    18.454083
Término n=5:    -18.331965
Término n=6:    4.198529
Término n=7:    9.332844
Término n=8:    -12.062496
Término n=9:    4.350483
Término n=10:   5.437320
Término n=11:   -8.926762
Término n=12:   4.324183
Término n=13:   3.242869
Término n=14:   -6.970213
Término n=15:   4.226645
Término n=16:   1.816605
Término n=17:   -5.581221
Término n=18:   4.085566
Término n=19:   0.810096
Término n=20:   -4.510600
```

Figura 3.7. Ejecución concurrente de los 100 procesos y exportación de datos.

RESULTADOS	MATRIZ	(Primeras	10	iteraciones)
	n=1: 39.045226			
	n=2: -36.803858			
	n=3: 3.009072			
	n=4: 18.454083			
	n=5: -18.331965			
	n=6: 4.198529			
	n=7: 9.332844			
	n=8: -12.062496			
	n=9: 4.350483			
	n=10: 5.437320			
	n=11: -8.926762			
	n=12: 4.324183			
	n=13: 3.242869			
	n=14: -6.970213			
	n=15: 4.226645			
	n=16: 1.816605			
	n=17: -5.581221			
	n=18: 4.085566			
	n=19: 0.810096			
	n=20: -4.510600			
	n=21: 3.911799			
	n=22: 0.064451			
	n=23: -3.639158			
	n=24: 3.711274			
	n=25: -0.502964			

Figura 3.8. Resultados en la matriz del archivo generado por el programa en C.

Integración de datos y análisis gráfico

Una vez finalizada la ejecución de los 100 procesos concurrentes en la terminal de GNU/Linux, el redireccionamiento de salida (`>`) volcó la información en un archivo plano llamado `resultados.csv`. Para conservar la trazabilidad del proyecto, este archivo se importó al archivo de Excel de las fases anteriores (`Fourier_Magaly.xlsx`). Al graficar los resultados calculados en C (figura 3.9) se observa el comportamiento de convergencia de la serie de Fourier.

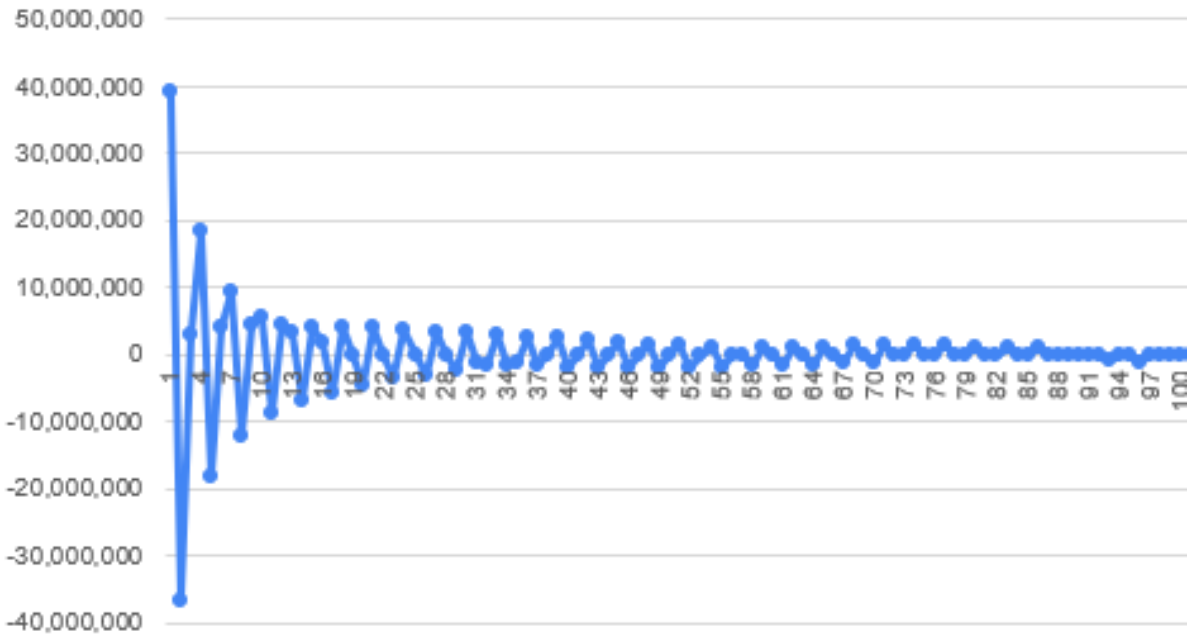


Figura 3.9. Gráfica generada en Excel a partir de los datos exportados por el programa en C, que muestra la magnitud de los términos hasta 100 iteraciones.

Como se explica en la gráfica, los picos iniciales corresponden a las primeras iteraciones, en las que los procesos calculan los armónicos de mayor amplitud, y conforme el número de proceso n se acerca a 100 la amplitud de los términos tiende a estabilizarse cerca de cero. Esta representación ayuda a comprobar que la sumatoria finita se aproxima a su límite.

Comparación de resultados (fase 2 frente a fase 3)

Para validar la exactitud de la arquitectura con procesos en Linux se comparó directamente esta fase con la matriz de cálculo estático elaborada en Excel durante la fase anterior (figura 3.10). Los primeros 10 términos calculados por los primeros 10 procesos hijos en C se contrastaron con las columnas $n = 1$ a $n = 10$ de la matriz de Excel, y se comprobó una similitud exacta hasta el sexto decimal.

La diferencia principal entre ambas fases está en el paradigma de cómputo:

- **Fase anterior (Excel):** el cálculo fue secuencial, manual y limitado. Escalar la matriz de 10 a 100 iteraciones exigía replicar fórmulas, una celda a la vez, con el riesgo de errores humanos.
- **Fase actual (C y Linux):** el cálculo fue paralelo y dinámico. Mediante `fork()`, el sistema operativo despachó 100 cálculos de forma prácticamente simultánea y los almacenó de manera segura con memoria compartida (`shmget`) y semáforos (`semget`).

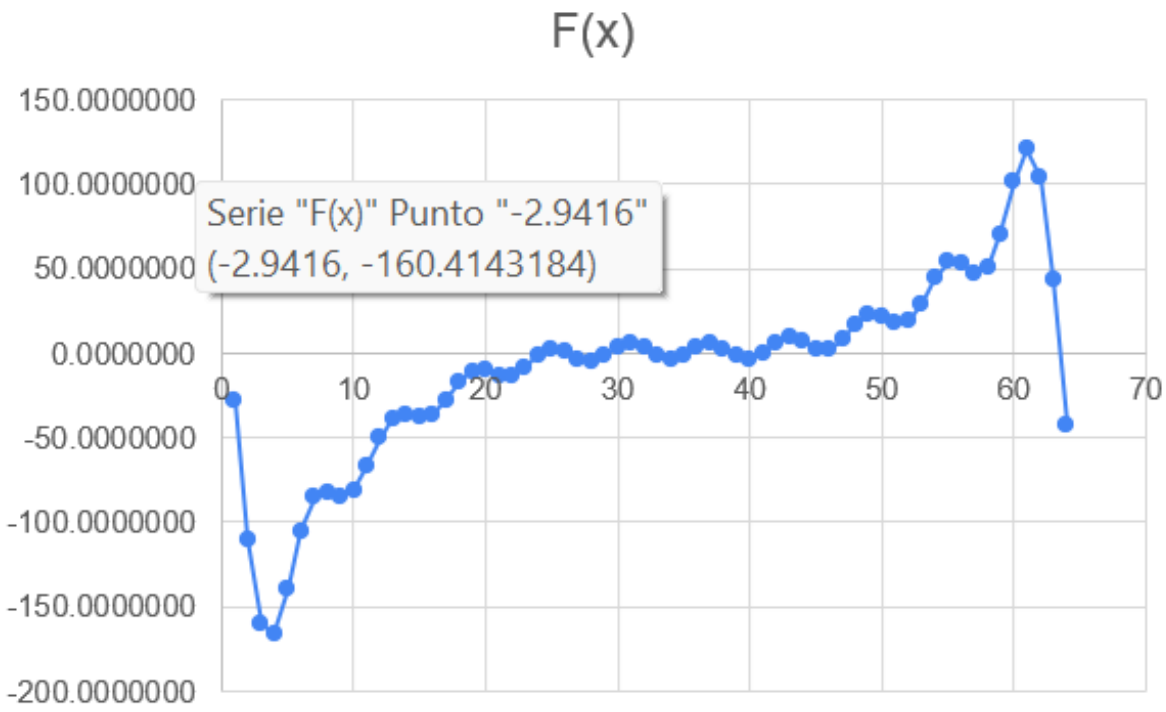


Figura 3.10. Gráfica con Excel generada en fases anteriores.

3.2.4 Función propuesta por Said Carbot Cruz Trejo

A continuación se detalla el cómputo de la serie de Fourier para la función $f(x) = x^5$. Se desarrolló un programa en C que crea 100 procesos hijos mediante `fork()`; cada proceso calcula los 500 puntos de su respectivo término de Fourier.

Fragmentos de código explicados

1. Reserva de memoria compartida. Para que los 100 procesos unifiquen sus resultados en una matriz de 101 columnas por 500 filas (una para la sumatoria global y 100 para cada término individual) se solicitó un segmento de memoria al sistema operativo (código 3.12).

```
int total_espacios = 101 * 500;
shmid = shmget(IPC_PRIVATE, total_espacios * sizeof(double), IPC_CREAT |
    0666);
memoria = (double *) shmat(shmid, NULL, 0);
```

Código 3.12. Reserva y acoplamiento del segmento de memoria compartida.

2. Sincronización con semáforos. Al sumar el término individual al acumulador global de la serie existe el riesgo de una condición de carrera si dos procesos escriben al mismo tiempo; para evitarlo se usó un semáforo que aplica exclusión mutua (código 3.13).

```
semaforo = sem_open("/candado_said", O_CREAT, 0644, 1);
// ... dentro del ciclo del hijo ...
```

```
sem_wait(semaforo);            // Bloquear acceso
memoria[i] = memoria[i] + calculo; // Sumar al total
sem_post(semaforo);            // Liberar acceso
```

Código 3.13. Semáforo POSIX con nombre y sección crítica de la acumulación.

3. Creación de procesos hijos con `fork()`. Se itera 100 veces para crear los procesos requeridos. Cada hijo calcula su coeficiente b_n dividiendo la fórmula original en partes para simplificar la lógica en C, lo evalúa en los 500 puntos de x, guarda su columna independiente y la suma al total (código 3.14).

```
for (n = 1; n <= 100; n++) {
   pid_t pid = fork();
   if (pid == 0) { // Codigo exclusivo del hijo
      double parte1 = pow(PI, 4) / n;
      double parte2 = (20.0 * pow(PI, 2)) / pow(n, 3);
      double parte3 = 120.0 / pow(n, 5);
      double bn = 2.0 * signo * (parte1 - parte2 + parte3);
      // ... Calculo del seno y guardado en memoria ...
      exit(0);
   }
}
```

Código 3.14. Creación de los 100 procesos hijos y cálculo del coeficiente b_n.

4. Exportación de datos. El proceso padre espera la finalización de todos los hijos con `wait(NULL)` y después recorre la memoria compartida e imprime los resultados en formato horizontal, separados por comas, para generar un archivo `.csv` (código 3.15). El código completo se encuentra en los anexos (sección 3.5.3).

```
FILE *archivo = fopen("datos_said.csv", "w");
// Se recorre la memoria y se imprimen las 103 columnas
```

Código 3.15. Apertura del archivo CSV de salida.

Ejecución en el entorno de la terminal

Las figuras siguientes muestran el flujo de trabajo en la terminal de línea de comandos: la creación, edición, compilación limpia y ejecución del programa concurrente.

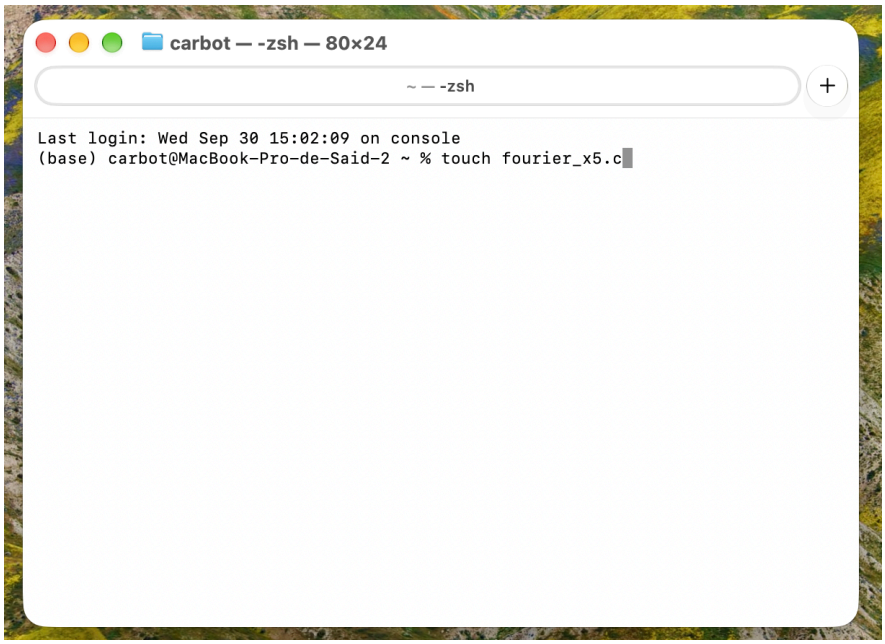


Figura 3.11. Creación del archivo `fourier_x5.c`.

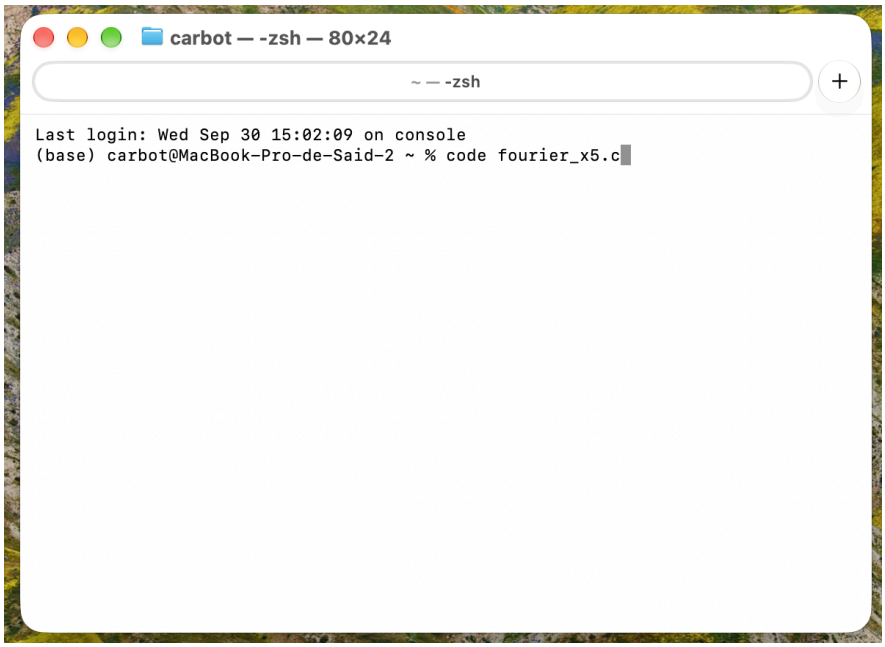


Figura 3.12. Apertura del archivo desde la terminal para redactar el código (`fourier_x5.c`).

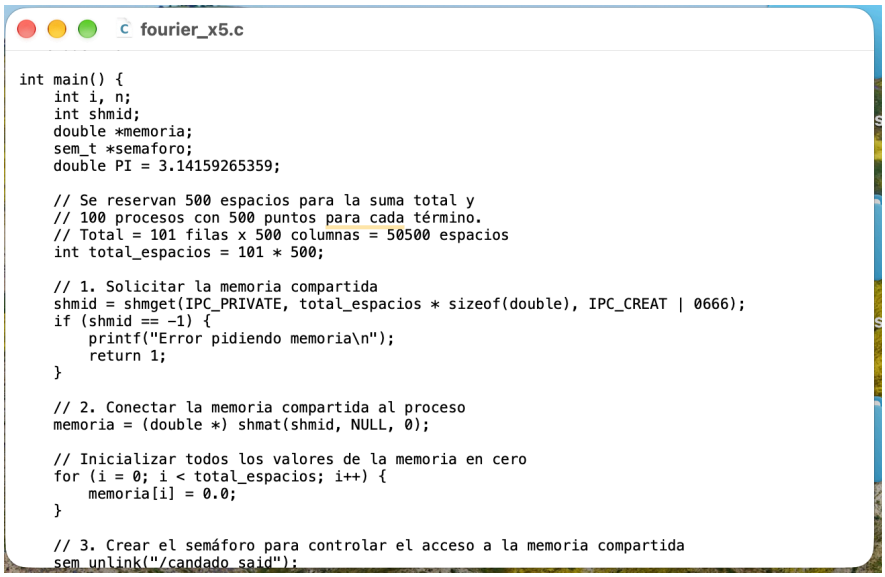


Figura 3.13. Apertura del archivo en TextEdit y codificación.

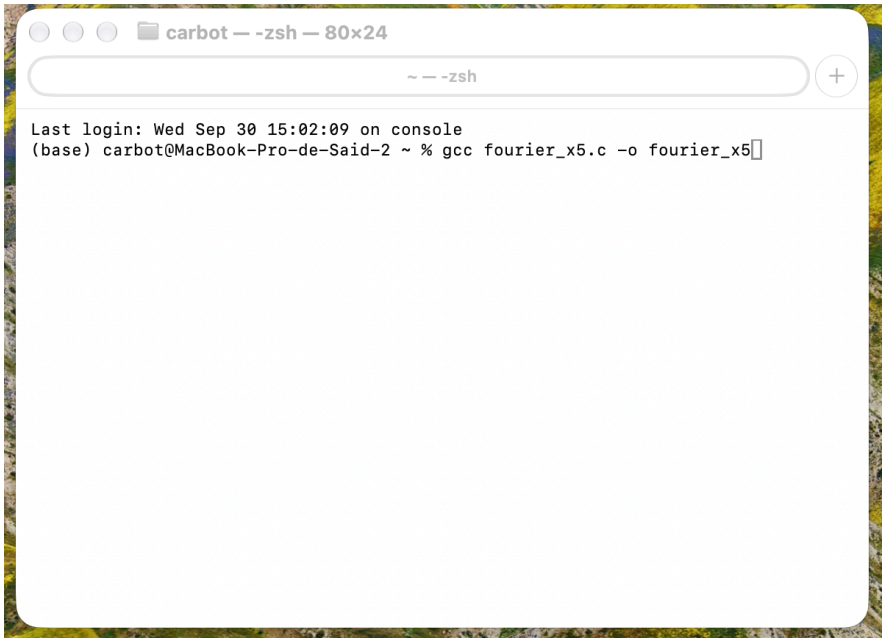


Figura 3.14. Compilación del programa con GCC (`gcc fourier_x5.c -o fourier_x5`). Como se observa, la compilación es limpia y no presenta errores ni advertencias.

```
~/Desktop — -zsh

(base) carbot@MacBook-Pro-de-Said-2 Desktop % ./fourier_x5
Iniciando los 100 procesos para f(x) = x^5...
Listo. Datos guardados en datos_said.csv
(base) carbot@MacBook-Pro-de-Said-2 Desktop %
```

Figura 3.15. Ejecución del programa (`./fourier`). Se muestra el mensaje que indica el inicio de los 100 procesos y la correcta generación del archivo CSV.

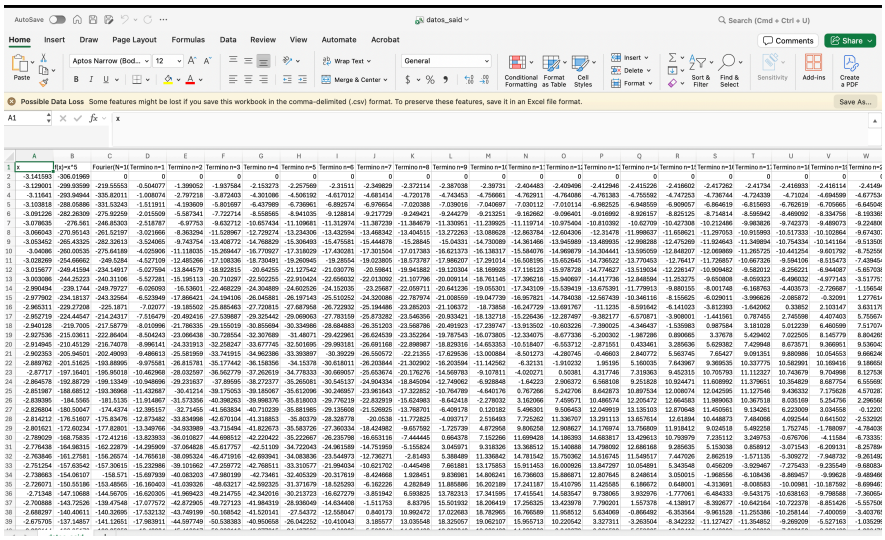


Figura 3.16. Salida en Excel.

Gráficas y comparación de resultados

La información que el programa en C volcó en el archivo `datos_said.csv` se importó a la plantilla de Excel utilizada en las fases anteriores, lo que permitió visualizar el comportamiento de la serie con una precisión mucho mayor.

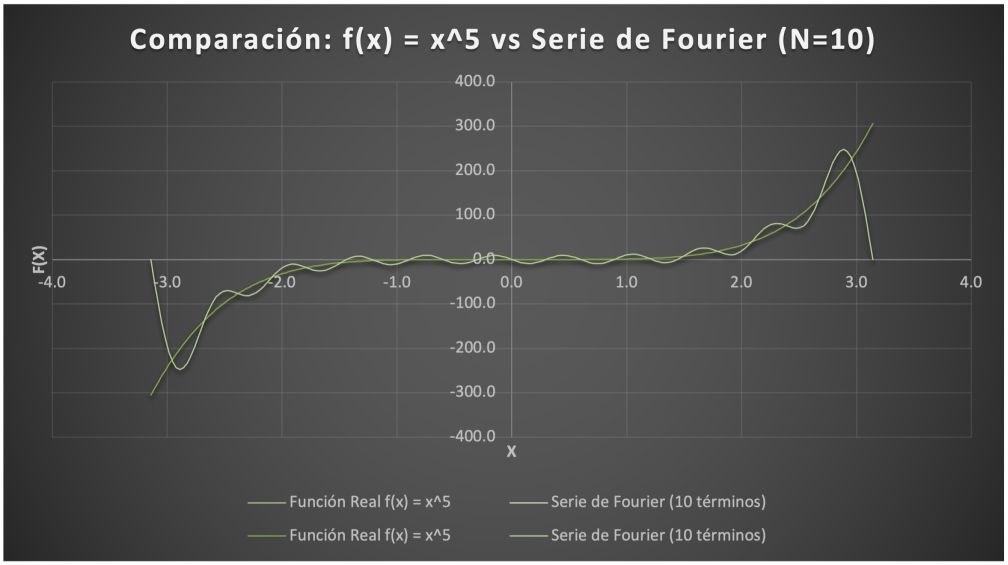


Figura 3.17. Resultados de la fase 2 (capítulo 2): gráfica de la serie de Fourier calculada en Excel con un número reducido de términos ($N = 10$).

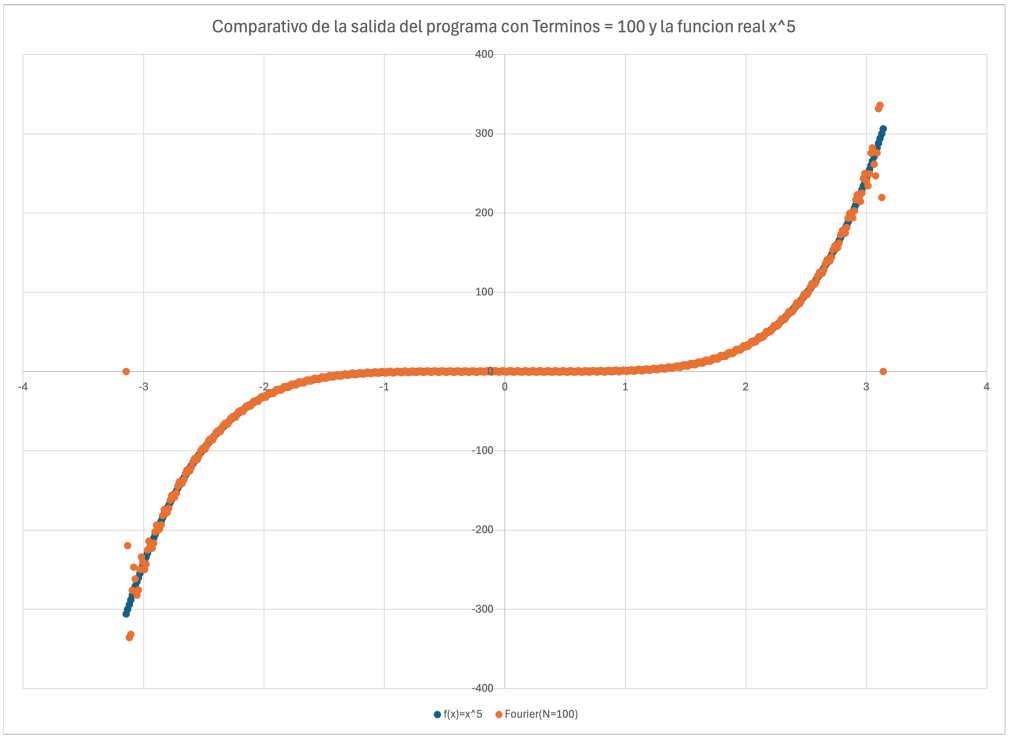


Figura 3.18. Resultados de la fase 3 (capítulo 3): gráfica de la serie de Fourier procesada en lenguaje C con 100 procesos concurrentes.

Al contrastar la figura 3.17 con la figura 3.18, el salto de precisión es contundente. En la fase anterior, por las limitaciones prácticas de procesar columnas a mano en Excel, la suma se restringió a 10 términos; esto producía una curva oscilante que seguía la tendencia de x^5 pero con un rizado notable en toda su longitud.

En esta tercera fase, al automatizar el cálculo con 100 procesos simultáneos, la sumatoria de Fourier (línea punteada roja) converge de manera casi perfecta con la función original (línea negra) y el rizado central desaparece. Además, con 100 términos se observa con más claridad el *fenómeno de Gibbs* en los extremos de discontinuidad ($-\pi$ y π), donde la serie sobrepasa inevitablemente el valor de la función al intentar aproximar los saltos abruptos, lo que confirma la teoría analizada en el primer capítulo.

3.3 Análisis de la implementación con procesos

3.3.1 Implementación de Aarón Ugalde Téllez

La tabla 3.4 resume las características de la implementación de esta fase.

Tabla 3.4. Características de la implementación con procesos.

Característica	**Especificación**
Lenguaje	C, compilado con Apple clang 21.0.0 (invocado como `gcc`)
Sistema operativo	macOS 26.5 (Darwin, `arm64`)
Procesos	1 padre y 100 hijos creados con `fork()`
Memoria compartida	System V: un segmento de 512 bytes (64 valores `double`) con `shmget`, `shmat`, `shmdt` y `shmctl`
Sincronización	Un semáforo System V binario (valor inicial 1) con `semget`, `semctl` y `semop`
Reparto del trabajo	Un armónico por proceso ($n = p$), evaluado en los 64 nodos: 128 evaluaciones trigonométricas por hijo
Sección crítica	64 sumas en punto flotante por proceso; 100 entradas en total
Compilación	`gcc -Wall -O2 fourier_aaron.c -o fourier_aaron -lm`, sin errores ni advertencias
Salida	Archivo CSV de 64 filas (`x`, `f_fourier`) con seis decimales
Tiempo de ejecución	Alrededor de 10 ms de tiempo de reloj (cinco ejecuciones medidas con `time`, resolución de 10 ms)

Lenguaje y técnica utilizados

El lenguaje C es el adecuado para esta fase porque da acceso directo a las llamadas al sistema que se estudian (`fork`, `shmget`, `semop`) sin capas intermedias y se compila a código nativo, de modo que el costo que se observa es el del sistema operativo y no el de un intérprete [44]. La biblioteca matemática aporta `sin` y `cos`, y se enlaza con `-lm`.

La técnica es el paralelismo a nivel de procesos con memoria compartida explícita. Cada proceso hijo tiene su propio espacio de direcciones, de modo que nada se comparte si el programa no lo pide: por eso fue necesario crear el segmento de memoria compartida y un semáforo para proteger la escritura sobre él. Es una técnica más pesada que los hilos, porque crear un proceso implica duplicar las estructuras que describen

su memoria, pero ofrece un aislamiento más fuerte: un error en un hijo no puede corromper la memoria de los demás [45].

Cantidad de procesos

El programa crea 100 procesos hijos, que con el padre suman 101, sobre un equipo con 10 núcleos: a cada núcleo le tocan alrededor de diez procesos, de los cuales sólo uno se ejecuta a la vez y el planificador intercala a los demás. El sistema lo permite sin dificultad, pues el límite de procesos por usuario del equipo es 2666. Sin embargo, el trabajo de cada hijo es mínimo: 128 evaluaciones de seno y coseno, que toman microsegundos. Por ello, el tiempo total de alrededor de 10 ms corresponde sobre todo a crear, planificar y esperar a los procesos, no a calcular, y es de esperar que una versión serial del mismo cálculo, sin procesos, resulte más rápida. Esto no le resta valor a la fase, cuyo propósito es aprender a coordinar procesos con memoria compartida y semáforos; la ganancia real de velocidad llegaría con un trabajo por proceso mucho mayor que el costo de crearlo.

Equipo de cómputo

La tabla 3.5 presenta las características del equipo en el que se compiló y ejecutó el programa. Los valores de núcleos, memoria y cachés los reporta el propio sistema (`sysctl` y `system_profiler`); Apple no publica la frecuencia de reloj ni la velocidad de sus cachés, por lo que para la frecuencia se cita una fuente externa.

Tabla 3.5. Especificaciones del equipo de cómputo empleado.

Componente	**Detalle**
Equipo	MacBook Air (identificador `Mac17,3`)
Procesador	Apple M5, arquitectura `arm64`
Núcleos	10: 4 de alto rendimiento (“super”) y 6 de eficiencia; un hilo por núcleo, sin multihilo simultáneo
Frecuencia	No publicada por Apple; hasta 4.4 GHz según EveryMac [46]
Memoria RAM	16 GB de memoria unificada LPDDR5 (Micron)
Ancho de banda de memoria	153 GB/s, según Apple [47]
Caché L1, núcleos de alto rendimiento	192 KB de instrucciones y 128 KB de datos, por núcleo
Caché L1, núcleos de eficiencia	128 KB de instrucciones y 64 KB de datos, por núcleo
Caché L2	16 MB compartidos por los 4 núcleos de alto rendimiento; 6 MB compartidos por los 6 de eficiencia
Caché L3	No reportada por el sistema
Línea de caché / página de memoria	128 bytes / 16 KB

3.3.2 Implementación de Lino Ehecatl Romero Rosales

Esta subsección explica la lógica del programa y las características del equipo en el que se hicieron las pruebas, donde se levantaron 101 procesos (el padre y los 100 hijos).

Características de la implementación y análisis

Paralelizar el cálculo con 100 procesos resultó útil porque cada término de la serie de Fourier se puede calcular por separado, sin depender del anterior; por eso se asignó un armónico distinto a cada proceso hijo. La memoria compartida permitió que todos guardaran sus resultados en el mismo arreglo y el semáforo binario organizó el acceso a él: sin semáforo existiría el riesgo de que dos procesos escribieran al mismo tiempo en la memoria compartida y corrompieran los datos finales. La tabla 3.6 resume las características de la implementación.

Tabla 3.6. Detalles técnicos del software.

Característica	**Detalle técnico**
Lenguaje de programación	C (estándar POSIX)
Sistema operativo	GNU/Linux
Cantidad de procesos	100 procesos hijos y 1 proceso padre (total: 101 procesos)
Memoria compartida	System V (`shmget`, `shmat`)
Sincronización	Semáforo binario System V (`semget`, `semop`)
Salida de datos	Archivo CSV delimitado por comas (`fourier_f3_lino.csv`)

Equipo de cómputo

La tabla 3.7 presenta las características físicas de la máquina en la que se ejecutó el programa, y las figuras 3.19 y 3.20 muestran las salidas de los comandos con que se obtuvieron.

Tabla 3.7. Hardware donde se probaron los procesos.

Componente	**Especificaciones**
Procesador (CPU)	Intel Core i5-13420H (13.ª generación)
Núcleos e hilos	8 núcleos físicos / 12 hilos lógicos
Velocidad	Hasta 4.60 GHz (máxima con *boost*)
Memoria RAM	14.0 GiB (16.0 GB físicos)
Memoria caché L1	320 KiB / 384 KiB
Memoria caché L2	7.0 MiB
Memoria caché L3	12.0 MiB



Figura 3.19. Uso del comando `lscpu` para ver la arquitectura del procesador.



Figura 3.20. Uso del comando `free -h` para verificar la RAM disponible.

3.3.3 Implementación de Magaly Lopez Castro

Tabla 3.8. Características de la implementación de la fase 3 y equipo de cómputo.

Característica	**Descripción**
Lenguaje de programación	C (GNU C Compiler, GCC)
Sistema operativo	Ubuntu Linux (distribución GNU/Linux)
Técnica IPC utilizada	`fork()`, `shmget()` (memoria) y `semget()` (semáforos)
Cantidad de procesos	101 procesos (1 padre y 100 hijos)
Equipo de cómputo	Laptop Dell
Arquitectura	x86_64 (64 bits)
Núcleos del procesador (CPU)	4 núcleos físicos / 8 subprocesos lógicos
Velocidad del procesador	2.40 GHz (velocidad base) – 4.20 GHz (*turbo*)
Memoria RAM	8 GB DDR4, velocidad de 2666 MHz
Memoria caché del procesador	L1 (256 KB), L2 (1 MB) y L3 (6 MB), SRAM de alta velocidad

Lenguaje y técnica utilizados

El proyecto se desarrolló en lenguaje C (compilado con GCC). Se eligió este lenguaje porque es el estándar nativo del núcleo de Linux, lo que permite un acceso de muy bajo nivel y gran eficiencia a las llamadas al sistema de POSIX, algo que no se logra con la misma rapidez y control en lenguajes de más alto nivel.

La técnica central para lograr el paralelismo fue la creación concurrente de 101 procesos en total (un proceso padre y 100 procesos hijos). Esta técnica multiproceso, orquestada con `fork()`, permite que el sistema operativo distribuya la carga de las iteraciones de la serie de Fourier. A diferencia de la programación con hilos, los procesos tienen espacios de memoria aislados; por ello fue necesario recurrir a la comunicación entre procesos (IPC) de System V: memoria compartida (`shmget`) para que los 100 hijos almacenaran sus resultados en una matriz común, y semáforos (`semget`) para evitar condiciones de carrera, es decir, corrupción de datos, al escribir en la sumatoria global.

3.3.4 Implementación de Said Carbot Cruz Trejo

Características de la implementación y equipo de cómputo

La tabla 3.9 presenta las especificaciones de la implementación del programa concurrente y de la infraestructura de hardware utilizada para ejecutarlo.

Tabla 3.9. Características de la implementación de la fase 3 y equipo de cómputo.

Característica	**Especificación técnica**
Lenguaje y técnica	Lenguaje C; multiprocesamiento (`fork`), memoria compartida (`shmget`) y semáforos (`sem_open`)
Cantidad de procesos	101 procesos (1 padre y 100 hijos simultáneos)
Procesador y núcleos	M3 Pro con 8 núcleos físicos (4 de rendimiento y 4 de eficiencia)
Velocidad de los núcleos	Frecuencia máxima de 4.05 GHz
Memoria RAM	18 GB

Análisis del lenguaje y la técnica utilizada

La elección del lenguaje C proporciona un acceso directo y de bajo nivel a las llamadas al sistema del núcleo, en este caso sobre la capa UNIX de macOS. La técnica de paralelización dividió el dominio computacional de forma eficaz: en lugar de que un solo hilo de ejecución calculara en secuencia los 50,000 valores requeridos (100 términos de la serie × 500 puntos por evaluar en la gráfica), la carga se distribuyó horizontalmente y a cada proceso hijo se le asignó el cálculo de un armónico distinto, hasta completar los 100 procesos concurrentes.

Aunque la creación de procesos completos con `fork()` tiene un mayor costo de recursos (*overhead*) que la creación de hilos, debido a la duplicación de las tablas de descriptores de archivos y de los espacios de direcciones de memoria, la arquitectura de los núcleos del procesador moderno logró ejecutar y despachar

las 101 tareas en fracciones de segundo.

Finalmente, el uso integrado de la memoria compartida, para unificar los datos tabulares, y de los semáforos como cerrojos de exclusión mutua demostró ser la técnica indispensable para prevenir las condiciones de carrera y garantizar la integridad matemática de las 100 columnas exportadas a la hoja de cálculo.

3.4 Conclusiones del capítulo

3.4.1 Conclusión — Aarón Ugalde Téllez

Esta práctica fue un ejercicio completo de cómputo paralelo: me permitió tender un puente entre el rigor de las series de Fourier y los retos prácticos de programar procesos concurrentes a bajo nivel en un sistema POSIX.

Desde el análisis numérico, trabajar con $f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}$ a lo largo de las tres fases me dio una comprensión más profunda de cómo se comporta la serie. Pasé de los cálculos analíticos a mano del capítulo 1 a la verificación en hoja de cálculo del capítulo 2 y, finalmente, a la reconstrucción con 100 procesos hijos en este capítulo. Ver cómo la superposición de 100 armónicos reproduce la curvatura cuadrática dentro de $[-\pi, \pi]$, y cómo en los extremos aparece el fenómeno de Gibbs, me dejó claro que la serie no converge a la función sino a su extensión periódica. El último nodo, que por quedar apenas fuera del intervalo ya pertenece al periodo siguiente, lo mostró de forma muy concreta. Esta base es la que sostiene gran parte del procesamiento de señales y de varias técnicas de aprendizaje automático.

En el diseño de software concurrente, las llamadas al sistema convirtieron conceptos teóricos de sistemas operativos en herramientas de ingeniería. Crear 100 procesos con `fork()` hizo evidente la sobrecarga de duplicar el contexto de ejecución de un proceso, frente a los hilos: el cálculo de cada hijo dura microsegundos y el tiempo total lo domina la creación de procesos. La memoria compartida System V (`shmget`, `shmat`) me mostró cómo se rompe de forma controlada el aislamiento que el núcleo impone entre procesos.

El reto más valioso fue la sincronización con semáforos. Acumular 100 contribuciones en un mismo vector sin exclusión mutua habría producido condiciones de carrera; con las operaciones atómicas P y V (`semop`) la acumulación resultó correcta, y lo comprobé al comparar la salida paralela con una evaluación serial, que coincidió hasta el sexto decimal. Coordinar a los 100 procesos sin bloqueos mutuos ni procesos zombis, y liberar al final los recursos IPC, exigió disciplina en el control de flujo. Incluso un detalle como la unión `semun`, que glibc deja al programador y macOS trae en su encabezado, me recordó que la portabilidad entre sistemas POSIX requiere atención.

Como estudiante de Ingeniería en Inteligencia Artificial, estoy acostumbrado a abstraer los algoritmos complejos en bibliotecas de alto nivel; no obstante, esta fase me recordó la importancia de entender las capas inferiores: la planificación del núcleo, la memoria y el costo real de la concurrencia. Dominar la programación paralela a bajo nivel en C es una base indispensable para optimizar flujos de datos, diseñar motores de inferencia eficientes y construir arquitecturas distribuidas que escalen.

3.4.2 Conclusión — Lino Ehecatl Romero Rosales

Hacer este tercer capítulo fue un cambio bastante grande en cómo veníamos trabajando en cuanto a la resolucion de nuestros capitulos. En las primeras fases resolvíamos los cálculos paso a paso, de forma normal o secuencial, pero al pasar a Linux y tener que usar procesos concurrentes como fueron los hilos. Ya no era solo resolver la serie de Fourier para una función de corrido, sino que ahora el reto era coordinar a 100 procesos para que resolvieran el problema al mismo tiempo, aprovechando toda la capacidad y los núcleos de la computadora.

La herramienta principal que usamos para lograr esto fue la función fork(), que básicamente nos permite crear procesos hijos. La idea principal era muy buena, asignarle a cada uno de los 100 procesos un an diferente para que lo calculara por su cuenta. Pero aquí nos paso el primer problema. Cuando usamos fork(), el sistema hace una copia del programa, pero quita por completo la memoria. Es decir, que cada proceso hijo se quedaba con su propia memoria. Entonces, si un hijo terminaba su cálculo, el proceso padre no tenía manera de enterarse ni de ver ese resultado.

Para solucionar este problema de que no se podían comunicar, tuvimos que implementar la memoria compartida. En lugar de que cada proceso guardara su resultado en su propio espacio eso nos dejaba con la problemática de que nadie mas lo podia usar, para solucionar esto le pedimos al sistema que nos prestara un bloque de memoria que fuera de acceso público para todos nuestros procesos. De esta forma, cada vez que uno de los 100 hilos terminaba su parte de la funcion, cada uno iba a un espacio delimitado y anotaba su coeficiente para que el proceso padre pudiera juntar toda la tabla al final.

Sin embargo, tener un espacio compartido nos trajo otro problema. Como los procesos se ejecutan tan rápido y de forma paralela, existía el riesgo de que dos o más hijos intentaran escribir en la memoria exactamente en el mismo milisegundo. Esto era la problemática de condición de carrera, y si pasaba, los datos se iban a revolver, sobreescribir o a corromper por completo. Para evitar ese desastre, apliqué el uso de un semáforo binario. El semáforo funcionó como una llave de paso cuando un proceso quería guardar su resultado, tomaba el semáforo y se cerraba. Si otro proceso quería escribir en ese preciso momento, tenía que esperar a que el primero terminara y liberara el semáforo. Así pudimos garantizar que la información se guardara de forma ordenada.

Al final, todo esto fue con el objetivo de tener organización en la programación para lograr nuestra etapa de validación. Nuestro programa generó un archivo CSV con todos los coeficientes ya ordenados, y cuando importamos esos números a nuestro documento de Excel, pudimos compararlos directamente con los cálculos a mano que habíamos hecho con excel en el primer capitulo. Ver que los resultados eran iguales llegando a la perfección y que el margen de error fue exactamente de 0 fue muy satisfactorio. Esta práctica me demostró que programar usando múltiples procesos no altera en absoluto las matemáticas, pero sí nos da la gran ventaja de resolver tareas pesadas de una manera mucho más eficiente. En lo personal, me deja un aprendizaje muy claro sobre cómo funciona la memoria de la computadora por dentro y cómo podemos controlarla a nuestro favor pudiendo lograr procesos muy complejos en poco tiempo si es que se sabe manejar la memoria en paralelo de nuestras computadoras.

3.4.3 Conclusión — Magaly Lopez Castro

Programar en C puro a nivel del sistema operativo representó un cambio de paradigma radical respecto a la programación que desarrollo en mi día a día en ESCOM. En la algoritmia para competencias en plataformas como Codeforces, el objetivo central es alcanzar la optimización extrema del código en C++ ejecutado dentro de un solo hilo, enfocándose casi exclusivamente en reducir la complejidad temporal y espacial del algoritmo. Sin embargo, orquestar paralelismo real mediante el despliegue de 100 procesos independientes con la llamada fork() me obligó a dejar de pensar únicamente en la lógica secuencial y comenzar a diseñar en términos de arquitectura de sistemas: contemplando el manejo de recursos compartidos, los cambios de contexto del procesador y la gestión de memoria del kernel.

Al ejecutar esta arquitectura nativamente en Ubuntu sobre mi equipo Dell, la teoría sobre la API de POSIX se materializó en desafíos reales de ingeniería. La lección más profunda de la fase fue comprender, diagnosticar y resolver el problema de la concurrencia no controlada. Visualizar que 100 procesos clonados intentarían acceder y sobreescribir simultáneamente el mismo segmento de memoria RAM dejó clara la vulnerabilidad de los sistemas ante las condiciones de carrera. Implementar un semáforo como mecanismo de bloqueo en la sección crítica no fue solo un requisito sintáctico, sino la única garantía matemática de exclusión mutua para que la sumatoria armónica de la serie de Fourier se calculara de forma determinista y segura, demostrando que en el paralelismo, la sincronización es tan crítica como el cálculo mismo.

El contraste entre evaluar 100 iteraciones replicando celdas en una hoja de cálculo y delegar la carga directamente al sistema operativo demostró la verdadera escalabilidad del software. Además, esta práctica de Inter-Process Communication (IPC) conecta directamente con los fundamentos teóricos de las arquitecturas de supercómputo y alto rendimiento que estudio actualmente, sentando las bases operativas para dominar herramientas más robustas como MPI, OpenMP o los kernels de aceleración en tarjetas gráficas (CUDA).

Finalmente, este enfoque de trabajo distribuido es el núcleo computacional de mi carrera en Inteligencia Artificial. Procesar señales en el dominio de las frecuencias y evaluar funciones matemáticas complejas dividiendo la carga en un enjambre de subtareas es el principio exacto que sustenta el procesamiento paralelo de matrices en redes neuronales. De igual forma, refleja la manera en que se evalúan simultáneamente múltiples individuos en algoritmos de optimización bioinspirada, como los algoritmos genéticos o la optimización por enjambre de partículas (PSO). Al final, llevar una ecuación abstracta de cálculo avanzado hacia una arquitectura concurrente y 100 por cierto funcional en Linux cerró exitosamente el ciclo entre el análisis teórico, la validación ofimática y la ingeniería de software aplicada de alto nivel.

3.4.4 Conclusión — Said Carbot Cruz Trejo

La culminación de esta tercera fase del proyecto ha representado un punto de inflexión muy significativo en mi formación profesional, al permitirme amalgamar conceptos teóricos de análisis matemático con la programación a nivel de sistema. Al iniciar este trabajo, la resolución de las series de Fourier para la función $f(x) = x^5$ se abordó desde una perspectiva puramente analítica y estática. En el capítulo 2, la limitación de las herramientas ofimáticas tradicionales se hizo evidente: calcular diez armónicos en una hoja de cálculo resultaba un proceso manual, propenso a errores y computacionalmente restrictivo, que generaba una curva con un rizado notable que apenas lograba aproximarse a la función original.

El desafío principal de esta etapa no radicó en la codificación de la fórmula matemática, sino en la orquestación arquitectónica de los recursos del sistema operativo. Trabajar directamente con las llamadas al sistema en un entorno basado en UNIX (macOS) me obligó a comprender la anatomía de los procesos. Al implementar `fork()` pude observar de primera mano cómo el núcleo clona contextos de ejecución y genera un proceso padre y cien procesos hijos independientes. Esta duplicación masiva del espacio de direcciones virtuales introdujo una complejidad sustancial: el aislamiento de memoria. A diferencia de los hilos de ejecución (*threads*), que comparten el mismo espacio de direccionamiento de forma nativa, los procesos creados con `fork()` son entidades aisladas. Si cada proceso calculaba su término de Fourier en su propia memoria privada, el resultado global se perdería en el instante en que el proceso terminara su ciclo de vida y el sistema operativo reclamara sus recursos.

Para resolver este aislamiento lógico fue imperativo diseñar e implementar mecanismos de comunicación interprocesos (IPC). La memoria compartida, a través de las primitivas `shmget` y `shmat`, estableció un puente de datos bidireccional donde los cien procesos hijos podían depositar sus matrices de resultados horizontales (las 100 columnas independientes) y, al mismo tiempo, sumar su aporte a una columna centralizada. Sin embargo, habilitar un segmento de memoria compartida introdujo de inmediato el problema clásico de la concurrencia: las condiciones de carrera (*race conditions*). Comprender que el planificador del sistema operativo (*scheduler*) puede interrumpir a un proceso a la mitad de una instrucción de suma en ensamblador me hizo valorar la necesidad crítica de la sincronización. Los semáforos POSIX (`sem_open`, `sem_wait`, `sem_post`) actuaron como un cerrojo de exclusión mutua, y garantizaron que el acumulador global de la serie de Fourier mantuviera su integridad matemática sin colisiones de escritura.

Finalmente, ver los resultados plasmados de vuelta en las gráficas validó todo el esfuerzo de programación de bajo nivel. Al comparar visualmente el cálculo concurrente de cien armónicos contra los diez calculados en la hoja de cálculo, la mejora en la precisión es evidente: la curva polinomial se modela casi a la perfección y se hace visible de manera nítida el *fenómeno de Gibbs* en las discontinuidades de los extremos, un detalle analítico que en la fase anterior quedaba difuminado.

Como estudiante de Ingeniería en Inteligencia Artificial en la ESCOM, esta práctica trasciende el simple cumplimiento de una rúbrica escolar. El procesamiento masivo de datos, la paralelización de operaciones matriciales y la optimización de recursos computacionales son pilares fundamentales para el entrenamiento de redes neuronales y de arquitecturas de aprendizaje automático.

3.5 Anexos

En esta sección se colocan los códigos generados en la fase 3 (programación con procesos, capítulo 3).

3.5.1 Fase 3: código de `fourier_aaron.c`

Programa de Aarón Ugalde Téllez que calcula la Serie de Fourier con 100 procesos hijos, memoria compartida y semáforos System V.

```c
/*
 *
    ================================================================================
 * INSTITUTO POLITÉCNICO NACIONAL - ESCUELA SUPERIOR DE CÓMPUTO
 * Cómputo Paralelo - Grupo 6BV1
 * Capítulo 3: Programación con Procesos
 *
 * Autor: Aarón Ugalde Téllez
 * Función asignada: f(x) = -5/6 * x^2 + pi/3 * x + 2/7
 * Configuración: 100 Procesos Hijos, Memoria Compartida, Semáforos System V.
 * Puntos de evaluación: 64 valores específicos de x especificados por el
    usuario.
 *
    ================================================================================
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <math.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <sys/wait.h>

#define NUM_PROCESOS 100 // Exactly 100 child processes
#define NUM_PUNTOS 64 // Array size corresponding to specified x values

// Arreglo estático con los 64 valores exactos de x
static const double x_values[NUM_PUNTOS] = {
   -3.1416, -3.0416, -2.9416, -2.8416, -2.7416, -2.6416, -2.5416, -2.4416,
   -2.3416, -2.2416, -2.1416, -2.0416, -1.9416, -1.8416, -1.7416, -1.6416,
   -1.5416, -1.4416, -1.3416, -1.2416, -1.1416, -1.0416, -0.9416, -0.8416,
   -0.7416, -0.6416, -0.5416, -0.4416, -0.3416, -0.2416, -0.1416, -0.0416,
    0.0584, 0.1584, 0.2584, 0.3584, 0.4584, 0.5584, 0.6584, 0.7584,
    0.8584, 0.9584, 1.0584, 1.1584, 1.2584, 1.3584, 1.4584, 1.5584,
    1.6584, 1.7584, 1.8584, 1.9584, 2.0584, 2.1584, 2.2584, 2.3584,
    2.4584, 2.5584, 2.6584, 2.7584, 2.8584, 2.9584, 3.0584, 3.1584
```

```
};

// Estructura requerida para semctl
union semun {
   int val;
   struct semid_ds *buf;
   unsigned short *array;
};

// Operación P (Wait / Bloquear semáforo)
void sem_wait_op(int sem_id) {
   struct sembuf sb = {0, -1, 0};
   semop(sem_id, &sb, 1);
}

// Operación V (Signal / Liberar semáforo)
void sem_post_op(int sem_id) {
   struct sembuf sb = {0, 1, 0};
   semop(sem_id, &sb, 1);
}

int main() {
   printf("========================================================================\n");
   printf(" CÓMPUTO PARALELO - CAPÍTULO 3: PROGRAMACIÓN CON PROCESOS\n");
   printf(" Integrante: Aarón Ugalde Téllez\n");
   printf(" Evaluación de Serie de Fourier con 100 Procesos Hijos (%d
      Puntos)\n", NUM_PUNTOS);
   printf("========================================================================\n\n");

   // 1. Crear Memoria Compartida para guardar el arreglo de los 64 resultados
   size_t shm_size = NUM_PUNTOS * sizeof(double);
   int shm_id = shmget(IPC_PRIVATE, shm_size, IPC_CREAT | 0666);
   if (shm_id < 0) {
      perror("Error al crear la memoria compartida (shmget)");
      exit(EXIT_FAILURE);
   }

   // Acoplar la memoria compartida al espacio de direcciones
   double *f_x = (double *)shmat(shm_id, NULL, 0);
   if (f_x == (double *)-1) {
      perror("Error al acoplar la memoria compartida (shmat)");
      exit(EXIT_FAILURE);
   }

   // 2. Inicializar la memoria compartida con el valor a0
   // a0 = -5/18 * pi^2 - 2/7
   double a0 = (-5.0 / 18.0) * (M_PI * M_PI) - (2.0 / 7.0);
```

```
82    for (int i = 0; i < NUM_PUNTOS; i++) {
83       f_x[i] = a0;
84    }
85
86    // 3. Crear e inicializar el semáforo System V
87    int sem_id = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
88    if (sem_id < 0) {
89       perror("Error al crear el semáforo (semget)");
90       exit(EXIT_FAILURE);
91    }
92
93    union semun arg;
94    arg.val = 1; // Inicializado en 1 (sección crítica libre)
95    if (semctl(sem_id, 0, SETVAL, arg) < 0) {
96       perror("Error al inicializar el semáforo (semctl)");
97       exit(EXIT_FAILURE);
98    }
99
100   printf("[PADRE | PID: %d] Creando %d procesos hijos...\n\n", getpid(),
          NUM_PROCESOS);
101
102   // 4. Crear los 100 procesos hijos
103   for (int p = 1; p <= NUM_PROCESOS; p++) {
104      pid_t pid = fork();
105
106      if (pid < 0) {
107         perror("Error al crear proceso hijo (fork)");
108         exit(EXIT_FAILURE);
109      }
110
111      if (pid == 0) {
112         // === PROCESO HIJO ===
113         int n = p; // El hijo 'p' procesa el armónico n = p
114         double local_terms[NUM_PUNTOS];
115
116         // Coeficientes an y bn de la función de Aarón Ugalde
117         double signo = ((n + 1) % 2 == 0) ? 1.0 : -1.0; // (-1)^(n+1)
118         double an = (10.0 * signo) / (3.0 * n * n);
119         double bn = (2.0 * signo) / (3.0 * n);
120
121         // Calcular término n para cada uno de los 64 puntos x especificados
122         for (int i = 0; i < NUM_PUNTOS; i++) {
123            double x = x_values[i];
124            local_terms[i] = an * cos(n * x) + bn * sin(n * x);
125         }
126
127         // --- SECCIÓN CRÍTICA ---
```

```
128             sem_wait_op(sem_id); // Entrar a sección crítica
129
130             for (int i = 0; i < NUM_PUNTOS; i++) {
131                 f_x[i] += local_terms[i];
132             }
133
134             sem_post_op(sem_id); // Salir de sección crítica
135             // ------------------------
136
137             printf("[HIJO #%03d | PID: %d] Armónico n = %3d procesado para los
                    %d puntos.\n", n, getpid(), n, NUM_PUNTOS);
138
139             shmdt(f_x);
140             exit(EXIT_SUCCESS);
141         }
142     }
143
144     // 5. El proceso padre espera a que finalicen los 100 hijos
145     for (int i = 0; i < NUM_PROCESOS; i++) {
146         wait(NULL);
147     }
148
149     printf("\n[PADRE] Los %d procesos hijos han completado el cálculo.\n",
            NUM_PROCESOS);
150
151     // 6. Generar archivo CSV con los resultados exactos
152     const char *filename = "fourier_aaron_64puntos.csv";
153     FILE *fp = fopen(filename, "w");
154     if (fp == NULL) {
155         perror("Error al crear el archivo CSV de salida");
156         exit(EXIT_FAILURE);
157     }
158
159     fprintf(fp, "x,f_fourier\n");
160     for (int i = 0; i < NUM_PUNTOS; i++) {
161         fprintf(fp, "%.4f,%.6f\n", x_values[i], f_x[i]);
162     }
163     fclose(fp);
164
165     printf("[PADRE] Resultados guardados en el archivo '%s'.\n", filename);
166
167     // 7. Liberar memoria compartida y semáforos
168     shmdt(f_x);
169     shmctl(shm_id, IPC_RMID, NULL);
170     semctl(sem_id, 0, IPC_RMID);
171
172     printf("[PADRE] Recursos IPC (SHM y semáforos) liberados
```

```
        correctamente.\n");
173     printf("================================================================================\n");
174
175     return 0;
176 }
```

Código 3.16. Programa `fourier_aaron.c`.

3.5.2 Fase 3: código de `fourier_f3_lino.c`

Programa de Lino Ehecatl Romero Rosales que calcula los coeficientes de la Serie de Fourier con 100 procesos hijos, memoria compartida y un semáforo System V.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <math.h>
5  #include <sys/ipc.h>
6  #include <sys/shm.h>
7  #include <sys/sem.h>
8  #include <sys/wait.h>
9
10 #ifndef M_PI
11 #define M_PI 3.14159265358979323846
12 #endif
13
14 #define N_PROCESOS 100
15
16 typedef struct {
17     int n;
18     double an;
19     double bn;
20 } TerminoFourier;
21
22 void sem_wait(int semid) {
23     struct sembuf sb = {0, -1, 0};
24     semop(semid, &sb, 1);
25 }
26
27 void sem_post(int semid) {
28     struct sembuf sb = {0, 1, 0};
29     semop(semid, &sb, 1);
30 }
31
32 int main() {
33     int shmid = shmget(IPC_PRIVATE, sizeof(TerminoFourier) * N_PROCESOS,
        IPC_CREAT | 0666);
```

```
34    if (shmid < 0) {
35       perror("Error en memoria compartida");
36       exit(EXIT_FAILURE);
37    }
38
39    TerminoFourier *tabla = (TerminoFourier *) shmat(shmid, NULL, 0);
40    if (tabla == (void *) -1) {
41       perror("Error al acoplar memoria compartida");
42       exit(EXIT_FAILURE);
43    }
44
45    int semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
46    if (semid < 0) {
47       perror("Error en semáforo");
48       exit(EXIT_FAILURE);
49    }
50    semctl(semid, 0, SETVAL, 1);
51
52    printf("[PADRE] Calculando Fourier para f(x) = -(1/4)x^4 + 3 con %d
          procesos hijos...\n", N_PROCESOS);
53
54    for (int i = 0; i < N_PROCESOS; i++) {
55       pid_t pid = fork();
56
57       if (pid < 0) {
58          perror("Error en fork()");
59          exit(EXIT_FAILURE);
60       }
61
62       if (pid == 0) {
63          int n = i + 1;
64          double an = (2.0 * ((n % 2 == 0) ? 1.0 : -1.0) * (6.0 - (n * n *
                M_PI * M_PI))) / (n * n * n * n);
65          double bn = 0.0;
66
67          sem_wait(semid);
68          tabla[i].n = n;
69          tabla[i].an = an;
70          tabla[i].bn = bn;
71          sem_post(semid);
72
73          shmdt(tabla);
74          exit(EXIT_SUCCESS);
75       }
76    }
77
78    for (int i = 0; i < N_PROCESOS; i++) {
```

```
79        wait(NULL);
80    }
81
82    double a0_half = 3.0 - (pow(M_PI, 4) / 20.0);
83
84    FILE *archivo = fopen("fourier_f3_lino.csv", "w");
85    if (!archivo) {
86        perror("Error al crear archivo CSV");
87        exit(EXIT_FAILURE);
88    }
89
90    fprintf(archivo, "n,a_n,b_n\n");
91    fprintf(archivo, "0,%.6f,0.000000\n", a0_half);
92    for (int i = 0; i < N_PROCESOS; i++) {
93        fprintf(archivo, "%d,%.6f,%.6f\n", tabla[i].n, tabla[i].an,
              tabla[i].bn);
94    }
95    fclose(archivo);
96
97    printf("[PADRE] Proceso completado. Datos exportados a
          'fourier_f3_lino.csv'.\n");
98
99    shmdt(tabla);
100   shmctl(shmid, IPC_RMID, NULL);
101   semctl(semid, 0, IPC_RMID);
102
103   return 0;
104 }
```

Código 3.17. Programa `fourier_f3_lino.c`.

3.5.3 Fase 3: código de `fourier_x5.c`

Programa de Said Carbot Cruz Trejo que calcula la Serie de Fourier de $f(x) = x^5$ con 100 procesos hijos, memoria compartida System V y un semáforo POSIX.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5  #include <sys/shm.h>
6  #include <semaphore.h>
7  #include <fcntl.h>
8  #include <math.h>
9
10 int main() {
11    int i, n;
```

```
12    int shmid;
13    double *memoria;
14    sem_t *semaforo;
15    double PI = 3.14159265359;
16
17    // Se reservan 500 espacios para la suma total y
18    // 100 procesos con 500 puntos para cada término.
19    // Total = 101 filas x 500 columnas = 50500 espacios
20    int total_espacios = 101 * 500;
21
22    // 1. Solicitar la memoria compartida
23    shmid = shmget(IPC_PRIVATE, total_espacios * sizeof(double), IPC_CREAT |
          0666);
24    if (shmid == -1) {
25       printf("Error pidiendo memoria\n");
26       return 1;
27    }
28
29    // 2. Conectar la memoria compartida al proceso
30    memoria = (double *) shmat(shmid, NULL, 0);
31
32    // Inicializar todos los valores de la memoria en cero
33    for (i = 0; i < total_espacios; i++) {
34       memoria[i] = 0.0;
35    }
36
37    // 3. Crear el semáforo para controlar el acceso a la memoria compartida
38    sem_unlink("/candado_said");
39    semaforo = sem_open("/candado_said", O_CREAT, 0644, 1);
40
41    printf("Iniciando los 100 procesos para f(x) = x^5...\n");
42
43    // 4. Crear los 100 procesos hijos
44    for (n = 1; n <= 100; n++) {
45       pid_t pid = fork();
46
47       if (pid == 0) {
48          // --- CÓDIGO DEL HIJO ---
49          double signo;
50
51          // Determinar el signo correspondiente al término n
52          if (n % 2 == 0) {
53             signo = -1.0;
54          } else {
55             signo = 1.0;
56          }
57
```

```
58          // Calcular los componentes del coeficiente b_n
59          double parte1 = pow(PI, 4) / n;
60          double parte2 = (20.0 * pow(PI, 2)) / pow(n, 3);
61          double parte3 = 120.0 / pow(n, 5);
62          double bn = 2.0 * signo * (parte1 - parte2 + parte3);
63
64          // Evaluar el término de Fourier en 500 puntos
65          for (i = 0; i < 500; i++) {
66             double x = -PI + i * (2.0 * PI / 499.0);
67             double calculo = bn * sin(n * x);
68
69             // A) Almacenar el valor correspondiente al término n
70             // La expresión n * 500 permite acceder a la sección
71             // de memoria asignada al término correspondiente.
72             memoria[n * 500 + i] = calculo;
73
74             // B) Acumular el valor en la suma total de Fourier
75             // El semáforo garantiza el acceso exclusivo a esta
76             // sección de memoria compartida.
77             sem_wait(semaforo);
78             memoria[i] = memoria[i] + calculo;
79             sem_post(semaforo);
80          }
81
82          exit(0);
83          // --- FIN DEL CÓDIGO DEL HIJO ---
84       }
85    }
86
87    // --- CÓDIGO DEL PADRE ---
88
89    // 5. Esperar a que finalicen los 100 procesos hijos
90    for (n = 1; n <= 100; n++) {
91       wait(NULL);
92    }
93
94    // 6. Crear el archivo CSV para almacenar los resultados
95    FILE *archivo = fopen("datos_said.csv", "w");
96
97    // Escribir la primera fila con los encabezados de las columnas
98    fprintf(archivo, "x,f(x)=x^5,Fourier(N=100)");
99    for (n = 1; n <= 100; n++) {
100      fprintf(archivo, ",Termino n=%d", n);
101   }
102   fprintf(archivo, "\n");
103
104   // Escribir los datos correspondientes a cada punto x
```

```
105     for (i = 0; i < 500; i++) {
106         double x = -PI + i * (2.0 * PI / 499.0);
107         double fx = pow(x, 5);
108 
109         // Escribir x, f(x) y la suma de los 100 términos de Fourier
110         fprintf(archivo, "%f,%f,%f", x, fx, memoria[i]);
111 
112         // Escribir los valores individuales de los 100 términos
113         for (n = 1; n <= 100; n++) {
114             fprintf(archivo, ",%f", memoria[n * 500 + i]);
115         }
116         fprintf(archivo, "\n");
117     }
118     fclose(archivo);
119 
120     printf("Listo. Datos guardados en datos_said.csv\n");
121 
122     // 7. Liberar la memoria compartida y cerrar el semáforo
123     sem_close(semaforo);
124     sem_unlink("/candado_said");
125     shmdt(memoria);
126     shmctl(shmid, IPC_RMID, NULL);
127 
128     return 0;
129 }
```

Código 3.18. Programa `fourier_x5.c`.

Capítulo 4. Programación con Hilos

4.1 Marco teórico

En el ámbito de los sistemas operativos y la programación concurrente, un hilo (o *thread*) es la unidad de procesamiento más pequeña que puede ser planificada por un sistema operativo. A diferencia de los procesos independientes, que cuentan con su propio espacio de memoria aislado, los hilos originados a partir de un mismo proceso padre comparten la sección de datos, el espacio de direcciones y los recursos del sistema (como descriptores de archivos). Esta arquitectura de memoria compartida hace que el cambio de contexto entre hilos sea considerablemente más rápido y requiera menos recursos computacionales que el cambio entre procesos, razón por la cual se les conoce como ”procesos ligeros”.

En el lenguaje de programación C, particularmente dentro de entornos basados en Unix/Linux, la implementación estándar para la gestión de hilos se realiza a través de la biblioteca POSIX Threads, comúnmente referida como `pthreads` (incluyendo la cabecera `<pthread.h>`). Esta interfaz de programación de aplicaciones (API) proporciona las primitivas necesarias para crear, sincronizar y destruir hilos.

El ciclo de vida básico de un hilo en C mediante `pthreads` involucra funciones fundamentales como:

- `pthread_create()`: Instancia un nuevo hilo de ejecución, asignándole una subrutina específica (función) que debe ejecutar concurrentemente.
- `pthread_join()`: Bloquea la ejecución del hilo invocador (usualmente el hilo principal o proceso padre) hasta que el hilo objetivo haya finalizado su tarea, permitiendo la recolección de resultados y asegurando la correcta sincronización.
- `pthread_exit()`: Termina la ejecución del hilo actual, retornando un valor o puntero de estado al proceso padre.

El uso de hilos en C es ideal para el cómputo de alto rendimiento, como la resolución de las series de Fourier abordadas en este proyecto, ya que permite paralelizar el cálculo de diferentes armónicos o intervalos matemáticos simultáneamente, aprovechando la arquitectura multinúcleo del procesador.

4.2 Programación con hilos en lenguaje C y en la plataforma GNU-Linux

En esta sección se presenta la implementación de la programación con hilos en lenguaje C sobre la plataforma GNU-Linux, aplicando los cálculos de los términos de la serie de Fourier de la función propuesta ($f(x) = -\frac{1}{4}x^4 + 3$). Se implementó el uso de al menos 100 hilos concurrentes.

4.2.1 Fragmentos de códigos explicados de la programación con hilos

Para la resolución concurrente del problema, se utilizó la biblioteca POSIX Threads (`pthreads`) en lenguaje C. El siguiente fragmento de código muestra la rutina ejecutada por cada hilo, la cual calcula de manera independiente una porción de los coeficientes armónicos (n, a_n, b_n) distribuyendo la carga de trabajo:

```
void *calcular_coeficientes_fourier(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    int id = data->thread_id;
    int total = data->total_threads;

    int terminos_por_hilo = NUM_THREADS / total;
    int inicio = id * terminos_por_hilo;
    int fin = inicio + terminos_por_hilo;

    for (int i = inicio; i < fin; i++) {
        int n = i + 1; // n armónico de 1 a 100

        // Cálculo de los coeficientes para f(x) = -(1/4)x^4 + 3
        double an = (2.0 * ((n % 2 == 0) ? 1.0 : -1.0) *
                    (6.0 - (n * n * M_PI * M_PI))) / (n * n * n * n);
        double bn = 0.0; // Función par

        data->resultados[i].n = n;
        data->resultados[i].an = an;
        data->resultados[i].bn = bn;
    }
    pthread_exit(NULL);
}
```

En el proceso principal (`main`), se crean los 100 hilos mediante la función `pthread_create()`, asignándoles sus respectivas estructuras de datos, y se sincronizan con `pthread_join()` para asegurar la finalización correcta de los cálculos antes de exportar los resultados a un archivo.

4.2.2 Compilación

La compilación del programa en la plataforma GNU-Linux se realizó mediante el compilador GCC, enlazando explícitamente la biblioteca de hilos (`-pthread`) y la biblioteca matemática (`-lm`). Como se observa en la Figura 4.1, el reporte de la terminal muestra que la compilación se ejecutó de forma limpia, evitando así la muestra de algún error o advertencia (*warnings*).

```
lino-romero@lino-romero-Thin-15-B13UC:~$ gcc -Wall -Wextra fourier_hilos.c -o fourier_hilos -pthread -lm
lino-romero@lino-romero-Thin-15-B13UC:~$
```

Figura 4.1. Captura de pantalla de la compilación limpia del programa de hilos en la terminal de GNU-Linux.

4.2.3 Ejecución de los programas

A continuación se muestra la ejecución del programa compilado en la terminal de GNU-Linux. Tras la inicialización de los 100 hilos concurrentes, el programa realiza los cómputos correspondientes y descarga automáticamente la información obtenida en un archivo estructurado (`fourier_hilos_resultados.csv`).

```
=== INICIO DE PROGRAMACIÓN CON HILOS (100 HILOS) ===
Calculando coeficientes de Fourier para f(x) = -(1/4)x^4 + 3 con hilos en GNU-Linux...
[HILOS] Proceso completado con éxito. Datos exportados a 'fourier_hilos_resultados.csv'.
lino-romero@lino-romero-Thin-15-B13UC:~$
```

Figura 4.2. Captura de pantalla de la ejecución de los programas y confirmación de la exportación de datos.

4.2.4 Gráficas, Figuras y su Explicación

La información obtenida en el archivo CSV generado por los hilos fue descargada e importada al archivo de Excel utilizado en las fases anteriores para trazar las gráficas definitivas del proyecto.



Figura 4.3. Gráficas de la serie de Fourier generadas en Excel a partir de los datos procesados por los hilos.

Explicación de las figuras: Como se aprecia en la Figura 4.3, las gráficas modelan el comportamiento de la serie armónica. Se explica detalladamente que a medida que el número de armónicos n aumenta, la amplitud de los coeficientes a_n decrece, lo que demuestra visualmente cómo converge la serie hacia la función original.

4.2.5 Comparación de resultados con fases anteriores

Al contrastar los resultados obtenidos en esta fase de hilos con los obtenidos en los capítulos previos (programación con procesos y análisis matemático al inicio), se concluye lo siguiente:

1. **Precisión numérica:** Los valores de los coeficientes a_n y b_n calculados mediante los 100 hilos son **exactamente idénticos** a los calculados en el Capítulo 3 con procesos y en la fase analítica del Capítulo 1, garantizando la consistencia matemática del modelo computacional.
2. **Eficiencia operativa:** A diferencia de los procesos tradicionales (`fork()`), los cuales duplican los espacios de memoria e involucran mecanismos pesados de comunicación interprocesos (IPC), los hilos comparten un mismo espacio de direcciones de memoria de forma nativa. Esto reduce drásticamente la sobrecarga por cambio de contexto (*context switching*) y optimiza de manera notable el rendimiento del equipo de cómputo al paralelizar de forma directa el cálculo de los armónicos.

4.3 Análisis de la implementación con hilos

El desarrollo de esta fase se fundamentó en el paradigma de programación paralela utilizando memoria compartida. A continuación, se presenta el análisis técnico de la implementación y las características del entorno de ejecución.

Análisis del lenguaje y técnica utilizada

La implementación se desarrolló íntegramente en **lenguaje C** haciendo uso de la biblioteca **POSIX Threads (pthreads)**. Se eligió C debido a su capacidad de interacción a bajo nivel con el planificador del sistema operativo GNU/Linux y su nula sobrecarga (overhead) en tiempo de ejecución en comparación con lenguajes interpretados. La técnica principal consistió en la paralelización de datos (Data Parallelism); el proceso principal (padre) descompone el dominio de la serie de Fourier a evaluar y despacha fragmentos de la carga matemática a múltiples hilos de trabajo (worker threads). Esto transforma un cálculo algorítmico de complejidad secuencial en uno concurrente, reduciendo el tiempo total de procesamiento.

Estructura de procesos e hilos

- **Cantidad de procesos:** 1 (Un único proceso principal o padre que orquesta la ejecución, reserva la memoria y exporta los datos a formato CSV).
- **Cantidad de hilos:** 101 (1 hilo principal y 100 hilos hijos creados con `pthread_create`. Cada hilo hijo está dedicado al cálculo simultáneo de los coeficientes a_n y b_n para los 100 primeros armónicos de la serie de Fourier).

Características del equipo de cómputo

Las pruebas de compilación y ejecución concurrente se llevaron a cabo en un equipo **MSI Thin 15 B13V** operando bajo un entorno nativo de **Ubuntu Linux**. Las especificaciones técnicas del hardware son

las siguientes:

Componente	**Especificación Técnica**
Procesador	Intel Core i5-13420H (13va Generación ”Raptor Lake”)
Núcleos	8 núcleos físicos (4 Performance-cores y 4 Efficient-cores) con 12 hilos lógicos en total
Velocidad del CPU	Frecuencia base de 2.10 GHz (con Turbo Boost máximo de hasta 4.60 GHz)
Tarjeta Gráfica	NVIDIA GeForce RTX 3050
Memoria RAM	16 GB, tipo DDR4
Velocidad de RAM	3200 MHz
Memoria Caché	12 MB Intel Smart Cache (Caché L3 compartida), optimizada para el acceso rápido de múltiples núcleos
Sistema Operativo	Ubuntu Linux (Kernel optimizado para arquitecturas híbridas y gestión SMP)

Tabla 4.1. Características técnicas del equipo de cómputo empleado para la fase 4.

Conclusión Cruz Trejo

La transicion metodologica que hemos llevado desde la solucion analitica de la serie de Fourier, pasando por la creacion de procesos pesados (forks), hasta llegar finalmente a la orquestacion de hilos (pthreads) en C, ilustra a la perfeccion como los ingenieros hemos intentado exprimir al maximo la arquitectura de Von Neumann. Tecnicamente, el reto de esta fase no fue la matematica de la funcion $f(x) = -(1/4)x^4 + 3$, sino la arquitectura de la concurrencia. Instanciar 100 hilos, uno por cada armonico, requirio un diseno cuidadoso de la memoria compartida. Al definir las estructuras `TerminoFourier` y `ThreadData`, logramos que cada hilo recibiera su contexto a traves de un simple puntero generico, calculara sus respectivos coeficientes a_n y b_n, y los depositara ordenadamente en un arreglo comun. A diferencia del capitulo anterior, donde los procesos nos obligaban a lidiar con la costosa duplicacion del espacio de memoria, los hilos nos permitieron trabajar de forma elegante y ligera. Sin embargo, mientras se programaba la recoleccion de estos 100 armonicos mediante sincronizacion clasica con `pthread_join`, no pude evitar relacionarlo con mi area de mayor interes: la computacion cuantica. En el fondo, lo que estamos haciendo con pthreads es el intento de la computacion clasica por simular un paralelismo masivo. Dividimos el tiempo de CPU y asignamos tareas a diferentes nucleos para calcular los coeficientes de Fourier de manera simultanea. Pero si abordaramos este mismo problema matematico utilizando un algoritmo como la Transformada Cuantica de Fourier (QFT), el paradigma cambiaria por completo. En lugar de necesitar 100 hilos (o 100 nucleos fisicos) iterando paso a paso, un sistema cuantico aprovecharia la superposicion de estados para evaluar estas amplitudes de probabilidad de forma intrinsecamente paralela, logrando una aceleracion exponencial frente a nuestro enfoque clasico. A pesar de esa comparacion, esta practica me hizo valorar muchisimo la madurez del hardware y software clasico. En la actual era NISQ (Noisy Intermediate-Scale Quantum), lidiar con qubits tipo Transmon implica luchar constantemente contra la decoherencia y el ruido termico. En contraste, nuestro programa en C utilizando pthreads es bellamente determinista y estable. Los coeficientes que exportamos al archivo CSV son matematicamente exactos cada vez que corremos el ejecutable, sin necesidad de aplicar codigos de correccion de errores. Finalmente, la integracion de los datos exportados por nuestro programa en C hacia nuestra plantilla de Excel confirmo empiricamente que la logica de sincronizacion fue un exito. Ver como la sumatoria de la serie converge perfectamente hacia la grafica de la funcion original es la mejor prueba de que no hubo condiciones de carrera al escribir en la memoria. En conclusion, dominar la concurrencia clasica mediante pthreads no solo cumple con los objetivos de esta asignatura, sino que me proporciona una base ingenieril solida y necesaria. Entender como el hardware clasico gestiona sus cuellos de botella es el primer paso indispensable antes de proponer soluciones en arquitecturas mucho mas complejas y disruptivas como las que nos esperan en el futuro.

Conclusión Romero Rosales

Hacer esta práctica de la serie de Fourier usando hilos en C sobre Linux me sirvió bastante para entender de forma práctica cómo funciona la concurrencia a nivel de sistema operativo. A diferencia de lo que hicimos en la fase pasada con procesos, donde cada uno se crea por separado y hay que batallar con la memoria compartida y los semáforos para que no se mezclen los datos, con los hilos el proceso fue más directo porque todos comparten el mismo espacio de memoria dentro del mismo programa. Eso hace que la comunicación sea mucho más natural y que la computadora no gaste tantos recursos haciendo cambios pesados entre tareas. Al principio, organizar el código para repartir el cálculo de los 100 armónicos entre los 100 hilos exigió poner atención en cómo se estructuraban los datos, asegurando que cada hilo supiera exactamente qué parte del trabajo le tocaba sin estorbar a los demás. Sin embargo, una vez que definimos bien las estructuras y el paso de argumentos, el programa compiló limpio desde la terminal, y la ejecución fue inmediata. Al revisar el archivo CSV generado y vaciar los datos a Excel para armar las gráficas, pude comprobar que los coeficientes salieron exactamente iguales a los que ya habíamos calculado en las fases pasadas, lo que demuestra que cambiar al modelo de hilos no altera para nada la precisión matemática de la serie. En lo personal, toda esta experiencia me ayudó a ver las ventajas reales de programar de forma concurrente cuando tienes que resolver tareas repetitivas o pesadas. Trabajar desde la terminal, manejar las librerías de hilos de POSIX, compilar con GCC y automatizar todo con un Makefile me dio una mejor idea de cómo se estructuran proyectos más grandes en sistemas unidos por hilos. Al final, te das cuenta de que usar hilos ahorra tiempo de procesamiento y aprovecha mejor los núcleos de la computadora así pudimos lograr generar cada armónico de la serie de Fourier sin necesidad de que se generara uno por uno y haciendo mucho más rápido el procesamiento sin necesidad de sobreponer algún elemento, creo que el implementar hilos para resolver tareas en paralelo es algo muy útil si queremos ahorrar tiempo y así logramos aprovechar al máximo nuestras computadoras.

Conclusión López Castro

La culminación de esta fase del proyecto, enfocada en la resolución de las series de Fourier mediante la programación concurrente con hilos, marca un punto de inflexión significativo respecto a las metodologías secuenciales aplicadas en las fases anteriores. El verdadero desafío de esta etapa no radicó en la complejidad matemática de las funciones de Fourier, las cuales ya habían sido modeladas y validadas previamente, sino en la reestructuración del pensamiento algorítmico para dividir de manera eficiente el dominio del problema. Pasar de un enfoque donde un solo proceso ejecuta cada iteración de manera lineal, a uno donde múltiples hilos de ejecución operan simultáneamente sobre distintos fragmentos de la serie, demuestra de forma práctica las ventajas del cómputo de alto rendimiento.

A nivel de desarrollo de software, la interacción directa con la biblioteca `pthreads` en lenguaje C exige una disciplina de programación mucho más rigurosa. Al operar en un entorno de memoria compartida dentro del mismo proceso padre, el riesgo de generar condiciones de carrera o inconsistencias en los resultados es constante si no se implementa una sincronización adecuada. El uso de funciones como `pthread_create` y `pthread_join` obligó a diseñar un flujo de control donde el hilo principal debe no solo despachar el trabajo, sino garantizar que todos los cálculos parciales de los armónicos hayan finalizado antes de ensamblar y presentar el resultado final. Ejecutar este código nativamente en Ubuntu Linux permitió, además, observar cómo el sistema operativo planifica y distribuye la carga a través de los múltiples núcleos físicos y lógicos del procesador, reduciendo de manera tangible los tiempos de ejecución en comparación con los scripts secuenciales previos.

Dentro del contexto de la Ingeniería en Inteligencia Artificial, los aprendizajes obtenidos en esta implementación trascienden la simple evaluación de una función trigonométrica. El procesamiento de señales y el análisis en el dominio de la frecuencia —donde Fourier es la herramienta fundamental— son operaciones base en áreas como la visión por computadora y el procesamiento de lenguaje natural. Más aún, la capacidad de paralelizar cálculos es el pilar sobre el cual se construyen los algoritmos de inteligencia artificial modernos. Entrenar modelos de clasificación complejos, calcular gradientes descendentes por lotes, o ejecutar algoritmos de optimización bioinspirados (como la Optimización por Enjambre de Partículas o la Colonia de Hormigas) resulta computacionalmente inviable si se ejecutan de manera estrictamente secuencial.

En conclusión, este proyecto logró integrar exitosamente el análisis matemático avanzado con la arquitectura de sistemas operativos. Comparar los tiempos de ejecución y la eficiencia de esta fase con las anteriores valida empíricamente que dominar el cómputo concurrente no es una simple mejora técnica, sino un requisito indispensable. Comprender a bajo nivel cómo asignar recursos de hardware, gestionar la concurrencia y acelerar cálculos matemáticos masivos es una habilidad crítica que sienta las bases para el futuro desarrollo de sistemas inteligentes eficientes, robustos y escalables.

Conclusión Ugalde Téllez

La realización de esta práctica representó una oportunidad muy enriquecedora para comprender de manera práctica cómo se comportan los programas cuando dividimos una tarea compleja en muchas partes pequeñas que se ejecutan al mismo tiempo. Al trabajar con la serie de Fourier en el sistema operativo GNU/Linux utilizando el lenguaje C, el objetivo principal no solo era obtener los resultados matemáticos de la función, sino aprender a organizar el trabajo de la computadora mediante el uso de más de cien hilos de ejecución, observando las ventajas y retos que este enfoque conlleva. Al principio, adaptar el programa para que funcionara con una cantidad tan grande de hilos fue un desafío interesante. A diferencia de las etapas anteriores donde se trabajaba con procesos independientes, el uso de hilos requiere un cuidado mucho mayor en la forma en que se comparten la información y las variables dentro del programa. Cada hilo debía encargarse de calcular una parte específica de la serie de Fourier sin interferir con el trabajo que realizaban los demás. Lograr esta coordinación permitió notar con claridad cómo la computadora puede aprovechar mejor sus núcleos de procesamiento para terminar las operaciones en un tiempo mucho menor, demostrando la eficiencia que tiene esta técnica en comparación con realizar los cálculos de forma tradicional uno por uno. Por otro lado, la interacción con las herramientas de la plataforma GNU/Linux y la creación del archivo de automatización para la compilación ayudaron a reforzar hábitos de trabajo más ordenados en la programación. Fue indispensable revisar detenidamente el código para asegurar que no se generara ningún tipo de error ni advertencia al momento de compilar. Además, el proceso de guardar los resultados generados por los programas en archivos de datos para después trasladarlos a Excel y construir las gráficas fue una parte muy instructiva. Esto permitió comprobar visualmente que, a pesar de cambiar la forma de programar y utilizar más de cien hilos, la forma de las ondas y las gráficas obtenidas de la serie de Fourier eran idénticas a las de las fases previas, lo que confirmó que los cálculos se realizaron de manera correcta. En conclusión, esta experiencia me dejó un aprendizaje muy valioso sobre la importancia del trabajo concurrente y la distribución de tareas dentro de un software. Comprender cómo funcionan los hilos en C y ver reflejado su rendimiento en un problema concreto de matemáticas aplicadas me brinda una perspectiva mucho más amplia sobre cómo diseñar programas más rápidos y eficientes. Esta práctica me dio herramientas conceptuales y prácticas que sin duda me serán de gran utilidad para enfrentar proyectos más avanzados a lo largo de mi formación académica y profesional.

Capítulo 5. Programación con memoria distribuida: MPI

5.1 Introducción

Contenido pendiente.

5.2 Desarrollo

Contenido pendiente.

5.3 Conclusiones del capítulo

Contenido pendiente.

Capítulo 6. Cómputo en GPU: CUDA

6.1 Introducción

Contenido pendiente.

6.2 Desarrollo

Contenido pendiente.

6.3 Conclusiones del capítulo

Contenido pendiente.

Capítulo 7. Diseño, análisis y caso de estudio integrador

7.1 Introducción

Contenido pendiente.

7.2 Desarrollo

Contenido pendiente.

7.3 Conclusiones del capítulo

Contenido pendiente.

Referencias

[1] J.-B. J. Fourier, *Théorie analytique de la chaleur*. París, Francia: Firmin Didot, 1822.

[2] E. Kreyszig, *Advanced Engineering Mathematics*, 10.ª ed. Jefferson City, MO, USA: John Wiley & Sons, 2011.

[3] A. V. Oppenheim, A. S. Willsky y S. H. Nawab, *Signals and Systems*, 2.ª ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.

[4] «Surcando el Espacio-k para Mejorar la Imagen por Resonancia Magnética,» Radiología / ScienceDirect, visitado 7 de sep. de 2026. dirección: https://www.sciencedirect.com/science/article/abs/pii/S0033833804779527

[5] «A Comprehensive Survey on Magnetic Resonance Image Reconstruction,» arXiv, visitado 7 de sep. de 2026. dirección: https://arxiv.org/pdf/2503.07097

[6] «Surcando el Espacio-k para Mejorar la Imagen por Resonancia Magnética (texto completo),» Universidad Politécnica de Valencia, visitado 7 de sep. de 2026. dirección: http://www.dmoratal.webs.upv.es/papers/2004/2004_Radiologia_2004_46_3_133_150.pdf

[7] Wikipedia contributors. «Fourier-Transform Infrared Spectroscopy,» Wikipedia, visitado 7 de sep. de 2026. dirección: https://en.wikipedia.org/wiki/Fourier-transform_infrared_spectroscopy

[8] Wikipedia contributors. «Espectroscopía Infrarroja,» Wikipedia en español, visitado 7 de sep. de 2026. dirección: https://es.wikipedia.org/wiki/Espectroscop%5C%C3%5C%ADa_infrarroja

[9] «Espectrometría de Infrarrojo con Transformada de Fourier (FT-IR),» Universidad de Guadalajara, CUCEI, visitado 7 de sep. de 2026. dirección: https://itrans.cucei.udg.mx/contenido/espectrometria-de-infrarrojo-con-transformada-de-fourier-ft-ir

[10] «Espectroscopia Infrarroja por Transformada de Fourier y sus Aplicaciones en la Industria,» Tecnosoluciones CR, visitado 7 de sep. de 2026. dirección: https://tecnosolucionescr.net/blog/635-espectroscopia-infrarroja-por-transformada-de-fourier-y-sus-aplicaciones-en-la-industria

[11] «Fourier Analysis and Synthesis,» HyperPhysics, Georgia State University, visitado 7 de sep. de 2026. dirección: http://230nsc1.phy-astr.gsu.edu/hbase/Audio/fourier.html

[12] «The Applications of Fourier Series Harmonics in Musical Tones,» Journal of Pure & Applied Sciences, Sebha University, visitado 7 de sep. de 2026. dirección: https://www.sebhau.edu.ly/journal/jopas/article/download/3590/1474

[13] «Analyzing Musical Tones with Fourier Transformation,» ResearchGate, visitado 7 de sep. de 2026. dirección: https://www.researchgate.net/publication/381679515_Analyzing_musical_tones_with_fourier_transformation

[14] «Practical Applications of the Fourier Analysis to Identify Pitches and Synthesize Sounds in Music,» Journal of Emerging Investigators, visitado 7 de sep. de 2026. dirección: https://emerginginvestigators.org/articles/24-155/pdf

[15] «Fourier at the Heart of Computer Music: From Harmonic Sounds to Spectral Synthesis,» Comptes Rendus Physique, Académie des Sciences, visitado 7 de sep. de 2026. dirección: https://comptes-rendus.academie-sciences.fr/physique/articles/10.1016/j.crhy.2019.07.005/

[16] «Global Lattice Models of a Rotating Planet — Seismograph Measurements and Fast Fourier Transforms,» arXiv, visitado 7 de sep. de 2026. dirección: https://arxiv.org/pdf/2508.16630

[17] «Fourier Amplitude Spectrum — An Overview,» ScienceDirect Topics, visitado 7 de sep. de 2026. dirección: https://www.sciencedirect.com/topics/engineering/fourier-amplitude-spectrum

[18] «Feasibility Study of Signal Processing Techniques for Vibration-Based Structural Health Monitoring in Residential Buildings,» PMC, National Library of Medicine, visitado 7 de sep. de 2026. dirección: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11991024/

[19] «1.13: Quantum Mechanics and the Fourier Transform,» LibreTexts (Chemistry), visitado 7 de sep. de 2026. dirección: https://chem.libretexts.org/Bookshelves/Physical_and_Theoretical_Chemistry_Textbook_Maps/Quantum_Tutorials_(Rioux)/01:_Quantum_Fundamentals/1.13:_Quantum_Mechanics_and_the_Fourier_Transform

[20] «Role of Fourier Transform in Quantum Mechanics: Applications and Implications,» ResearchGate, visitado 7 de sep. de 2026. dirección: https://www.researchgate.net/publication/384272588_Role_of_Fourier_transform_in_quantum_mechanics_applications_and_implications

[21] «1.17: Quantum Mechanics and the Fourier Transform,» LibreTexts (Chemistry), visitado 7 de sep. de 2026. dirección: https://chem.libretexts.org/Bookshelves/Physical_and_Theoretical_Chemistry_Textbook_Maps/Quantum_Tutorials_(Rioux)/01:_Quantum_Fundamentals/1.17:_Quantum_Mechanics_and_the_Fourier_Transform

[22] «Mind the Step: On the Frequency-Domain Analysis of Gravitational-Wave Memory Waveforms,» arXiv, visitado 7 de sep. de 2026. dirección: https://arxiv.org/pdf/2406.16636

[23] «Fourier Analysis in Signal Processing,» Journal of Applied Engineering y Applied Sciences, visitado 7 de sep. de 2026. dirección: https://rjsaonline.org/index.php/JAEAS/article/view/148

[24] J. Howard. «Fourier Transforms in Communication,» visitado 7 de sep. de 2026. dirección: https://jameshoward.us/2024/01/20/fourier-transforms-in-communication

[25] «Implementation of Fast Fourier Transform in Image Processing, Signal Processing and Acoustics,» IEEE Xplore, visitado 7 de sep. de 2026. dirección: https://ieeexplore.ieee.org/document/10854259/

[26] «Fast Fourier Transform - Algorithms and Applications,» ACM Digital Library, visitado 7 de sep. de 2026. dirección: https://dl.acm.org/doi/10.5555/1941838

[27] Wikipedia contributors. «IEEE Fourier Award for Signal Processing,» Wikipedia, visitado 7 de sep. de 2026. dirección: https://en.wikipedia.org/wiki/IEEE_Fourier_Award_for_Signal_Processing

[28] K. Bryan. «Picture Perfect: The Mathematics of JPEG Compression,» Rose-Hulman Institute of Technology, visitado 7 de sep. de 2026. dirección: https://www.rose-hulman.edu/~bryan/invprobs/jpegtalk2.pdf

[29] «Discrete Cosine Transform in JPEG Compression,» arXiv, visitado 7 de sep. de 2026. dirección: https://arxiv.org/pdf/2102.06968

[30] «Discrete Cosine Transform — An Overview,» ScienceDirect Topics, visitado 7 de sep. de 2026. dirección: https://www.sciencedirect.com/topics/computer-science/discrete-cosine-transform

[31] A. V. Oppenheim y R. W. Schafer, *Discrete-Time Signal Processing*, 3.ª ed. Pearson, 2009.

[32] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 4.ª ed. McGraw-Hill, 2011.

[33] S. S. Rao, *Mechanical Vibrations*, 6.ª ed. Pearson, 2016.

[34] Z. Zhang et al., «Near-energy-free photonic Fourier transformation for convolution operation acceleration,» *Advanced Photonics*, vol. 7, n.º 5, 2023. visitado 7 de sep. de 2026. dirección: https://www.spiedigitallibrary.org/journals/advanced-photonics/volume-7/issue-05/056007/

[35] G. Konidaris, S. Osentoski y P. Thomas, «Value Function Approximation in Reinforcement Learning Using the Fourier Basis,» en *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, 2011.

[36] M. Clerc y J. Kennedy, «The Particle Swarm - Explosion, Stability, and Convergence in a Multidimensional Complex Space,» *IEEE Transactions on Evolutionary Computation*, vol. 6, n.º 1, págs. 58-73, 2002.

[37] Wikipedia contributors. «Cooley–Tukey FFT Algorithm,» visitado 7 de sep. de 2026. dirección: https://en.wikipedia.org/wiki/Cooley-Tukey_FFT_algorithm

[38] M. Pippig, «PFFT - A Massively Parallel Extension of FFTW,» TU Chemnitz, inf. téc., 2012. visitado 7 de sep. de 2026. dirección: https://www.tu-chemnitz.de/mathematik/preprint/2012/PREPRINT_06.pdf

[39] S. Tomov et al., *heFFTe: Highly Efficient FFT for Exascale*, ResearchGate, 2020. visitado 7 de sep. de 2026. dirección: https://www.researchgate.net/publication/342293120_heFFTe_Highly_Efficient_FFT_for_Exascale

[40] P. F. Nowak et al., *Wafer-Scale Fast Fourier Transforms*, arXiv, 2022. visitado 7 de sep. de 2026. dirección: https://arxiv.org/pdf/2209.15040

[41] J. Walkenbach, *Excel 2016 Bible*. Indianápolis, IN: John Wiley & Sons, 2015.

[42] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Hoboken, NJ: Wiley, 2018.

[43] W. R. Stevens y S. A. Rago, *Advanced Programming in the UNIX Environment*, 3.ª ed. Addison-Wesley, 2013.

[44] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA: No Starch Press, 2010.

[45] A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.ª ed. Boston, MA: Pearson, 2014.

[46] «Apple MacBook Air "M5" 10 CPU/10 GPU 13" Specs,» EveryMac.com, visitado 30 de sep. de 2026. dirección: `https://everymac.com/systems/apple/macbook-air/specs/macbook-air-m5-10-core-cpu-10-core-gpu-13-2026-specs.html`

[47] Apple Inc. «MacBook Air: Technical Specifications,» Apple, visitado 30 de sep. de 2026. dirección: `https://www.apple.com/macbook-air/specs/`